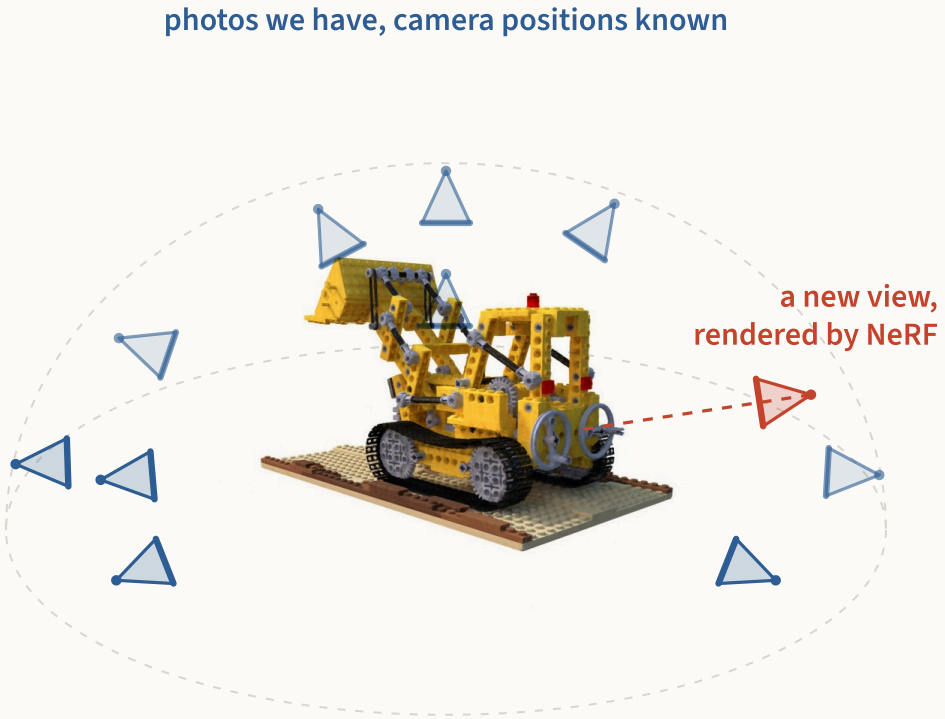
Neural Radiance Fields

NeRF learns one scene from a set of photos and can then draw it from any new viewpoint.



Lego scene rendered by NeRF (Mildenhall et al. 2020); cameras drawn by me.

Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, Ren Ng
UC Berkeley, Google Research and UC San Diego. ECCV 2020.
With Instant-NGP (Müller et al., SIGGRAPH 2022) and 3D Gaussian splatting (Kerbl et al., SIGGRAPH 2023).

Presented by **Ertugrul Senturk**
November 2026

New views of a scene from its photos

100 photos of one scene, with each camera's position and direction known

NeRF

one network, trained on these photos only

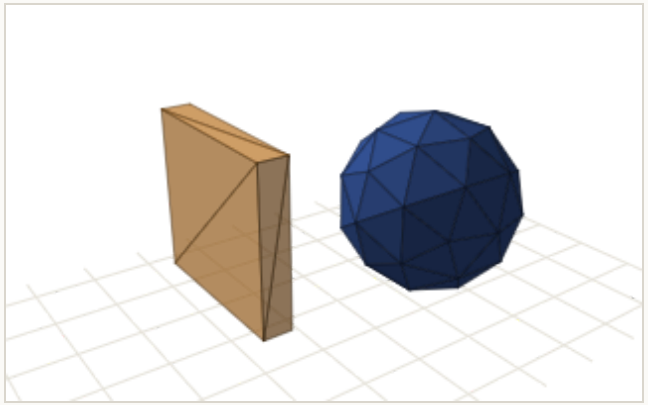
Two new views rendered by NeRF

We start with ordinary photos of one scene, and for each photo we know where the camera stood and which way it pointed. We want the photo a camera would take from a position nobody photographed, with the right perspective, the right hidden parts and the right reflections.

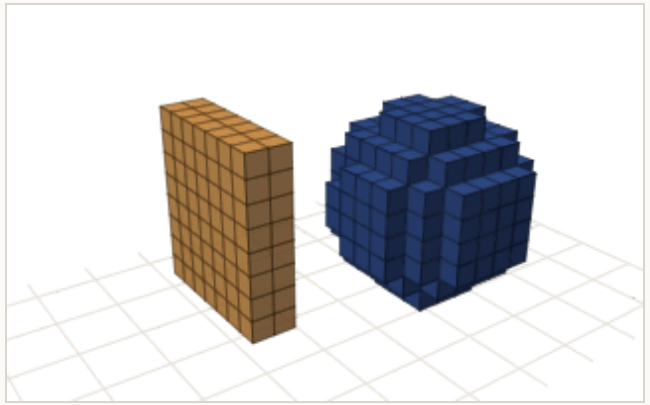
This is called view synthesis. NeRF trains one network per scene, from scratch, on that scene's photos and their camera positions. No 3D shape or depth is given. After training, the whole scene is inside that one network.

Mildenhall et al. 2020, Fig. 1: the synthetic Drums scene, 100 input views on a hemisphere, two rendered new views.

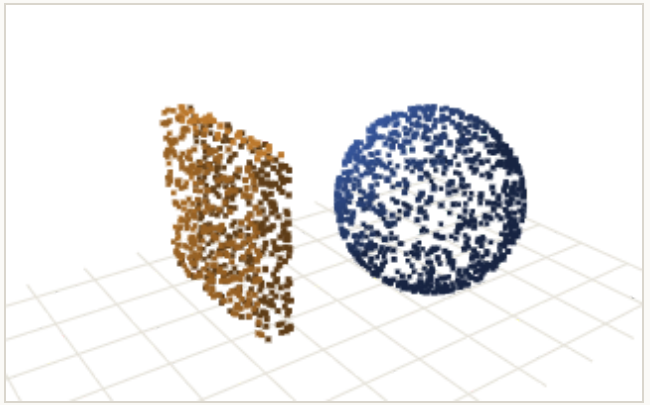
How a 3D scene is usually stored



A mesh of triangles



A grid of small cubes



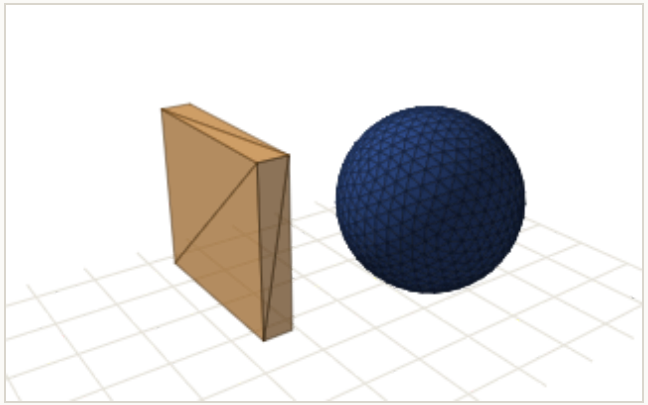
A cloud of points

	A mesh of triangles	A grid of small cubes	A cloud of points
Memory at this detail	92 triangles, 50 corners: 2 KB	16 per side, 4,096 cubes: 16 KB	1,600 points: 24 KB
Good at	Small and fast to draw; how video games and 3D printing describe objects.	Easy to fill in bit by bit, and it can hold see-through things like smoke.	What a laser scanner or photo matching gives you directly, as on my globe later.
Problem	Hard to get from photos, and only hard surfaces: no smoke, no windows, no fur.	Halving the cube size needs eight times the memory, mostly for empty air.	Holes between the points and no real surface, so it falls apart up close.

Shown at **medium** detail; the next page shows very high.

A toy scene I use on the next slides: orange tinted glass and a solid blue ball. Memory: 12 bytes per corner and per triangle, 4 per cube of the whole grid, 15 per point. Neural Volumes, a baseline in the paper, predicts a 128^3 grid; the photo matching is COLMAP.

How a 3D scene is usually stored

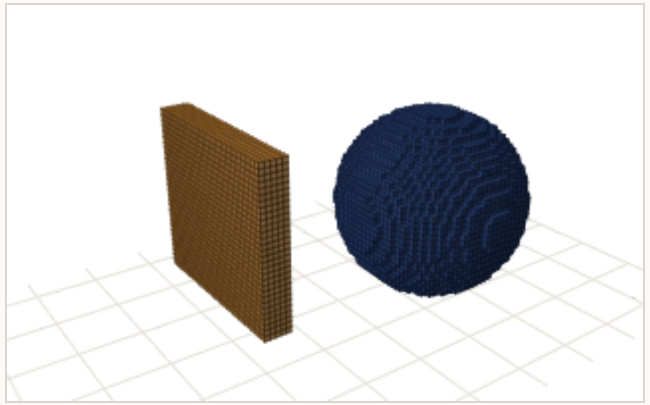


A mesh of triangles

Memory at this detail: 1,292 triangles, 650 corners: 23 KB

Good at: Small and fast to draw; how video games and 3D printing describe objects.

Problem: Hard to get from photos, and only hard surfaces: no smoke, no windows, no fur.

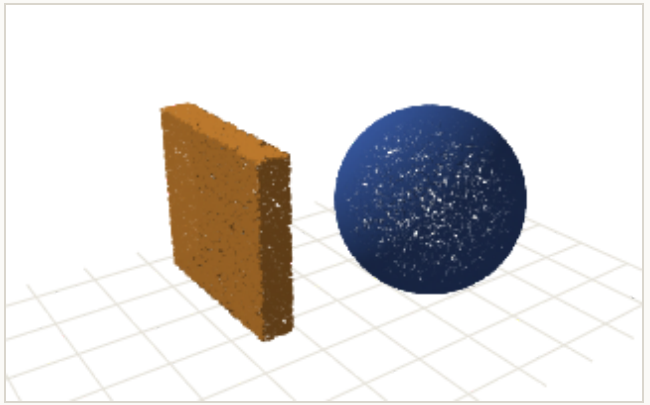


A grid of small cubes

Memory at this detail: 64 per side, 262,144 cubes: 1.0 MB

Good at: Easy to fill in bit by bit, and it can hold see-through things like smoke.

Problem: Halving the cube size needs eight times the memory, mostly for empty air.



A cloud of points

Memory at this detail: 25,600 points: 384 KB

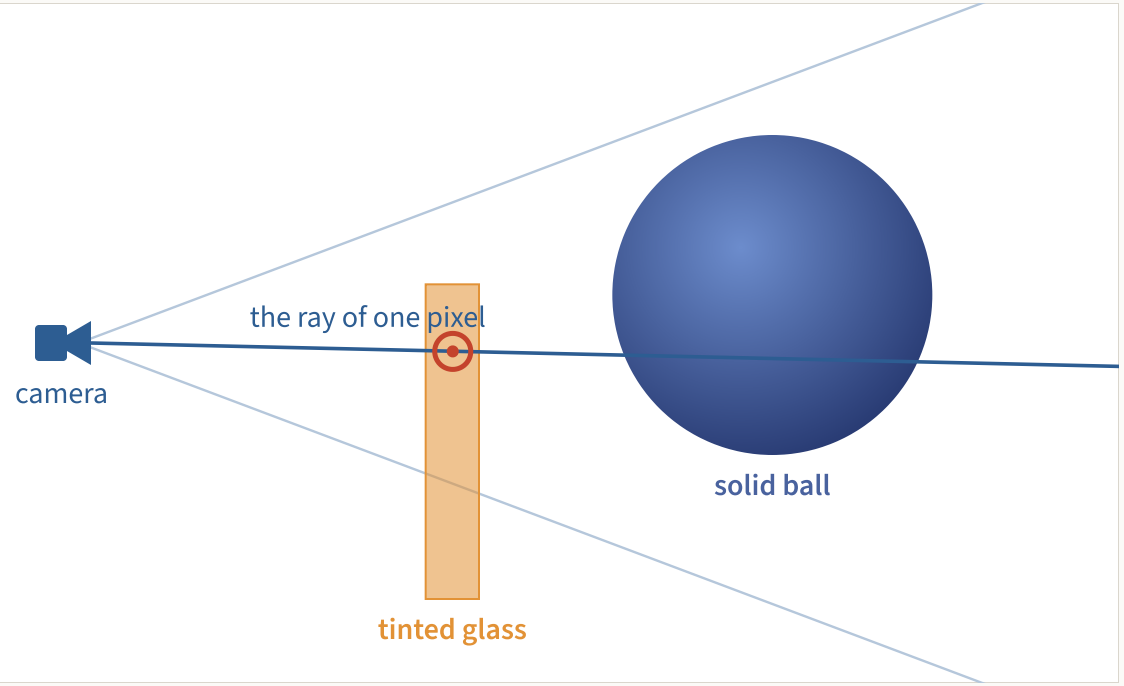
Good at: What a laser scanner or photo matching gives you directly, as on my globe later.

Problem: Holes between the points and no real surface, so it falls apart up close.

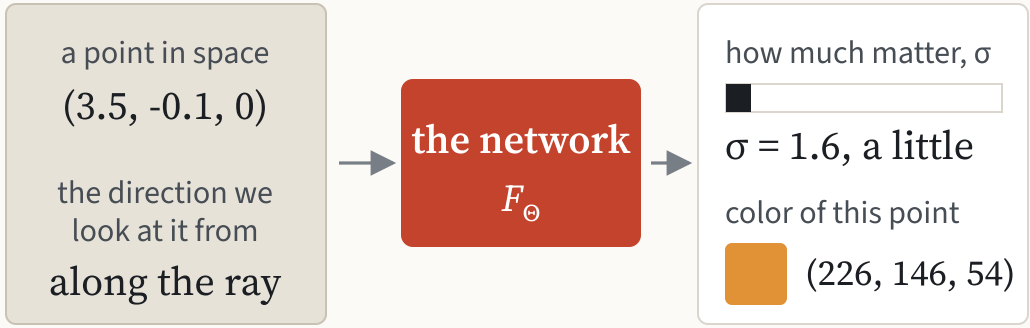
Shown at **very high** detail.

A toy scene I use on the next slides: orange tinted glass and a solid blue ball. Memory: 12 bytes per corner and per triangle, 4 per cube of the whole grid, 15 per point. Neural Volumes, a baseline in the paper, predicts a 128^3 grid; the photo matching is COLMAP.

What the network takes in and gives back



The toy scene from above, with the camera and the ray of one of its pixels. The red ring marks the point on that ray we ask the network about; the arrow key moves it. Made-up density values, 0 to 18.



Step 1 of 3

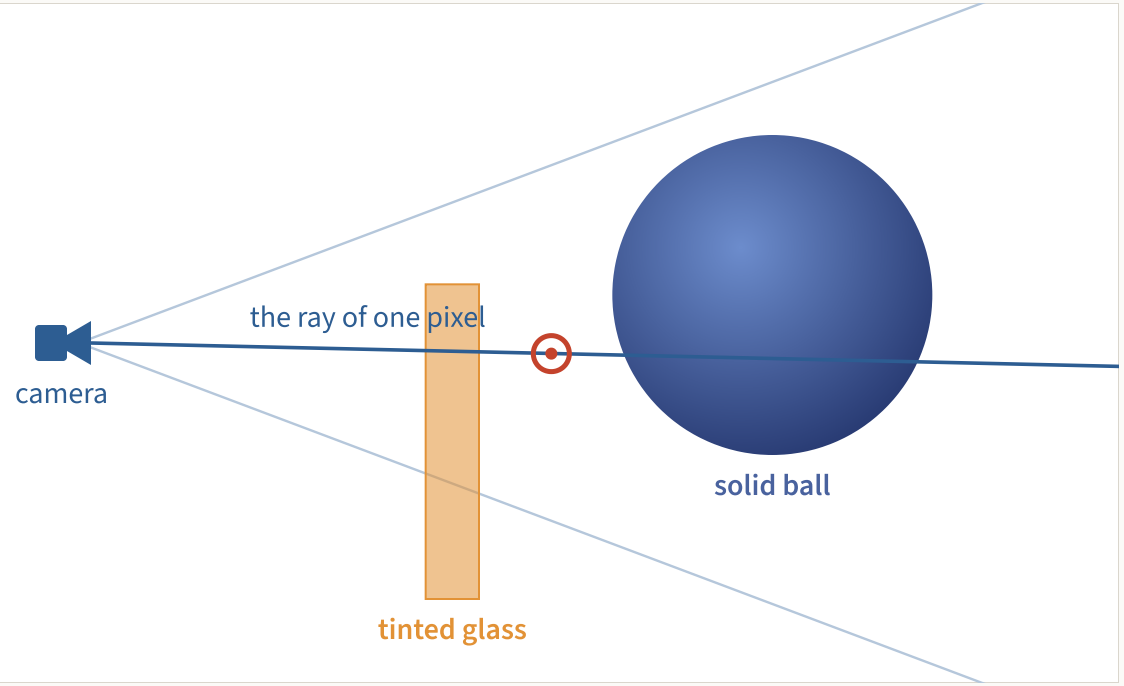
First stop on the ray, a point inside the tinted glass: a small density, so there is something here, but most light passes through it, and its color is the orange of the glass.

The whole scene is one function. Give it a point and the direction you look at it from, and it answers two things: the density σ, a number for how much matter sits at that point, and the color that point sends back along the ray.

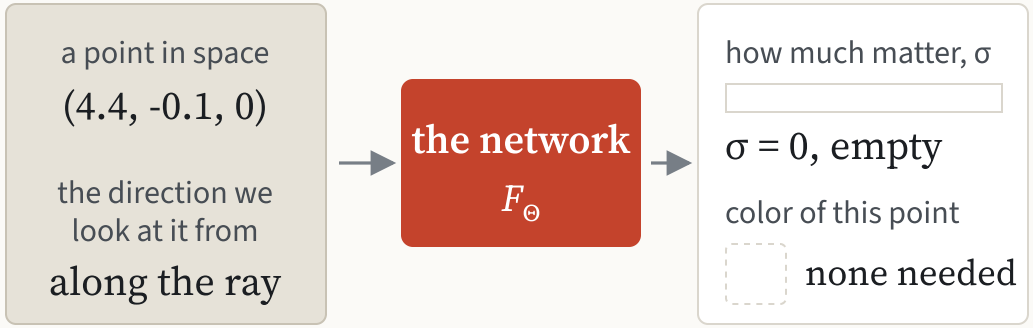
Density depends only on the position. Color may also change with the direction, which is how a shiny surface looks different from two sides.

The density values are made up for the toy scene; NeRF learns them. Mildenhall et al. 2020, Sec. 3: $F_\Theta : (x, d) \rightarrow (c, \sigma)$, with the direction d given as two angles (θ, ϕ).

What the network takes in and gives back



The toy scene from above, with the camera and the ray of one of its pixels. The red ring marks the point on that ray we ask the network about; the arrow key moves it. Made-up density values, 0 to 18.



Step 2 of 3

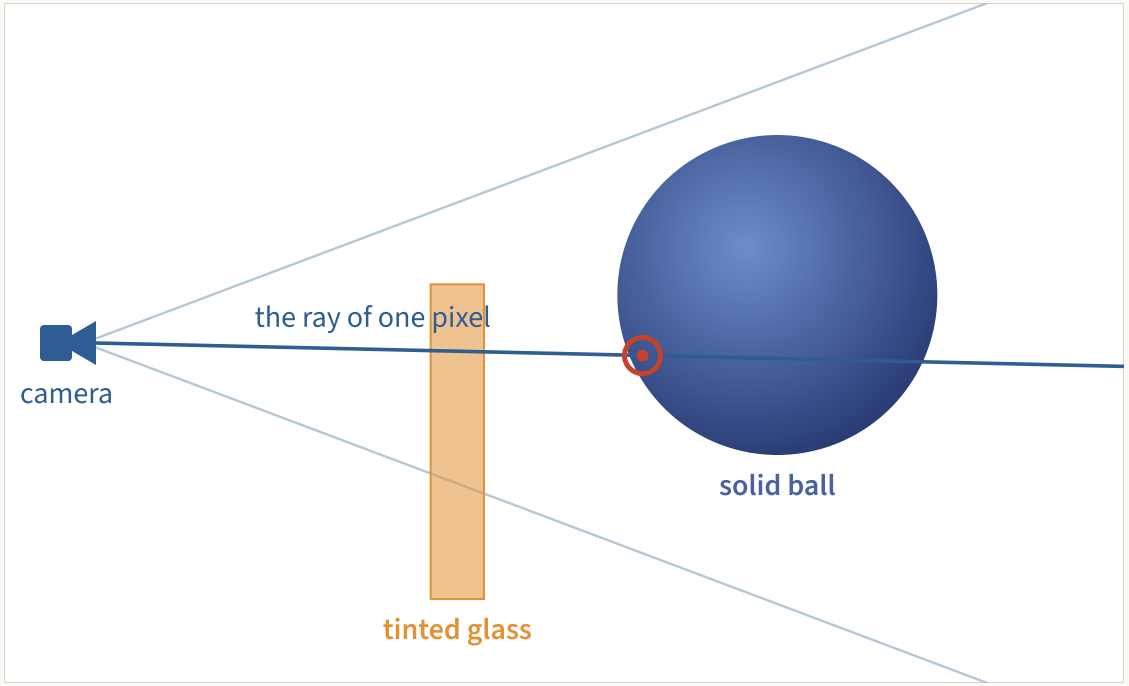
Further along, a point in the air between the glass and the ball: density 0, nothing here, so its color does not matter. Most points along any ray are like this one.

The whole scene is one function. Give it a point and the direction you look at it from, and it answers two things: the density σ, a number for how much matter sits at that point, and the color that point sends back along the ray.

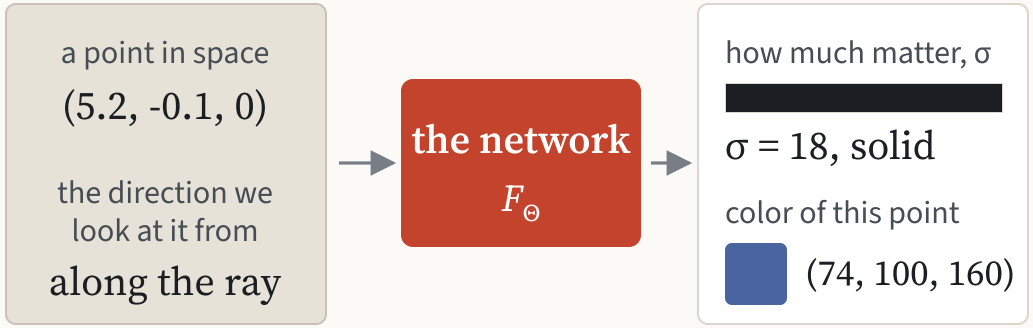
Density depends only on the position. Color may also change with the direction, which is how a shiny surface looks different from two sides.

The density values are made up for the toy scene; NeRF learns them. Mildenhall et al. 2020, Sec. 3: $F_\Theta : (x, d) \rightarrow (c, \sigma)$, with the direction d given as two angles (θ, ϕ).

What the network takes in and gives back



The toy scene from above, with the camera and the ray of one of its pixels. The red ring marks the point on that ray we ask the network about; the arrow key moves it. Made-up density values, 0 to 18.



Step 3 of 3

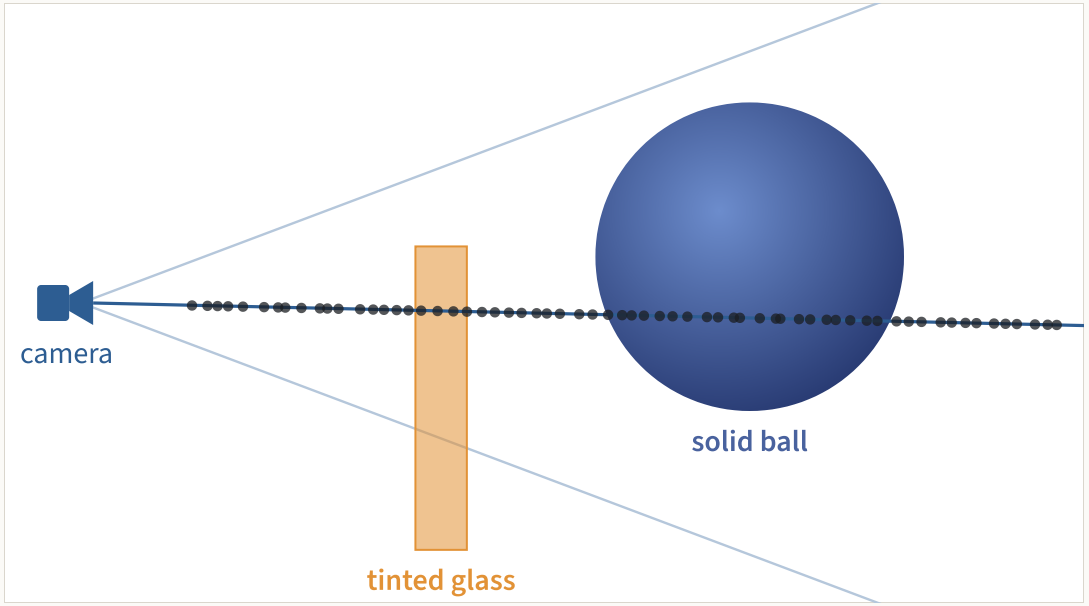
Then a point just inside the solid ball: a high density, and the ball's own blue. This is where most of the pixel's color will come from.

The whole scene is one function. Give it a point and the direction you look at it from, and it answers two things: the density σ, a number for how much matter sits at that point, and the color that point sends back along the ray.

Density depends only on the position. Color may also change with the direction, which is how a shiny surface looks different from two sides.

The density values are made up for the toy scene; NeRF learns them. Mildenhall et al. 2020, Sec. 3: $F_\Theta : (x, d) \rightarrow (c, \sigma)$, with the direction d given as two angles (θ, ϕ).

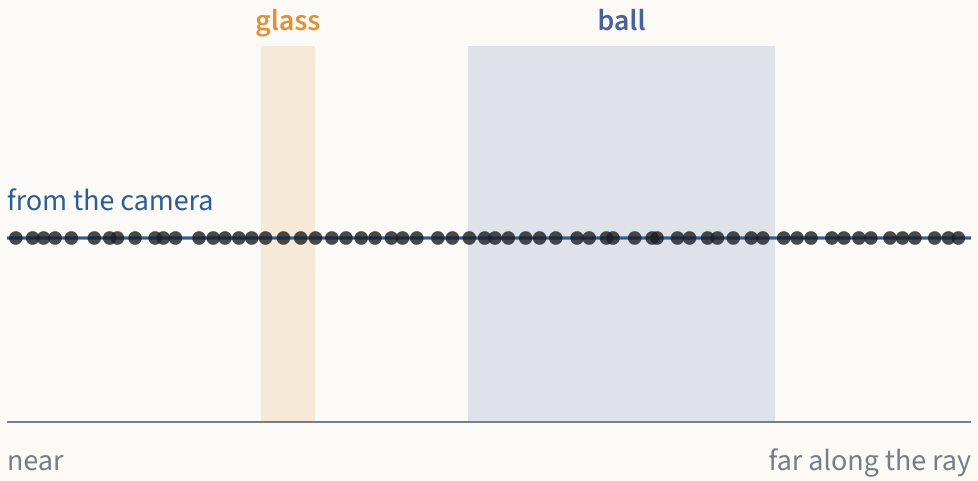
Rendering one pixel



From above, as before. The orange pane is tinted glass.

The camera's image row, each pixel from the same sum **pixel 23 of 48**

The ray, straightened out, with its 64 points



$$C = \Sigma_i \, T_i \, (1 - e^{-\sigma_i \delta_i}) \, c_i$$

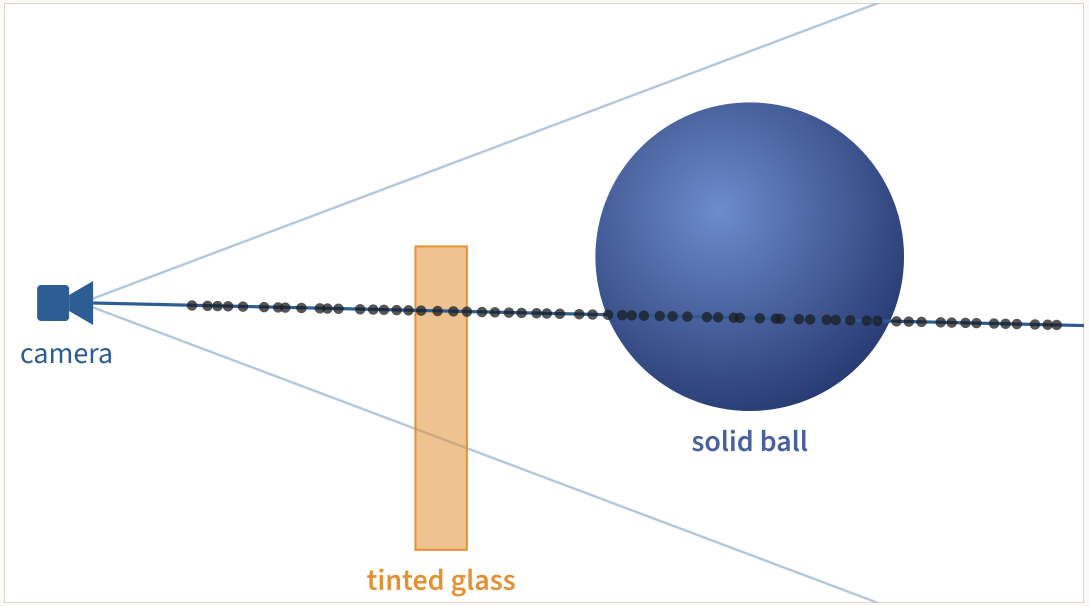
light still getting through × opacity of the point × its color, summed over the 64 points

Step 1 of 4

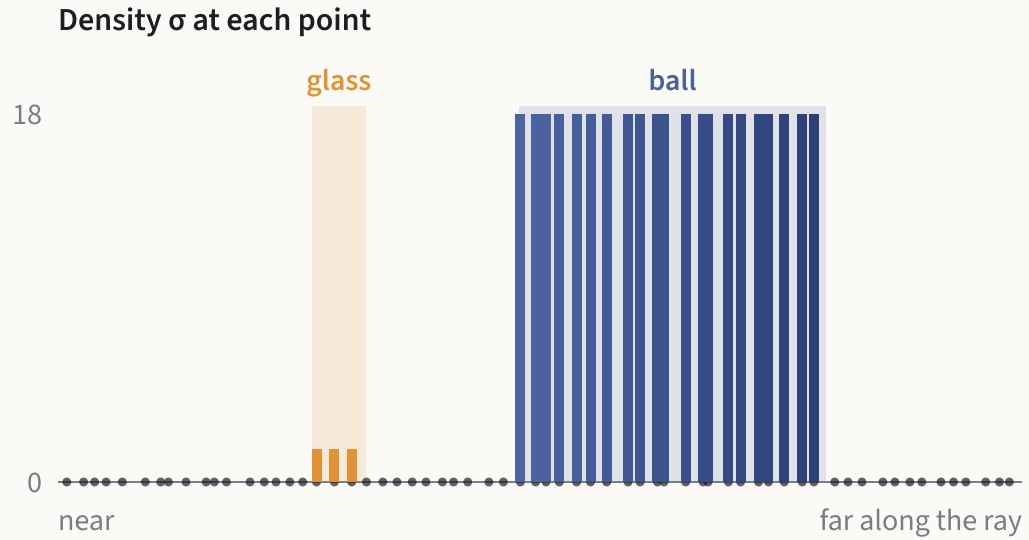
Each pixel looks along one ray. NeRF asks the network for density and color at 64 points along it, spread evenly with a little jitter. The network never returns a pixel: the pixel is a weighted mix of those 64 answers, built up in the next steps.

Mildenhall et al. 2020, Eq. 3: T_i is the light still getting through at point i, $1 - e^{-\sigma_i \delta_i}$ its opacity, δ_i the gap to the next point and c_i its color. Toy scene and made-up densities as on the previous slide, 64 points per ray; the image row is every pixel of the camera, rendered with the same sum.

Rendering one pixel



From above, as before. The orange pane is tinted glass.



The camera's image row, each pixel from the same sum **pixel 23 of 48**

$$C = \Sigma_i \, T_i \, (1 - e^{-\sigma_i \delta_i}) \, c_i$$

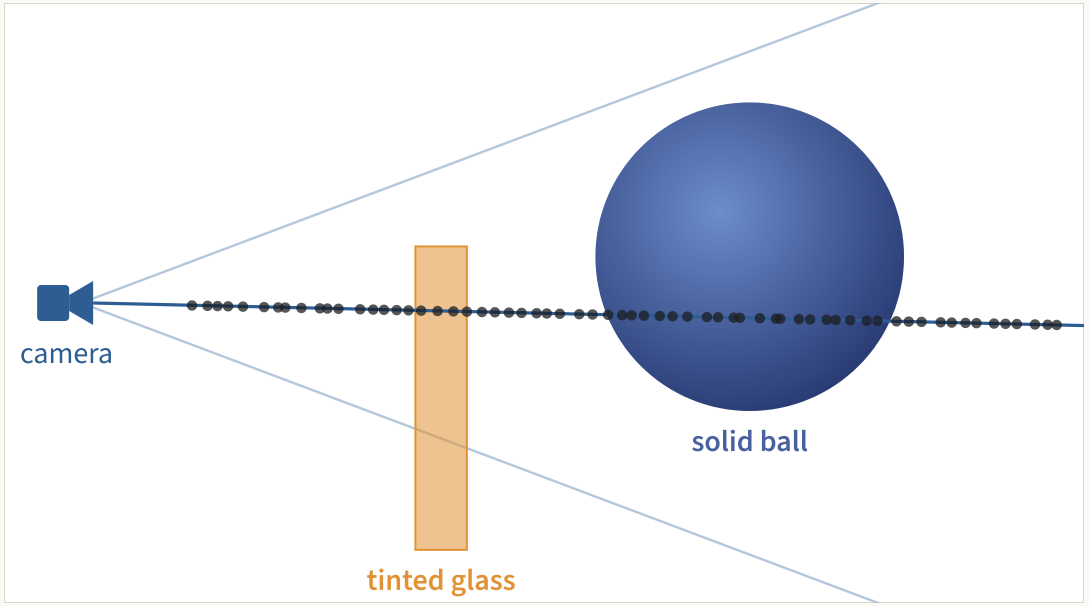
light still getting through × opacity of the point × its color, summed over the 64 points

Step 2 of 4

The network returns a density at each point: zero in empty air, small inside the glass pane, which only tints what is behind it, and high inside the ball.

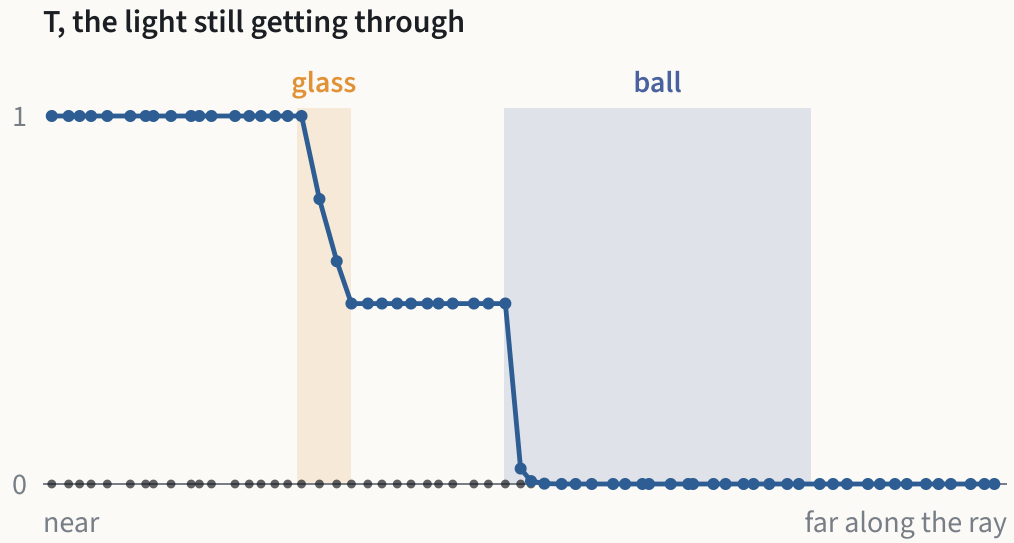
Mildenhall et al. 2020, Eq. 3: T_i is the light still getting through at point i, $1 - e^{-\sigma_i \delta_i}$ its opacity, δ_i the gap to the next point and c_i its color. Toy scene and made-up densities as on the previous slide, 64 points per ray; the image row is every pixel of the camera, rendered with the same sum.

Rendering one pixel



From above, as before. The orange pane is tinted glass.

The camera's image row, each pixel from the same sum **pixel 23 of 48**



$$C = \Sigma_i \, T_i \, (1 - e^{-\sigma_i \delta_i}) \, c_i$$

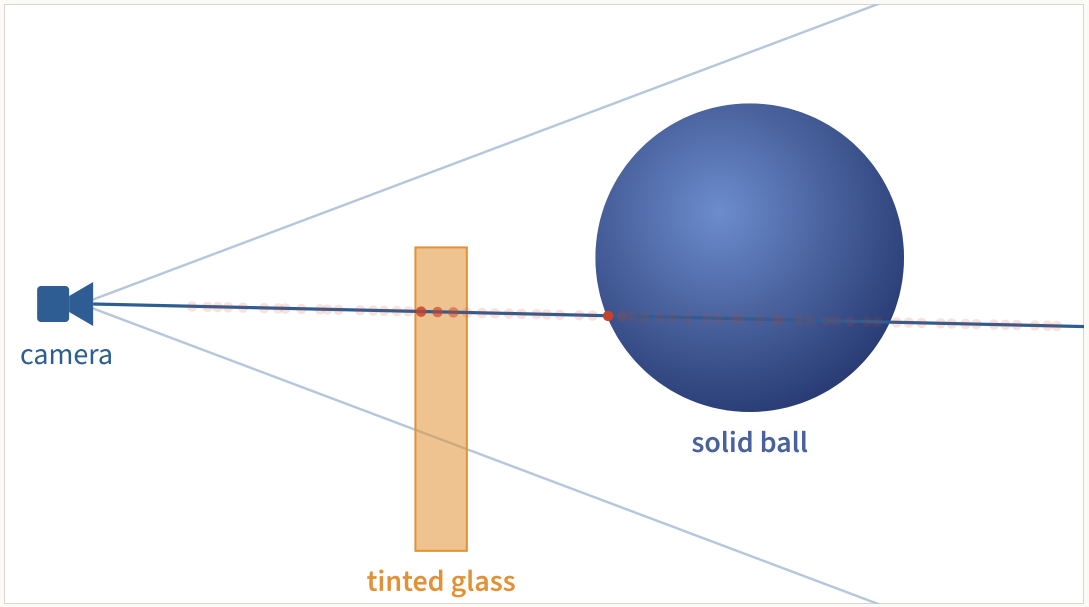
light still getting through × opacity of the point × its color, summed over the 64 points

Step 3 of 4

T is the share of a point's light that still gets through to the camera. It starts at 1, drops where the ray passes the pane, and falls to zero at the front of the ball.

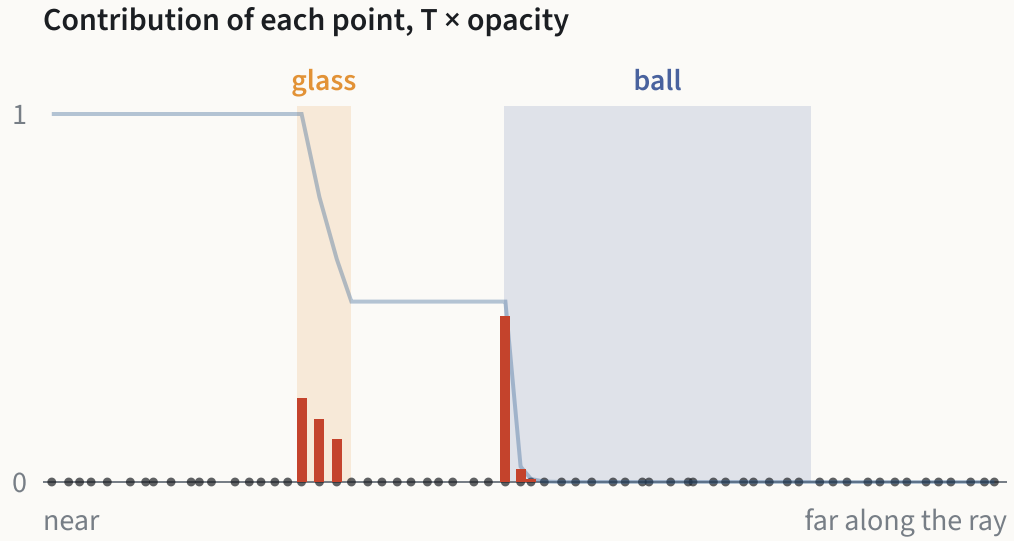
Mildenhall et al. 2020, Eq. 3: T_i is the light still getting through at point i, $1 - e^{-\sigma_i \delta_i}$ its opacity, δ_i the gap to the next point and c_i its color. Toy scene and made-up densities as on the previous slide, 64 points per ray; the image row is every pixel of the camera, rendered with the same sum.

Rendering one pixel



From above, as before. The orange pane is tinted glass.

The camera's image row, each pixel from the same sum **pixel 23 of 48**



$C = \Sigma_i \, T_i \, (1 - e^{-\sigma_i \delta_i}) \, c_i$ this pixel

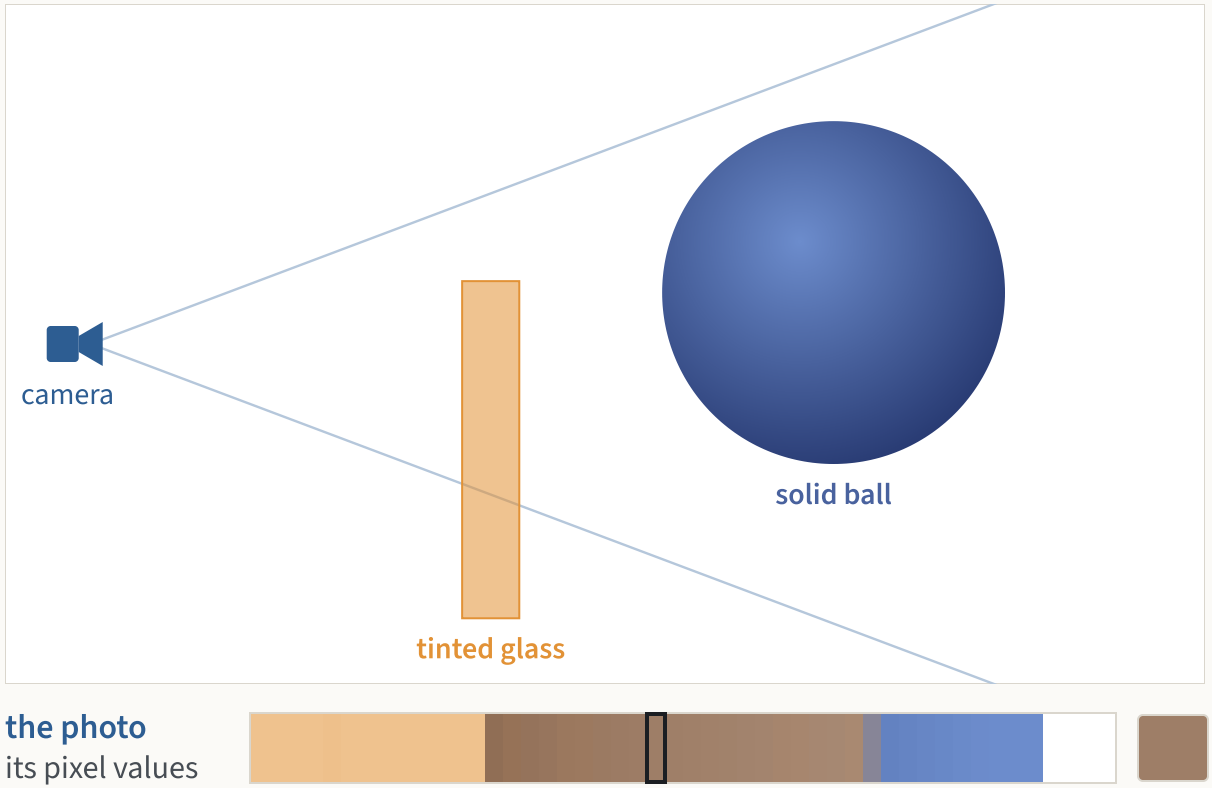
light still getting through × opacity of the point × its color, summed over the 64 points

Step 4 of 4

Each point contributes T times its opacity, and the pixel is the sum of these contributions times the point colors. This ray crosses the glass, then hits the ball, so the pixel is the ball's blue with an orange tint.

Mildenhall et al. 2020, Eq. 3: T_i is the light still getting through at point i, $1 - e^{-\sigma_i \delta_i}$ its opacity, δ_i the gap to the next point and c_i its color. Toy scene and made-up densities as on the previous slide, 64 points per ray; the image row is every pixel of the camera, rendered with the same sum.

How NeRF is trained

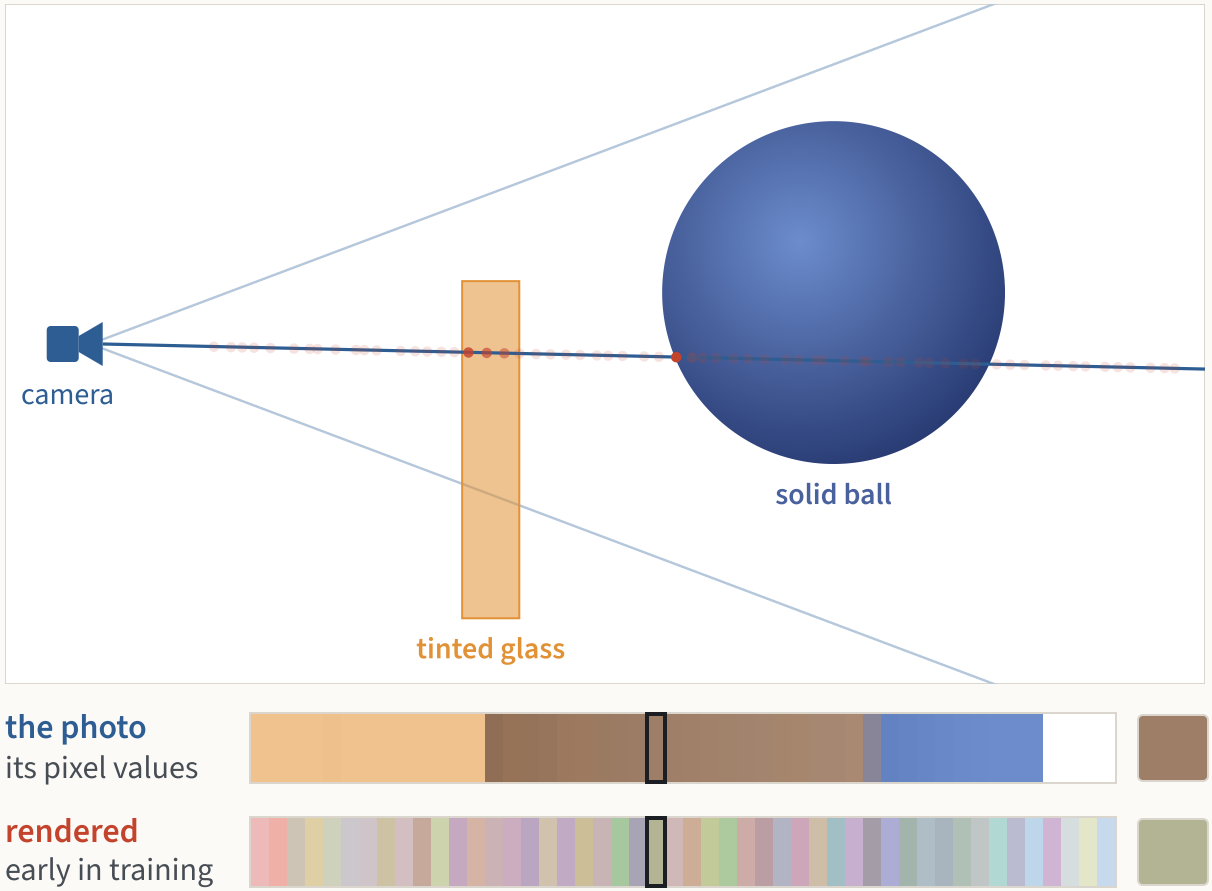


Step 1 of 4

Training needs only the photos and the position of the camera that took each one. The photo's pixel values are the targets. Here is one camera's photo, a row of 48 pixels for our 2D scene, and one chosen pixel whose actual color we read straight from the photo.

Mildenhall et al. 2020, Eq. 6 (written there as a sum over the rays of the batch, with a second term for the fine network introduced later; a mean differs from the sum only by a constant) and Sec. 5.3: 4,096 rays per step, 100,000 to 300,000 steps, one to two days on one V100. Toy scene as before; the photo of a 2D scene is one row of pixels, computed with the rendering sum. The "early in training" row is a blurred, noisy copy of it, standing in for an untrained network.

How NeRF is trained

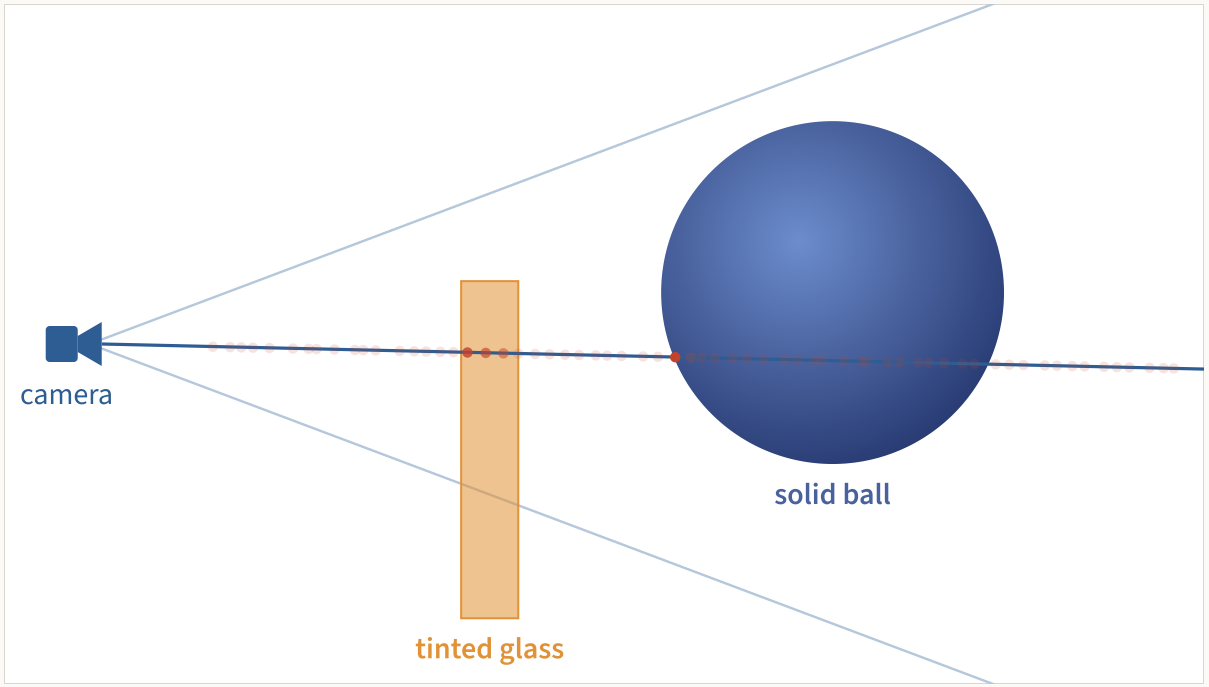


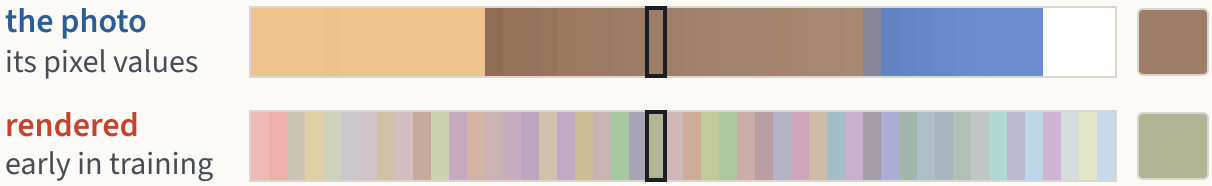
Step 2 of 4

The camera pose tells us which ray that pixel looks along. The current network renders that ray with the sum from the previous slide and gives its own color for the pixel. Early in training the weights are random, so the rendered row is a blur and this pixel comes out wrong.

Mildenhall et al. 2020, Eq. 6 (written there as a sum over the rays of the batch, with a second term for the fine network introduced later; a mean differs from the sum only by a constant) and Sec. 5.3: 4,096 rays per step, 100,000 to 300,000 steps, one to two days on one V100. Toy scene as before; the photo of a 2D scene is one row of pixels, computed with the rendering sum. The "early in training" row is a blurred, noisy copy of it, standing in for an untrained network.

How NeRF is trained





Step 3 of 4

We use the mean squared error between the rendered color and the actual pixel color as the loss. It flows back through the rendering sum into the weights, which move a little so that the rendered color gets closer to the photo.

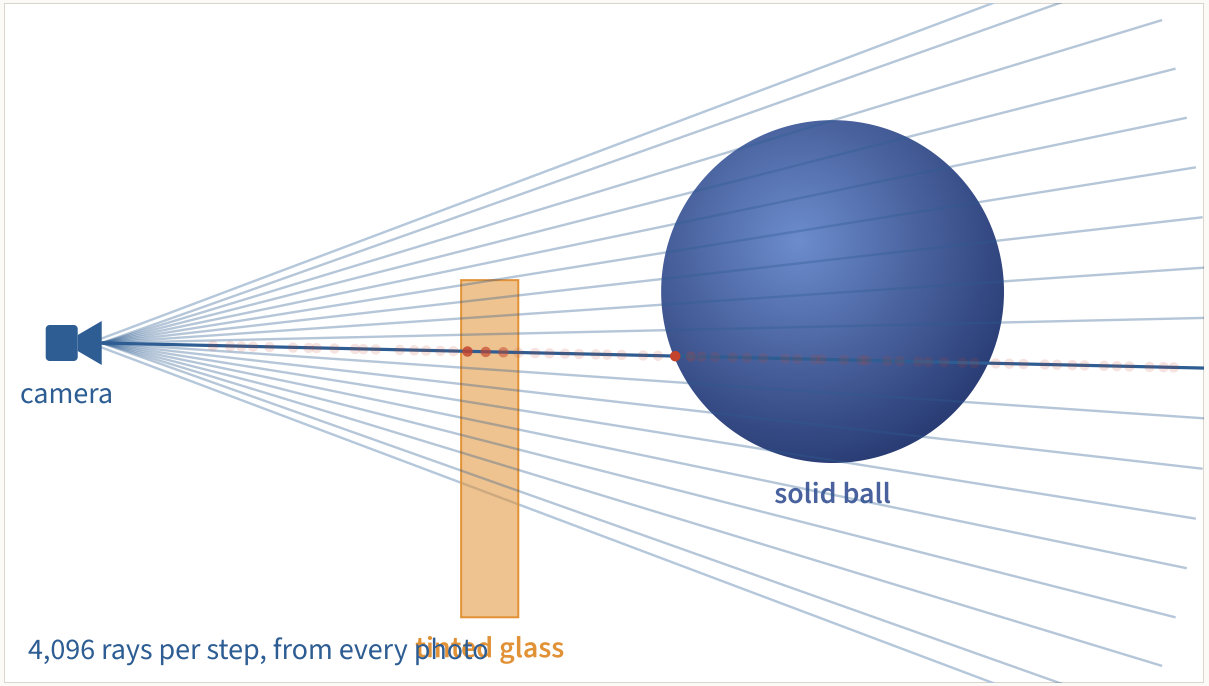
$$\text{loss} = \text{mean over the 4,096 rays of } \left(C_{\text{rendered}} - C_{\text{photo}} \right)^2$$

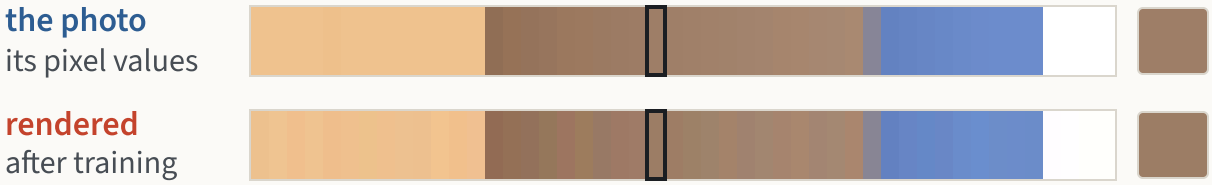
C_{rendered} is the color the network rendered along the pixel's ray, C_{photo} the actual pixel value in the photo; the squared error is averaged over the step's 4,096 rays.

for this pixel: (rendered − photo)^2 = **0.08**

Mildenhall et al. 2020, Eq. 6 (written there as a sum over the rays of the batch, with a second term for the fine network introduced later; a mean differs from the sum only by a constant) and Sec. 5.3: 4,096 rays per step, 100,000 to 300,000 steps, one to two days on one V100. Toy scene as before; the photo of a 2D scene is one row of pixels, computed with the rendering sum. The "early in training" row is a blurred, noisy copy of it, standing in for an untrained network.

How NeRF is trained





Step 4 of 4

Each step does this for 4,096 random pixels drawn from all the photos at once, and training repeats it 100,000 to 300,000 times, one to two days on one V100 GPU. By then the rendered row matches the photo and the loss is near zero.

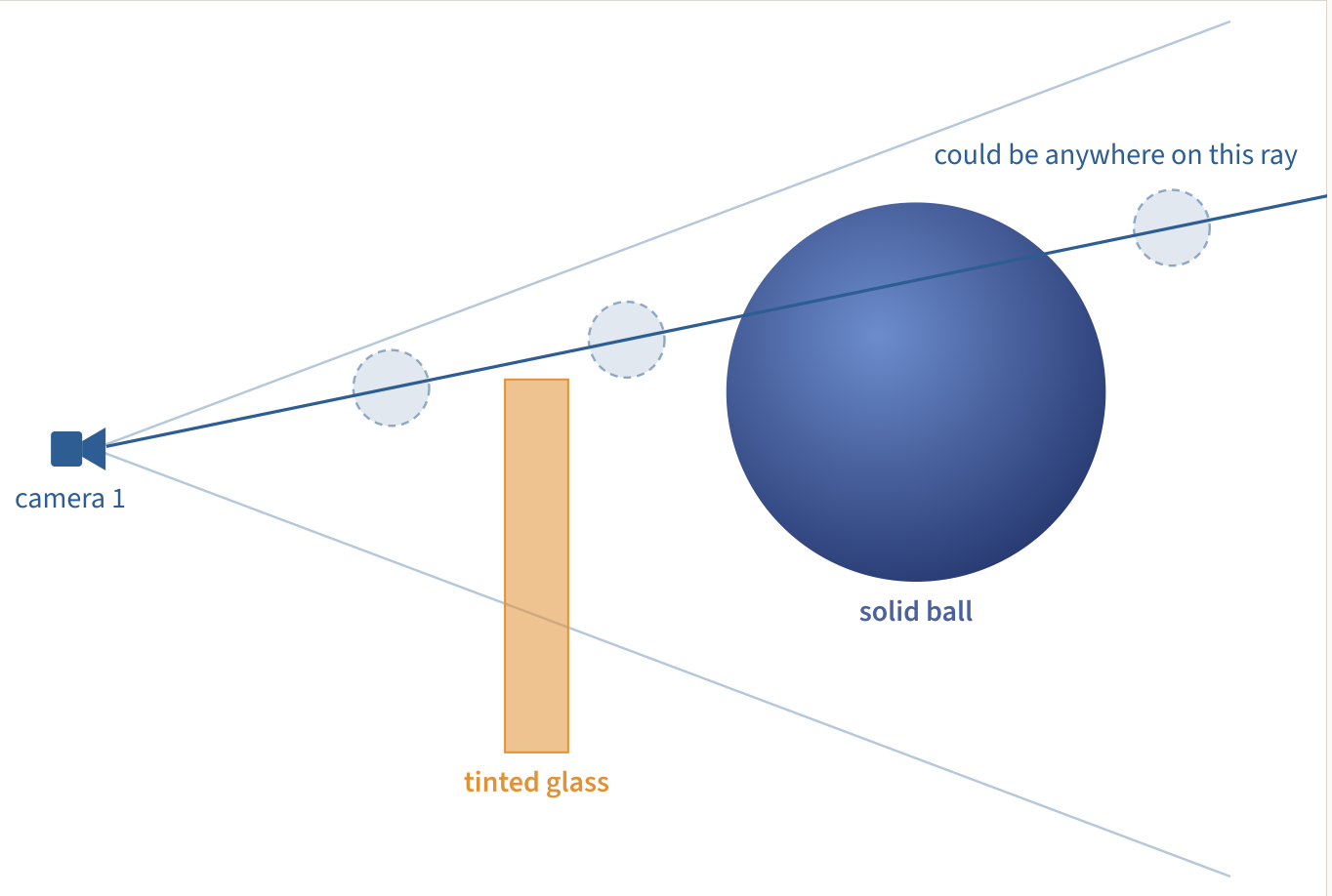
$$\text{loss} = \text{mean over the 4,096 rays of } \left(C_{\text{rendered}} - C_{\text{photo}} \right)^2$$

C_{rendered} is the color the network rendered along the pixel's ray, C_{photo} the actual pixel value in the photo; the squared error is averaged over the step's 4,096 rays.

for this pixel: (rendered − photo)² = **0.00**

Mildenhall et al. 2020, Eq. 6 (written there as a sum over the rays of the batch, with a second term for the fine network introduced later; a mean differs from the sum only by a constant) and Sec. 5.3: 4,096 rays per step, 100,000 to 300,000 steps, one to two days on one V100. Toy scene as before; the photo of a 2D scene is one row of pixels, computed with the rendering sum. The "early in training" row is a blurred, noisy copy of it, standing in for an untrained network.

Where the 3D shape comes from

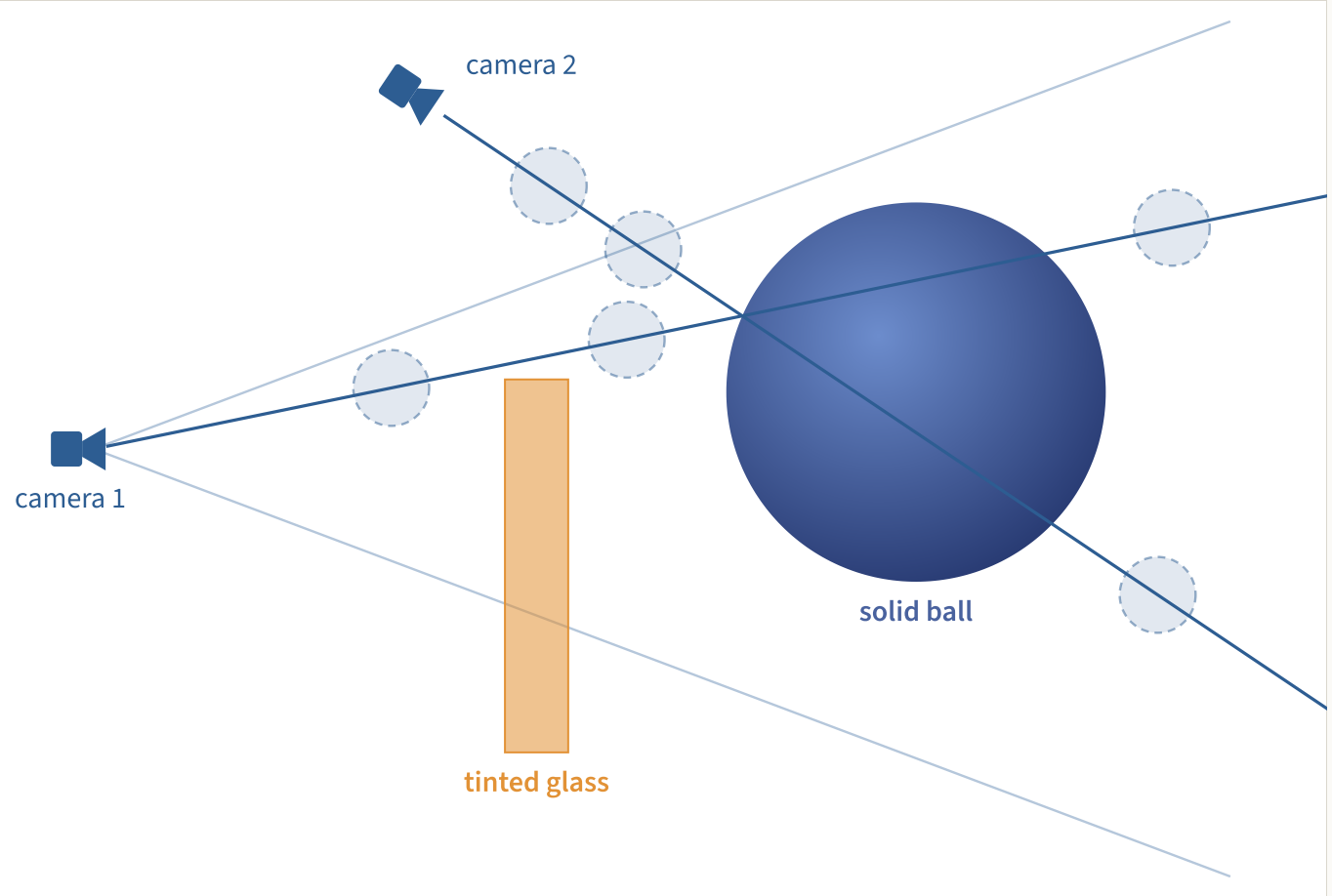


Step 1 of 3

NeRF has no light sources or shading in it; the network only has to match the pixel colors the cameras actually saw. Camera 1 saw blue in this pixel. From camera 1 alone, the blue thing could be anywhere along that pixel's ray, near or far: one photo does not tell the distance.

Mildenhall et al. 2020, Sec. 3 and 5.3 (the loss only compares rendered colors with the photos' pixels; training needs the photos, their camera poses and the scene bounds; no lights or shading model anywhere) and Sec. 6.1 (100 training views per synthetic scene, 20 to 62 photos per real scene). Toy scene as before, seen from above, with a second camera added for this slide; the faint marks are places a single camera cannot tell apart.

Where the 3D shape comes from

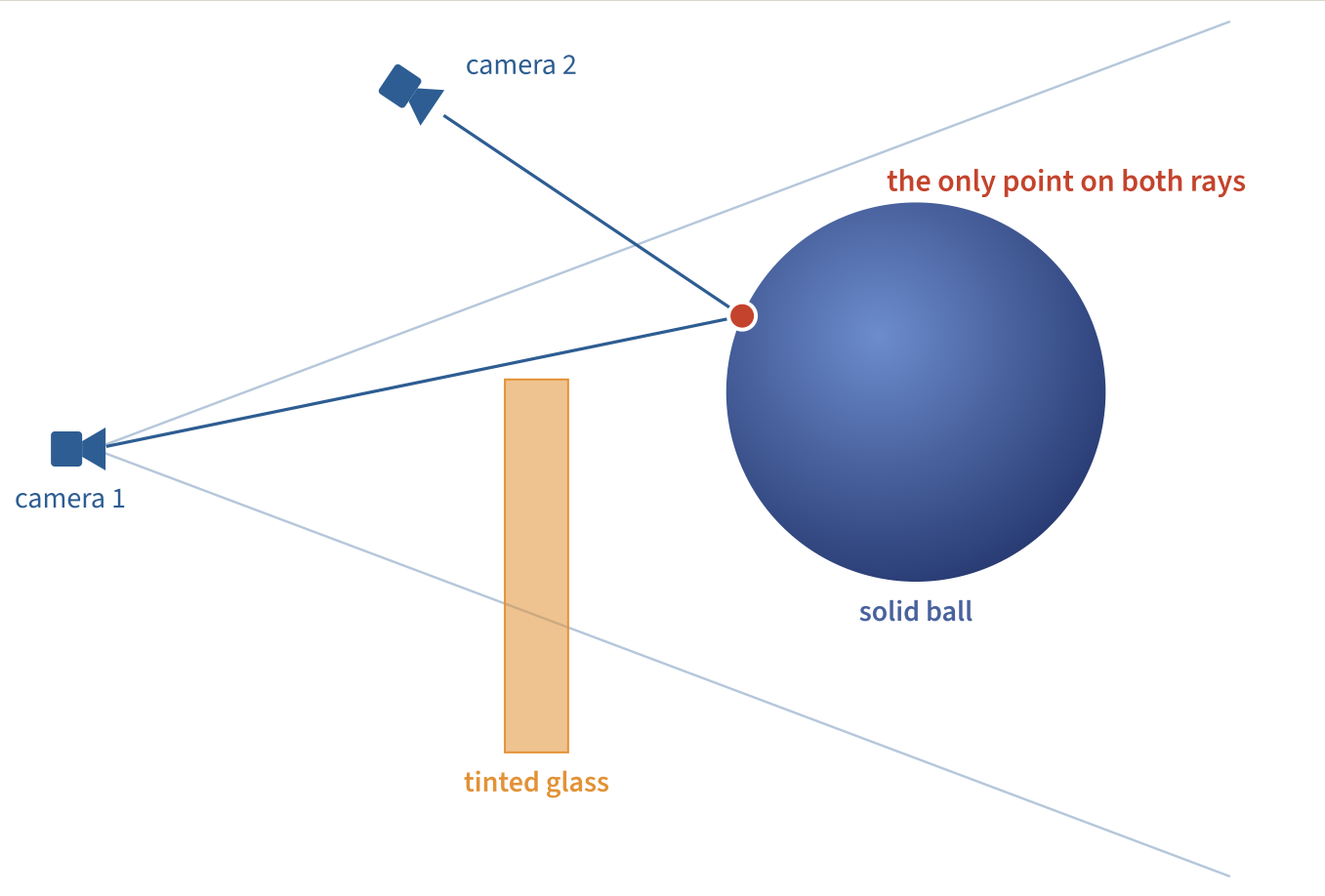


Step 2 of 3

Camera 2 also has a pixel that saw blue, looking from a quite different angle. From camera 2 alone, the blue thing could again be anywhere along its own ray.

Mildenhall et al. 2020, Sec. 3 and 5.3 (the loss only compares rendered colors with the photos' pixels; training needs the photos, their camera poses and the scene bounds; no lights or shading model anywhere) and Sec. 6.1 (100 training views per synthetic scene, 20 to 62 photos per real scene). Toy scene as before, seen from above, with a second camera added for this slide; the faint marks are places a single camera cannot tell apart.

Where the 3D shape comes from



Step 3 of 3

Only one point lies on both rays, and it is on the surface of the ball. Density anywhere else on one ray would show up in the wrong pixel of the other camera, so that crossing is where the network ends up putting its density. With the paper's 20 to 100 photos of a scene, every surface is pinned down this way, although nobody gave any 3D information.

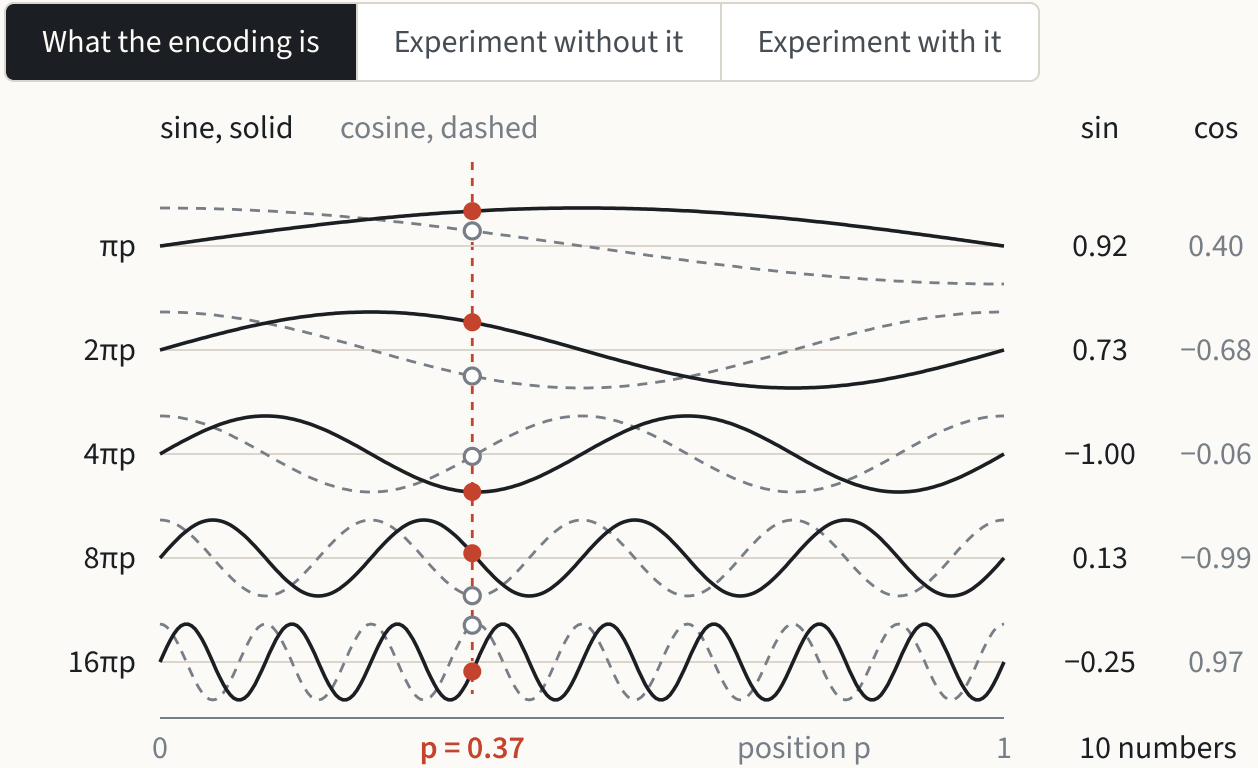
Mildenhall et al. 2020, Sec. 3 and 5.3 (the loss only compares rendered colors with the photos' pixels; training needs the photos, their camera poses and the scene bounds; no lights or shading model anywhere) and Sec. 6.1 (100 training views per synthetic scene, 20 to 62 photos per real scene). Toy scene as before, seen from above, with a second camera added for this slide; the faint marks are places a single camera cannot tell apart.

Positional encoding

A position p is one number. Instead of handing it to the network as it is, NeRF always replaces it with the sines and cosines of $2^k \pi p$ for $k = 0 \dots L-1$: the same waves a Fourier series is built from, at frequencies that double. The drawing reads them off for one p.

NeRF uses $L = 10$, so each coordinate becomes 20 numbers, and x, y, z together 60. The next two tabs show why this is needed.

$$\gamma(p) = (\sin 2^0 \pi p, \cos 2^0 \pi p, \dots, \sin 2^{L-1} \pi p, \cos 2^{L-1} \pi p)$$



Mildenhall et al. 2020, Sec. 5.1, Eq. 4, with $L = 10$ for the position (60 numbers) and $L = 4$ for the direction (24); the encoding is the Fourier-feature mapping of Tancik et al. 2020. Experiment: every second pixel of row 440 of the paper's 800 × 800 Lego render (256 pixels), a 3-layer network with 64 units, 600 Adam steps, same seed in both runs; dB is the PSNR of the fit to those pixels.

Positional encoding

To see the effect, I train a small network live in the browser on a much simpler task: it gets the position along one row of the Lego image and has to give back that pixel's color.

Given the plain position, it learns the smooth shapes quickly and the sharp edges very slowly. After 600 steps it is still a blur, about 18 dB.

$$\gamma(p) = (\ \sin 2^0\pi p,\ \cos 2^0\pi p,\ \ldots,\ \sin 2^{L-1}\pi p,\ \cos 2^{L-1}\pi p\ )$$

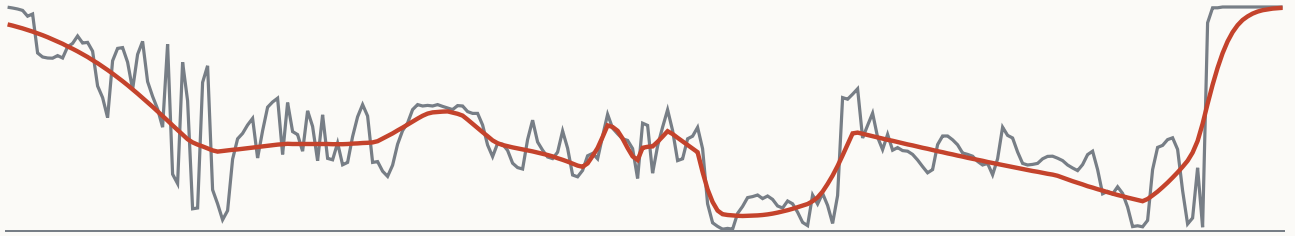
What the encoding is | **Experiment without it** | Experiment with it

The red line is one row of pixels of this Lego image. Below, those 256 pixels are stretched out side by side. The network's job: given the position along the row, output that pixel's color.

The image row, 256 pixels

Network fit, raw position — after 600 steps: 17.9 dB

Brightness along the row — image — network



Mildenhall et al. 2020, Sec. 5.1, Eq. 4, with $L = 10$ for the position (60 numbers) and $L = 4$ for the direction (24); the encoding is the Fourier-feature mapping of Tancik et al. 2020. Experiment: every second pixel of row 440 of the paper's 800 × 800 Lego render (256 pixels), a 3-layer network with 64 units, 600 Adam steps, same seed in both runs; dB is the PSNR of the fit to those pixels.

Positional encoding

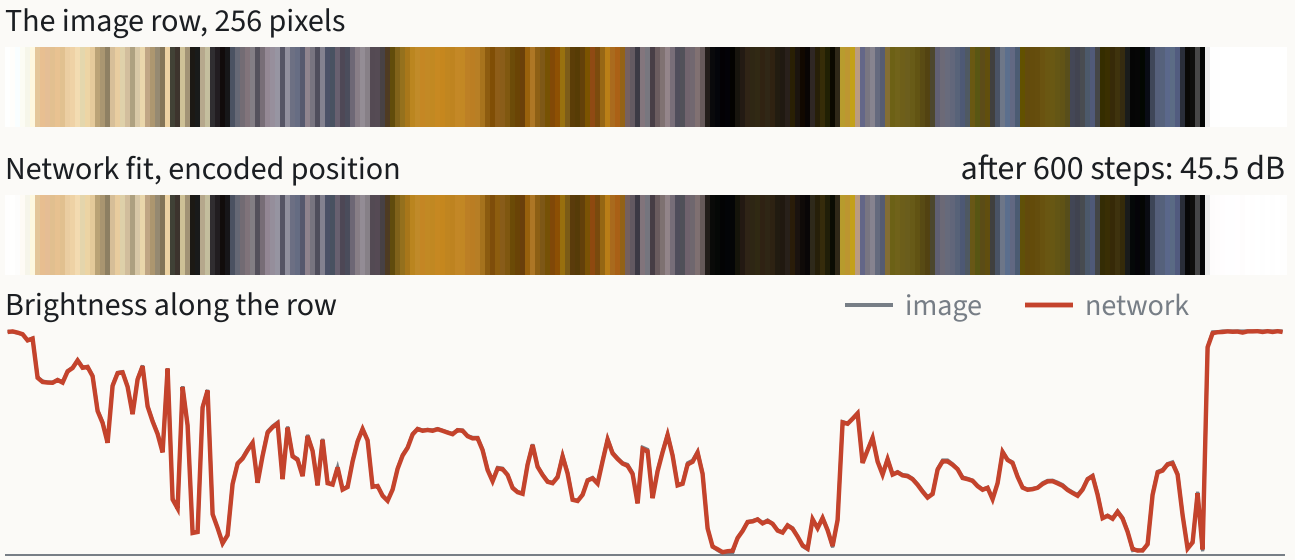
The same experiment again, same network, same seed, same 600 steps, but now the position goes in encoded. It reproduces the row almost exactly, above 40 dB.

The sines and cosines hand the network the sharp building blocks it would otherwise have to build itself, slowly. That is why NeRF encodes every position.

$$\gamma(p) = (\ \sin 2^0\pi p,\ \cos 2^0\pi p,\ \ldots,\ \sin 2^{L-1}\pi p,\ \cos 2^{L-1}\pi p\ )$$

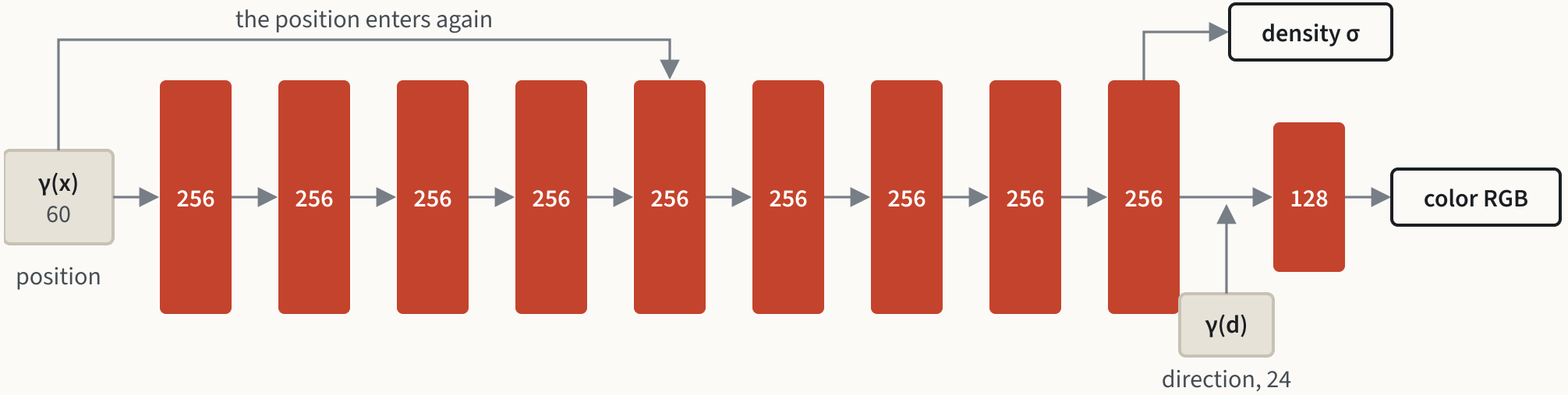
What the encoding is | Experiment without it | **Experiment with it**

The red line is one row of pixels of this Lego image. Below, those 256 pixels are stretched out side by side. The network's job: given the position along the row, output that pixel's color.



Mildenhall et al. 2020, Sec. 5.1, Eq. 4, with $L = 10$ for the position (60 numbers) and $L = 4$ for the direction (24); the encoding is the Fourier-feature mapping of Tancik et al. 2020. Experiment: every second pixel of row 440 of the paper's 800 × 800 Lego render (256 pixels), a 3-layer network with 64 units, 600 Adam steps, same seed in both runs; dB is the PSNR of the fit to those pixels.

Inside the network

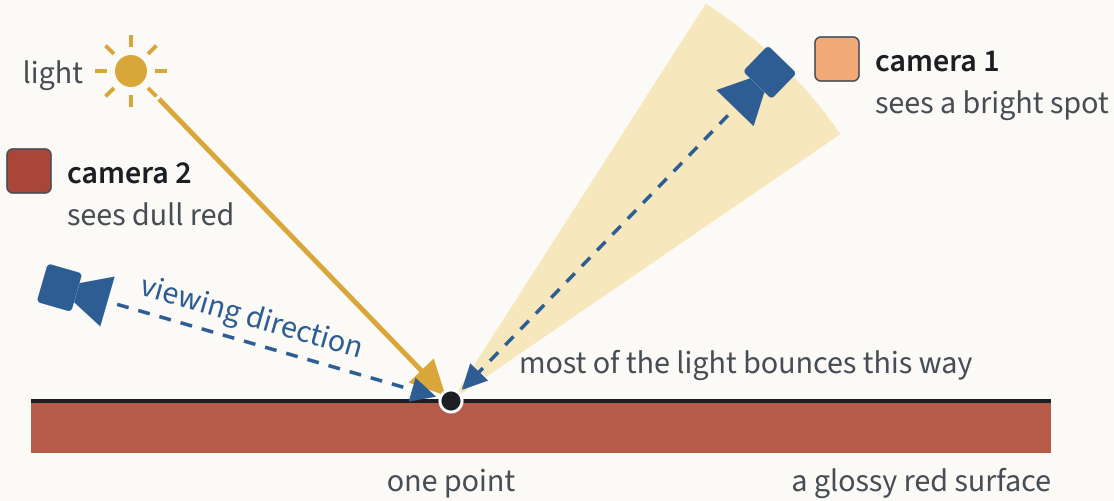


The network is a plain multilayer perceptron, not a CNN, because its input is one point, not an image: a position x, y, z and a viewing direction θ, ϕ. The positional encoding from the previous slide turns the position into 60 numbers, 10 sines and 10 cosines for each of x, y and z, and the direction into 24 the same way with 4 frequencies. Those 84 numbers are the input, the two gray boxes.

The paper uses eight fully connected layers of 256 units with ReLU and feeds the position in again at the fifth. A ninth layer outputs the density σ, kept non-negative by a ReLU, and a 256-number feature. Joined with the direction, that feature passes one layer of 128 units, and a last layer gives the RGB color through a sigmoid, so between 0 and 1. That is about 594,000 weights; two such networks per scene come to about 5 MB.

Redrawn from Mildenhall et al. 2020, Fig. 7 (Appendix A: eight ReLU layers of 256 channels, the density by a ReLU, a 256-dimensional feature joined with the direction for one ReLU layer of 128 channels, RGB by a sigmoid) and Sec. 3 (the input is a 5D coordinate, an MLP). γ: Sec. 5.1, L = 10 for the position (60 numbers) and L = 4 for the direction (24). Weight count computed from these layer sizes, about 594,000; two networks, coarse and fine, Sec. 5.2; 5 MB from Sec. 6.3.

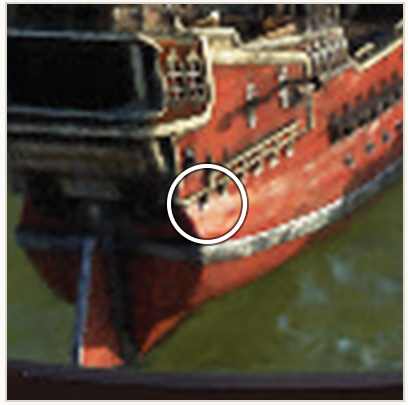
Why the viewing direction is an input



The viewing direction is the direction from which a camera looks at a point. Light that hits a glossy surface bounces off mostly one way: a camera standing in that way sees a bright spot, any other camera sees the plain paint.

Giving the network the direction next to the position lets the same point have a different color for each camera.

From camera 1



From camera 2

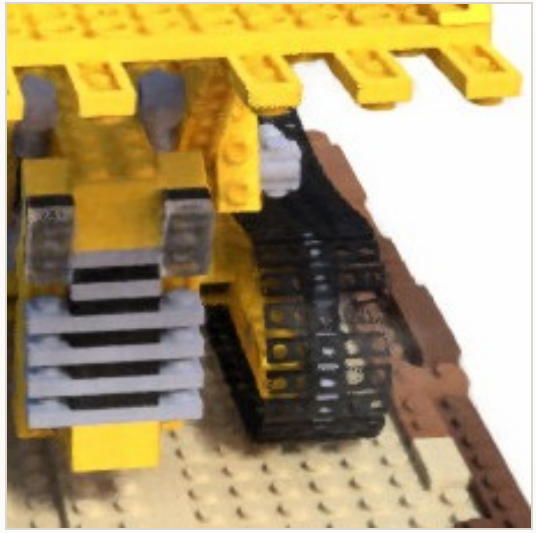
The paper shows it on its Ship scene. The ring marks the same point of the hull in both pictures: from camera 1 it is a dull red, from camera 2 the same point carries a bright highlight, and the network has learned both.

Mildenhall et al. 2020, Fig. 3, Ship scene: the two renders are the figure's own images, cut from the PDF around the hull point the paper marks (100 of the 400 pixels of each render). The drawing at the top is a sketch, not from the paper.

Without the encoding, without the direction

Ground truth

Complete model



Without the viewing direction

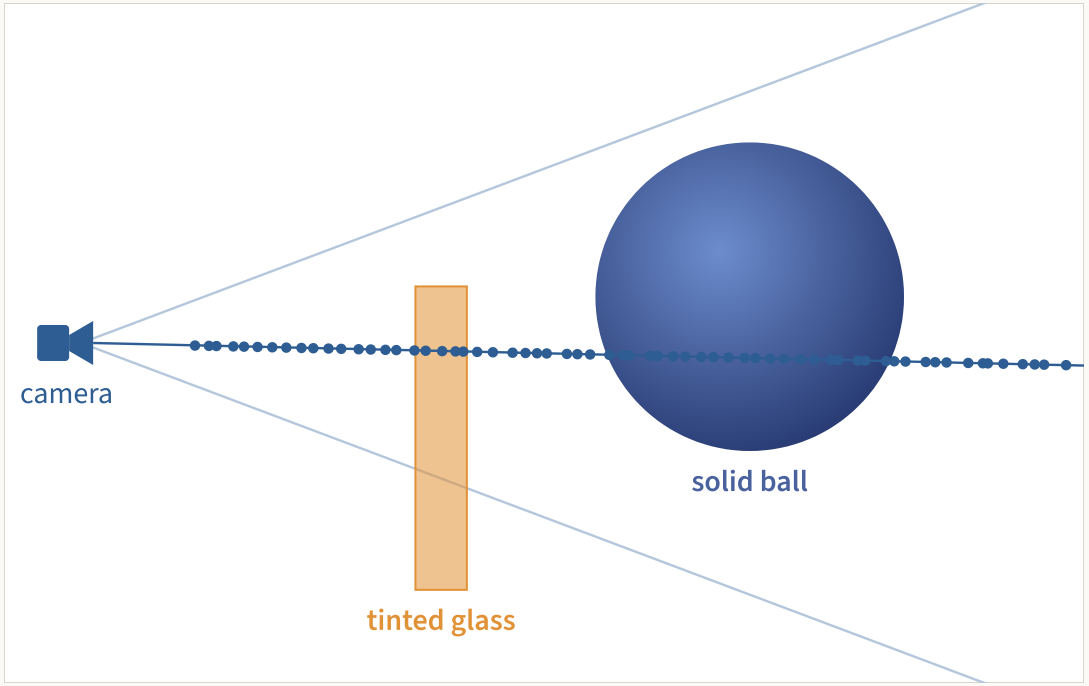
Without positional encoding

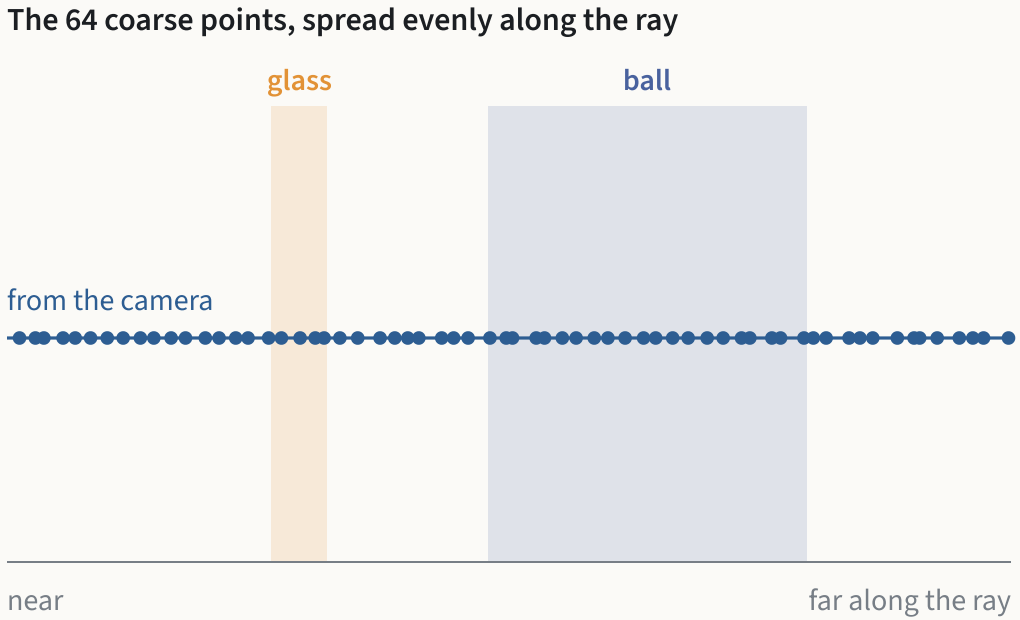
Without the viewing direction, the side the camera looks from, the network still gets the shape right, but every point must keep one color for all cameras, so the shine on the tread's metal pins is gone.

Without the positional encoding, the sines and cosines of the position from the positional encoding slide, the network cannot make sharp edges, and the tread and its studs melt into a smooth dark blur.

Mildenhall et al. 2020, Fig. 4: a zoom on the Lego bulldozer, a test view of the Realistic Synthetic 360° set, cut from the paper's full-resolution renders at the region the figure shows.

Coarse and fine sampling along the ray





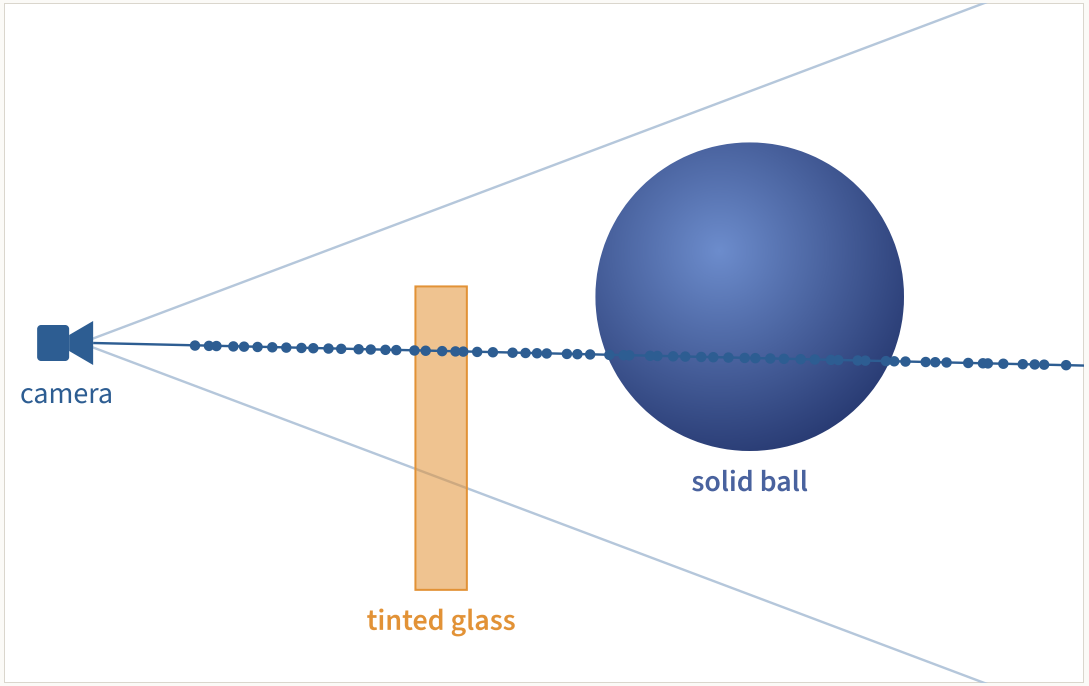
● 64 coarse points, evenly spread

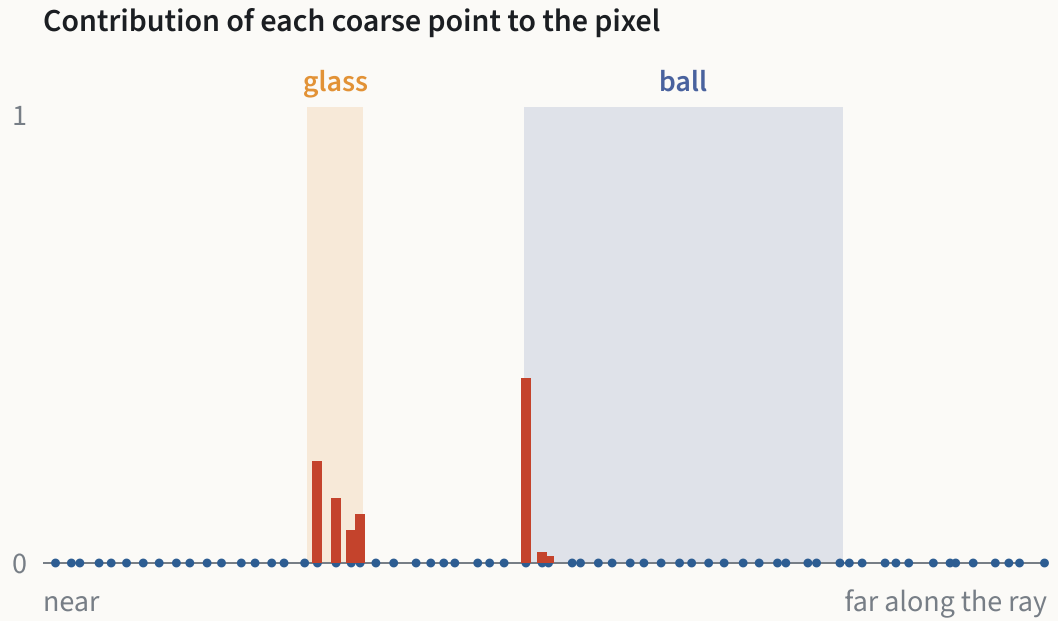
Step 1 of 3

NeRF first asks a coarse network at 64 points spread evenly along the ray. Most of them land in empty air or behind the ball, where the answer cannot change the pixel, so that work is wasted.

Mildenhall et al. 2020, Sec. 5.2, Eq. 5: $N_c = 64$ coarse and $N_f = 128$ fine points, drawn by inverse transform sampling from the normalized coarse weights. Toy scene, ray and contributions as on the rendering slide; the contribution bars are the $T_i\,(1 - e^{-\sigma_i \delta_i})$ of Eq. 3.

Coarse and fine sampling along the ray





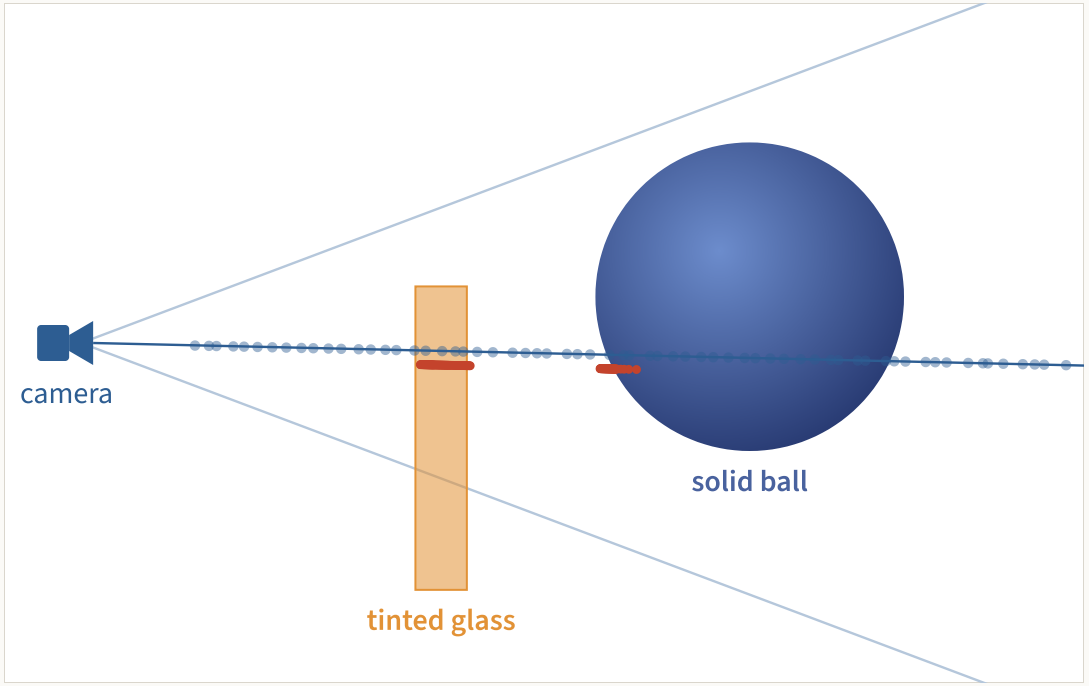
64 coarse points, evenly spread

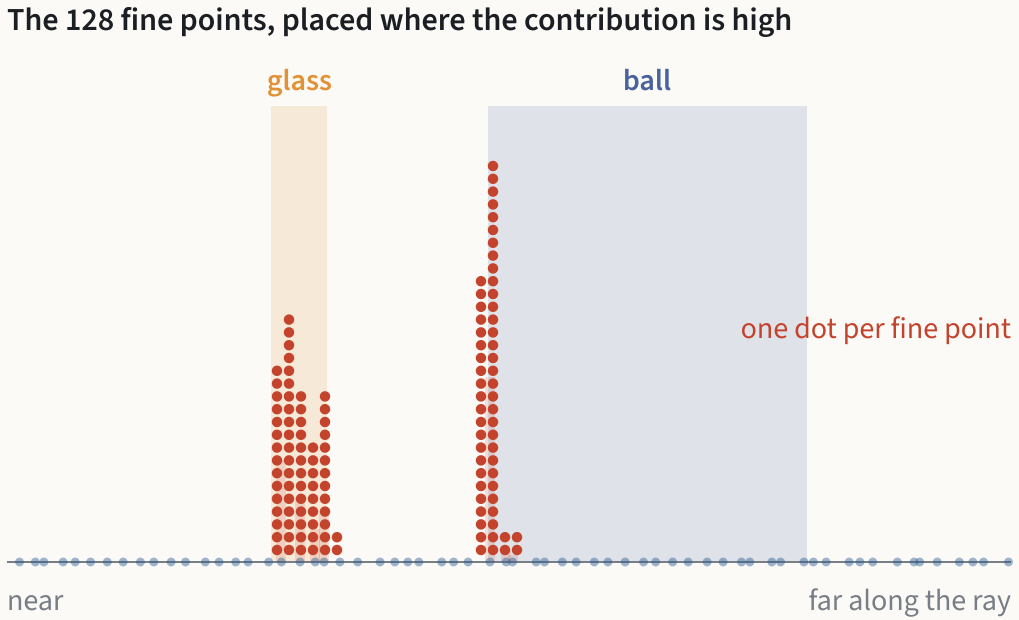
Step 2 of 3

The contribution of each coarse point, the same weight as in the rendering sum, shows where the pixel's color came from: the glass and the front of the ball. Everything behind the ball contributes almost nothing.

Mildenhall et al. 2020, Sec. 5.2, Eq. 5: $N_c = 64$ coarse and $N_f = 128$ fine points, drawn by inverse transform sampling from the normalized coarse weights. Toy scene, ray and contributions as on the rendering slide; the contribution bars are the $T_i\,(1 - e^{-\sigma_i \delta_i})$ of Eq. 3.

Coarse and fine sampling along the ray





● 64 coarse points, evenly spread ● 128 fine points, where the matter is

Step 3 of 3

NeRF then places 128 more points where the contribution is high, and a second, fine network renders the ray from all 192 points. The fine points crowd the glass and the front of the ball.

Mildenhall et al. 2020, Sec. 5.2, Eq. 5: $N_c = 64$ coarse and $N_f = 128$ fine points, drawn by inverse transform sampling from the normalized coarse weights. Toy scene, ray and contributions as on the rendering slide; the contribution bars are the $T_i\,(1 - e^{-\sigma_i \delta_i})$ of Eq. 3.

One training step in code

```
pixels, photo_rgb = pick_random_pixels(photos, n=4096)            # 4,096 pixels from all the photos
rays = ray_of_each_pixel(pixels, camera_poses)                    # the ray each pixel looks along
coarse = points_along_ray(rays, n=64)                             # 64 points, evenly spread near to far
answers = ask_network(coarse_net, coarse)                         # density and color at every point
color_c, weights = blend_along_ray(answers)                       # one color; weights = contributions
fine = place_more_points_where_matter_is(weights, n=128)          # 128 more where contribution is high
color_f = blend_along_ray(ask_network(fine_net, coarse + fine))   # render again from all 192 points
loss = error(color_c, photo_rgb) + error(color_f, photo_rgb)      # squared error of both renders, Eq. 6
update_both_networks(loss)                                        # Adam; learning rate 5e-4 to 5e-5
```

Each scene is trained from scratch with 100,000 to 300,000 of these steps. On one NVIDIA V100 GPU that takes one to two days.

For real photos the camera poses come from COLMAP, a free program. It finds the same small details in many photos and, from where they land in each one, works out where every camera stood and a sparse cloud of 3D points. NeRF uses only the camera poses.

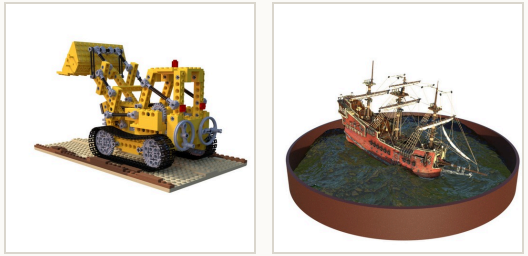
Rendering one 800 × 800 image means 640,000 rays and 150 to 200 million network queries, about 30 seconds per image.

Mildenhall et al. 2020, Sec. 5.3 (4,096 rays per step, 64 coarse and 128 fine points, 100,000 to 300,000 steps, one to two days on a V100, learning rate 5×10^{-4} decaying to 5×10^{-5}) and Appendix A. The code is plain pseudo-code written for this talk, not the official implementation; the highlighted lines are the coarse-to-fine part. COLMAP: Schönberger & Frahm 2016.

The three test sets

Diffuse Synthetic 360°

The easy set, borrowed from the DeepVoxels paper: four simple objects with matte paint, a chair, a pedestal, a cube and a vase, rendered by a computer from all around the upper half. Each one has 479 training views at 512 × 512 pixels. The NeRF paper prints no pictures of these.



Realistic Synthetic 360°

Eight objects the authors rendered themselves, with shiny and see-through parts: a Lego bulldozer, a ship in water, drums, a microphone, a ficus and more. Each has 100 training views and 200 held-out test views at 800 × 800 pixels.

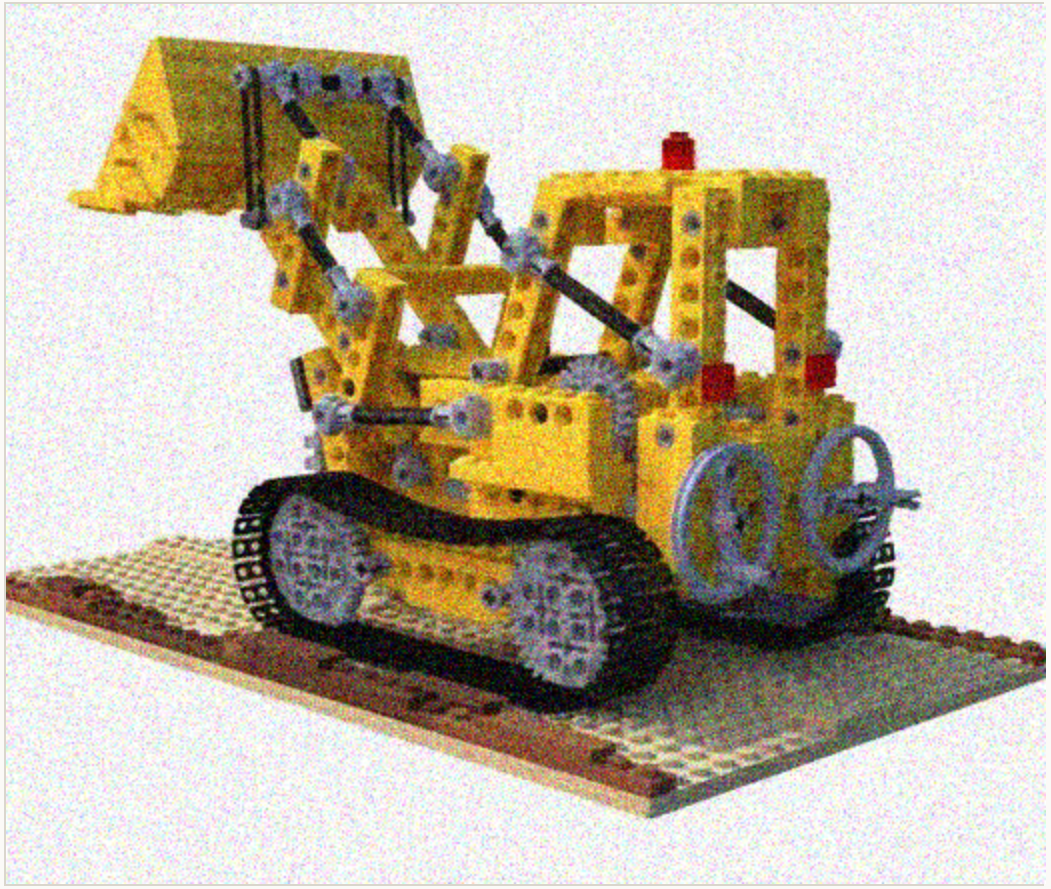
Real Forward-Facing

Eight real places photographed with a phone, always from the front: a fern, a T-Rex skeleton in a museum, flowers, a room. Between 20 and 62 photos each, and one photo in eight is held back for testing.

Every method renders the held-out views, and each render is compared with the real image with three numbers: PSNR, SSIM and LPIPS. The next two slides explain what they mean.

Mildenhall et al. 2020, Sec. 6.1 and Table 1. Images are test views from Fig. 5 and Fig. 6. DeepVoxels: Sitzmann et al. 2019.

What a decibel of PSNR means



A decibel measures how many times larger one power is than another: ten times the logarithm of their ratio. That is why it is used for sound, radio signals and images alike.

PSNR compares the brightest possible pixel with the average squared error between the render and the photo. Every 10 dB means ten times less squared error; 3 dB means half.

$$\mathrm{PSNR} = 10 \cdot \log_{10}(1 / \mathrm{MSE})$$

$$\mathrm{MSE} = 0.001 \;\rightarrow\; 10 \cdot \log_{10}(1000) = 30 \text{ dB}$$

$$\mathrm{MSE} = 0.0001 \;\rightarrow\; 10 \cdot \log_{10}(10\,000) = 40 \text{ dB}$$

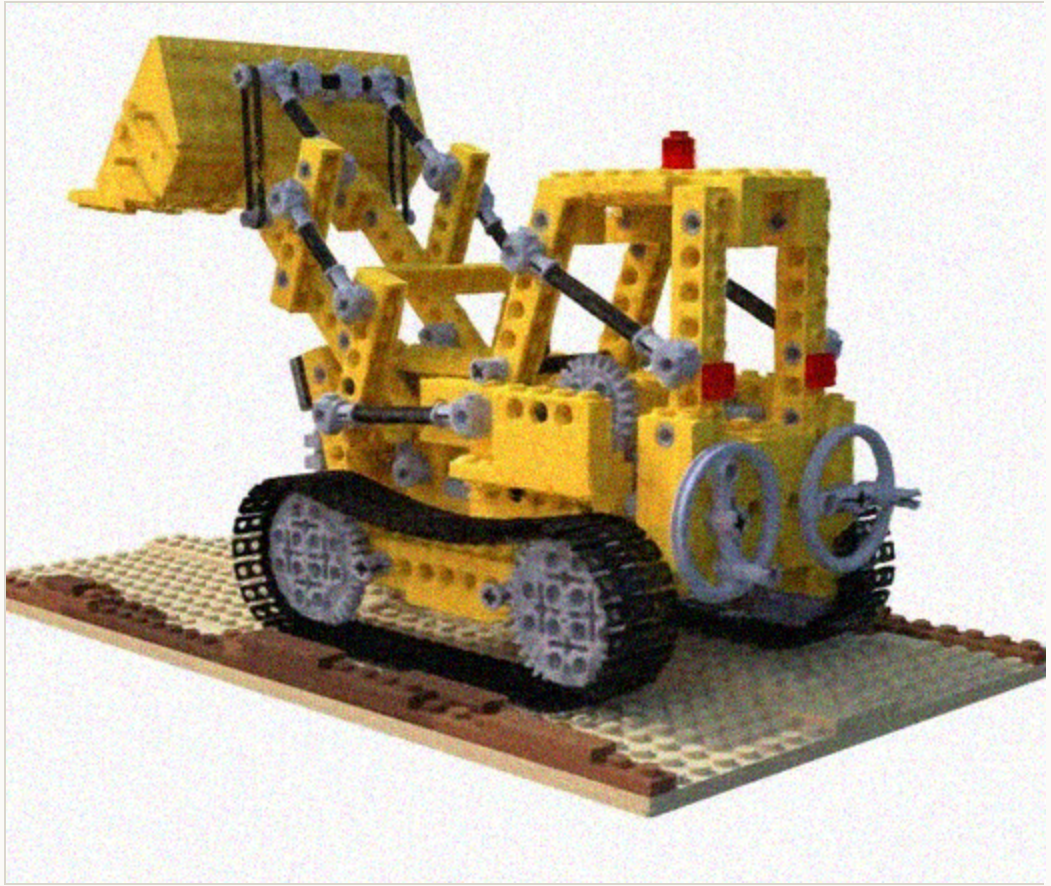
at 30 dB a typical pixel is off by $255 \cdot \sqrt{0.001} \approx 8$ levels

20 dB | 26 dB | 31 dB | 40 dB

At 20 dB a typical pixel is off by 26 of 255 brightness levels, and the error is obvious.

MSE is the mean squared error over all pixels and color channels, with pixel values from 0 to 1, so the peak value squared is 1. Image: NeRF's Lego render from Mildenhall et al. 2020, with random noise added until its PSNR against the clean image is the value shown.

What a decibel of PSNR means



A decibel measures how many times larger one power is than another: ten times the logarithm of their ratio. That is why it is used for sound, radio signals and images alike.

PSNR compares the brightest possible pixel with the average squared error between the render and the photo. Every 10 dB means ten times less squared error; 3 dB means half.

$$\text{PSNR} = 10 \cdot \log_{10}(1 / \text{MSE})$$

$$\text{MSE} = 0.001 \;\rightarrow\; 10 \cdot \log_{10}(1000) = 30 \text{ dB}$$

$$\text{MSE} = 0.0001 \;\rightarrow\; 10 \cdot \log_{10}(10\,000) = 40 \text{ dB}$$

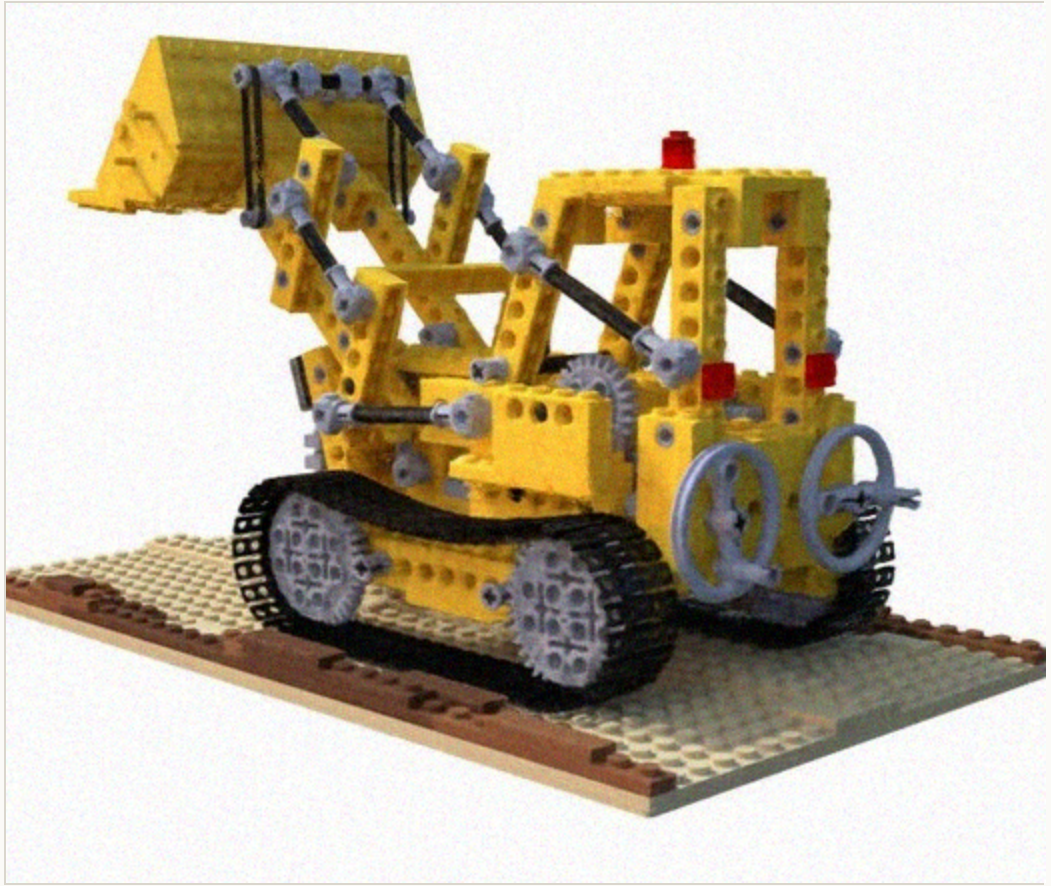
at 30 dB a typical pixel is off by $255 \cdot \sqrt{0.001} \approx 8$ levels

20 dB	**26 dB**	31 dB	40 dB

At 26 dB, about Neural Volumes on the synthetic scenes, a pixel is off by 13 levels.

MSE is the mean squared error over all pixels and color channels, with pixel values from 0 to 1, so the peak value squared is 1. Image: NeRF's Lego render from Mildenhall et al. 2020, with random noise added until its PSNR against the clean image is the value shown.

What a decibel of PSNR means



A decibel measures how many times larger one power is than another: ten times the logarithm of their ratio. That is why it is used for sound, radio signals and images alike.

PSNR compares the brightest possible pixel with the average squared error between the render and the photo. Every 10 dB means ten times less squared error; 3 dB means half.

$$\mathrm{PSNR} = 10 \cdot \log_{10}(1 / \mathrm{MSE})$$

$$\mathrm{MSE} = 0.001 \;\rightarrow\; 10 \cdot \log_{10}(1000) = 30 \text{ dB}$$

$$\mathrm{MSE} = 0.0001 \;\rightarrow\; 10 \cdot \log_{10}(10\,000) = 40 \text{ dB}$$

at 30 dB a typical pixel is off by $255 \cdot \sqrt{0.001} \approx 8$ levels

20 dB	26 dB	31 dB	40 dB

At 31 dB, NeRF on the same scenes, the error is 7.2 levels: visible only from close up.

MSE is the mean squared error over all pixels and color channels, with pixel values from 0 to 1, so the peak value squared is 1. Image: NeRF's Lego render from Mildenhall et al. 2020, with random noise added until its PSNR against the clean image is the value shown.

What a decibel of PSNR means

A decibel measures how many times larger one power is than another: ten times the logarithm of their ratio. That is why it is used for sound, radio signals and images alike.

PSNR compares the brightest possible pixel with the average squared error between the render and the photo. Every 10 dB means ten times less squared error; 3 dB means half.

$$\text{PSNR} = 10 \cdot \log_{10}(1 / \text{MSE})$$

$$\text{MSE} = 0.001 \;\rightarrow\; 10 \cdot \log_{10}(1000) = 30 \text{ dB}$$

$$\text{MSE} = 0.0001 \;\rightarrow\; 10 \cdot \log_{10}(10\,000) = 40 \text{ dB}$$

at 30 dB a typical pixel is off by $255 \cdot \sqrt{0.001} \approx 8$ levels

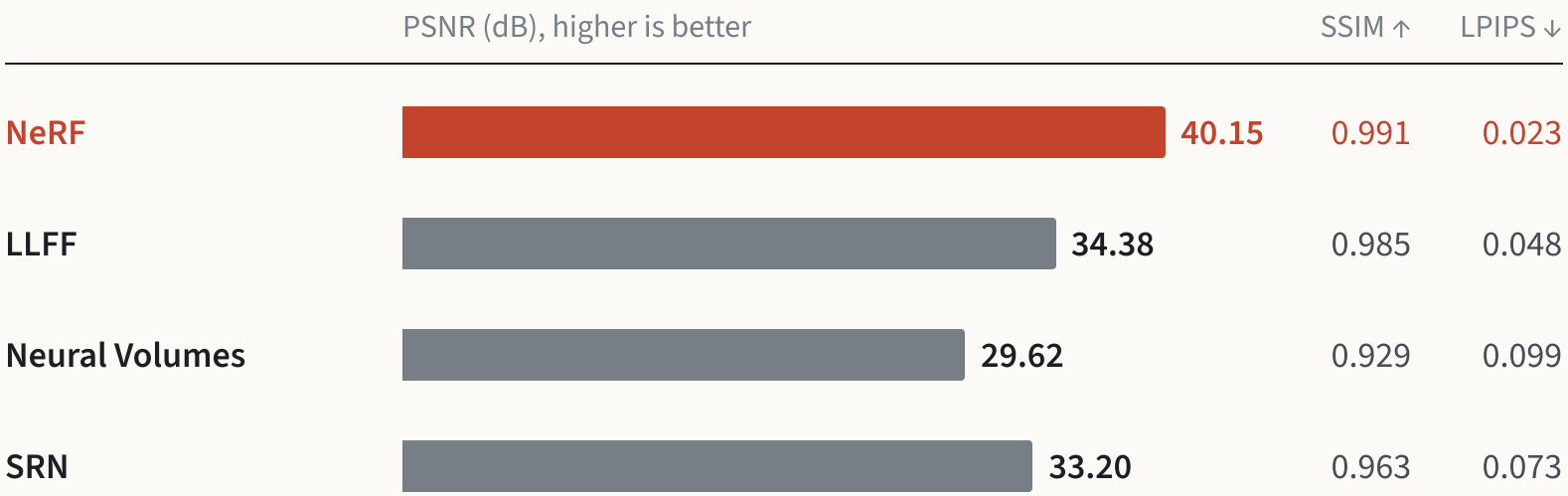
20 dB	26 dB	31 dB	40 dB

At 40 dB, NeRF on the matte objects, it is 2.6 levels and the image looks clean.

MSE is the mean squared error over all pixels and color channels, with pixel values from 0 to 1, so the peak value squared is 1. Image: NeRF's Lego render from Mildenhall et al. 2020, with random noise added until its PSNR against the clean image is the value shown.

Main results

Diffuse Synthetic | Realistic Synthetic | Real Forward-Facing



Four matte objects with simple shapes. Every method does well, but NeRF is almost 6 dB above the next best, LLFF, which means about a quarter of its squared error.

PSNR is the pixel error in decibels, and higher is better. 3 dB more means half the squared error, 10 dB more means a tenth.

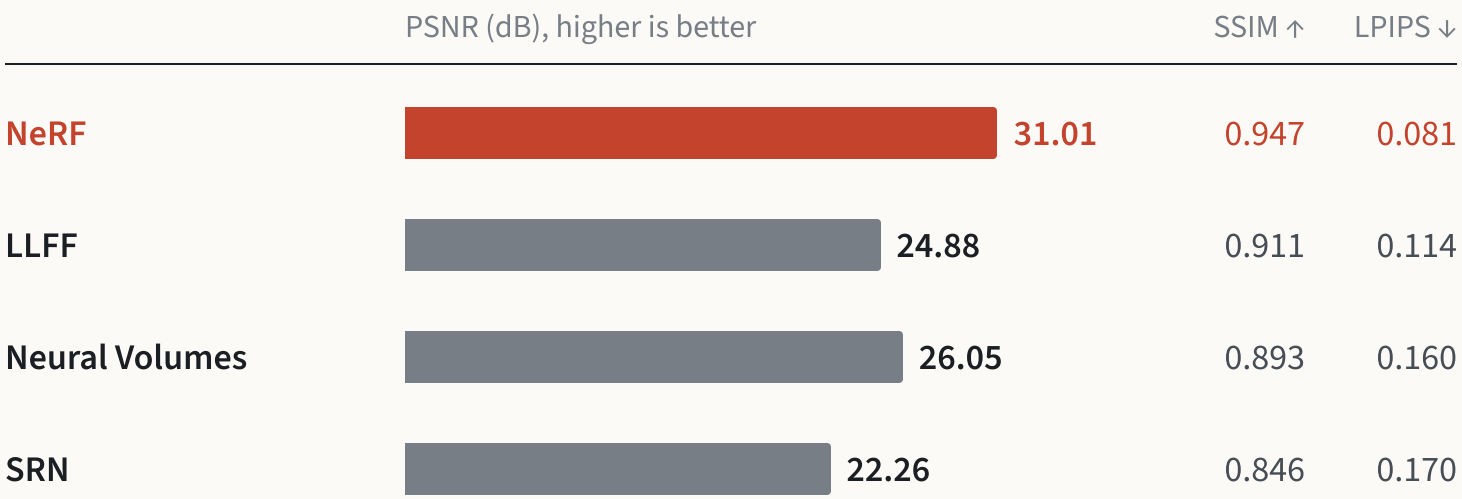
SSIM compares the brightness, contrast and local structure of small patches of the two images. 1 means identical. Wang et al. 2004.

LPIPS runs both images through a deep network and measures how far apart the features are, which matches what people see better than pixel error does. Lower is better. Zhang et al. 2018.

Mildenhall et al. 2020, Table 1. SRN: Sitzmann et al. 2019; Neural Volumes (NV): Lombardi et al. 2019; LLFF: Mildenhall et al. 2019. NV cannot run on the real scenes because it needs an object inside a bounded volume. SSIM: Wang et al. 2004; LPIPS: Zhang et al. 2018.

Main results

Diffuse Synthetic | **Realistic Synthetic** | Real Forward-Facing



Eight objects with fine detail and shiny or see-through materials. NeRF reaches 31.0 dB and the next best, Neural Volumes, 26.1 dB, so NeRF has about a third of the error.

PSNR is the pixel error in decibels, and higher is better. 3 dB more means half the squared error, 10 dB more means a tenth.

SSIM compares the brightness, contrast and local structure of small patches of the two images. 1 means identical. Wang et al. 2004.

LPIPS runs both images through a deep network and measures how far apart the features are, which matches what people see better than pixel error does. Lower is better. Zhang et al. 2018.

Mildenhall et al. 2020, Table 1. SRN: Sitzmann et al. 2019; Neural Volumes (NV): Lombardi et al. 2019; LLFF: Mildenhall et al. 2019. NV cannot run on the real scenes because it needs an object inside a bounded volume. SSIM: Wang et al. 2004; LPIPS: Zhang et al. 2018.

Main results

Diffuse Synthetic | Realistic Synthetic | **Real Forward-Facing**

	PSNR (dB), higher is better	SSIM ↑	LPIPS ↓
NeRF	**26.50**	0.811	0.250
LLFF	**24.13**	0.798	0.212
Neural Volumes	needs a bounded object	–	–
SRN	**22.84**	0.668	0.378

Real phone captures from the front, with 20 to 62 photos each. The margin shrinks to 2.4 dB over LLFF, which was designed for exactly these scenes and has the better LPIPS.

PSNR is the pixel error in decibels, and higher is better. 3 dB more means half the squared error, 10 dB more means a tenth.

SSIM compares the brightness, contrast and local structure of small patches of the two images. 1 means identical. Wang et al. 2004.

LPIPS runs both images through a deep network and measures how far apart the features are, which matches what people see better than pixel error does. Lower is better. Zhang et al. 2018.

Mildenhall et al. 2020, Table 1. SRN: Sitzmann et al. 2019; Neural Volumes (NV): Lombardi et al. 2019; LLFF: Mildenhall et al. 2019. NV cannot run on the real scenes because it needs an object inside a bounded volume. SSIM: Wang et al. 2004; LPIPS: Zhang et al. 2018.

What the difference looks like

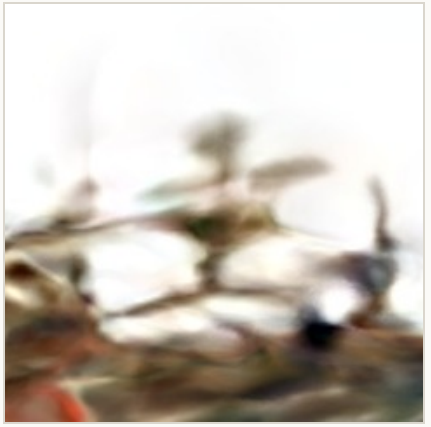
Ship | Lego | Microphone | Fern | T-Rex

Ground truth

NeRF

LLFF

Neural Volumes



SRN

The masts, the furled sails and the rigging against the sky. NeRF keeps the sails and the wooden parts close to the ground truth; LLFF smears the sails and the wood, Neural Volumes loses the thin ropes, and SRN blurs the whole ship.

Mildenhall et al. 2020, Fig. 5 (synthetic) and Fig. 6 (real): test views, cut from the paper's full-resolution renders. The same region is shown for every method.

What the difference looks like

Ship | **Lego** | Microphone | Fern | T-Rex

Ground truth

NeRF

LLFF

Neural Volumes

SRN

The gears, the steering wheels and the tread. NeRF keeps the spokes and the tread links sharp; LLFF shows some additional blur, Neural Volumes softens the detail and SRN keeps only the yellow shape.

Mildenhall et al. 2020, Fig. 5 (synthetic) and Fig. 6 (real): test views, cut from the paper's full-resolution renders. The same region is shown for every method.

What the difference looks like

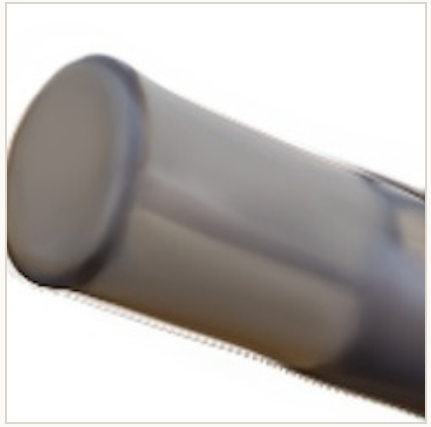
Ship | Lego | **Microphone** | Fern | T-Rex

Ground truth

NeRF

LLFF

Neural Volumes



SRN

The microphone's mesh grille. NeRF and LLFF both reproduce the fine mesh, and in a still image they are hard to tell apart; Neural Volumes turns it into a smudged cylinder and SRN into a smooth one.

Mildenhall et al. 2020, Fig. 5 (synthetic) and Fig. 6 (real): test views, cut from the paper's full-resolution renders. The same region is shown for every method.

What the difference looks like

Ship | Lego | Microphone | **Fern** | T-Rex

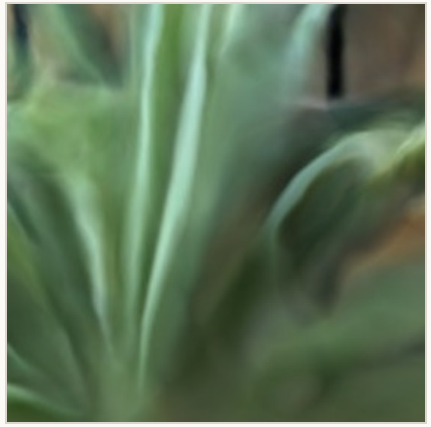
Ground truth

NeRF

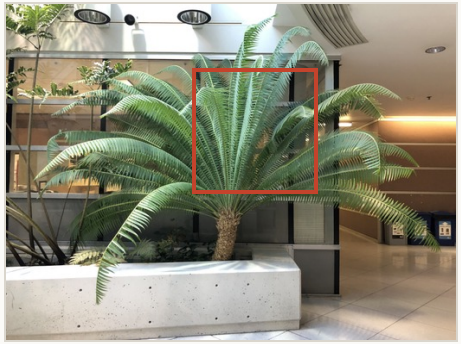
LLFF

not run on real scenes

Neural Volumes



SRN



A real photo, where Neural Volumes cannot run. The fern's thin leaves stay sharp in NeRF, LLFF is nearly as sharp and only a little softer on the right, and SRN keeps just the rough green shape.

Mildenhall et al. 2020, Fig. 5 (synthetic) and Fig. 6 (real): test views, cut from the paper's full-resolution renders. The same region is shown for every method.

What the difference looks like

Ship | Lego | Microphone | Fern | **T-Rex**

Ground truth

NeRF

LLFF

not run on real scenes

Neural Volumes

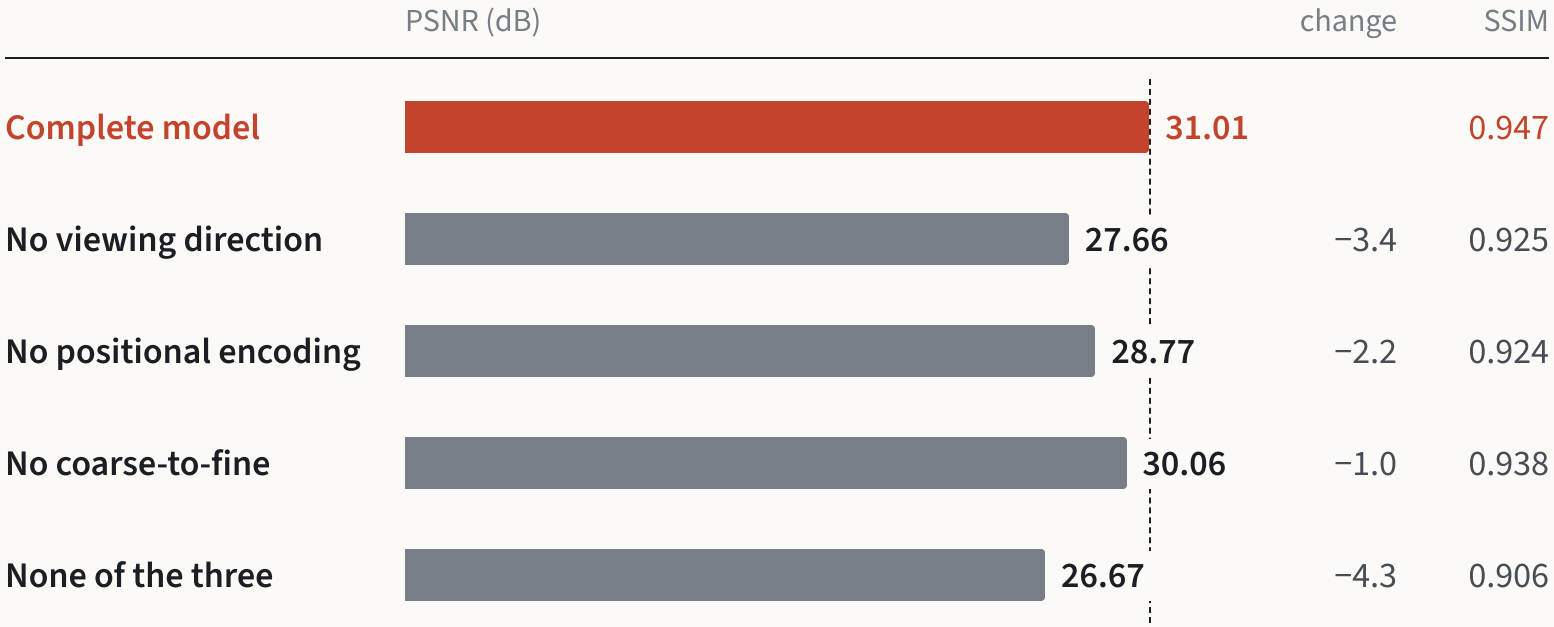
SRN

The skeleton's ribs and the railing behind them. NeRF keeps each rib and each bar of the railing separate; LLFF is close, only slightly blurrier on the railing. SRN merges the bones into one smooth shape.

Mildenhall et al. 2020, Fig. 5 (synthetic) and Fig. 6 (real): test views, cut from the paper's full-resolution renders. The same region is shown for every method.

What each part contributes

Remove a part | Fewer photos | Number of frequencies



PSNR is the pixel error, in decibels: higher is better, and the change column is the drop from the complete model. SSIM is how similar the local structure of the render is to the ground-truth image, where 1 means identical.

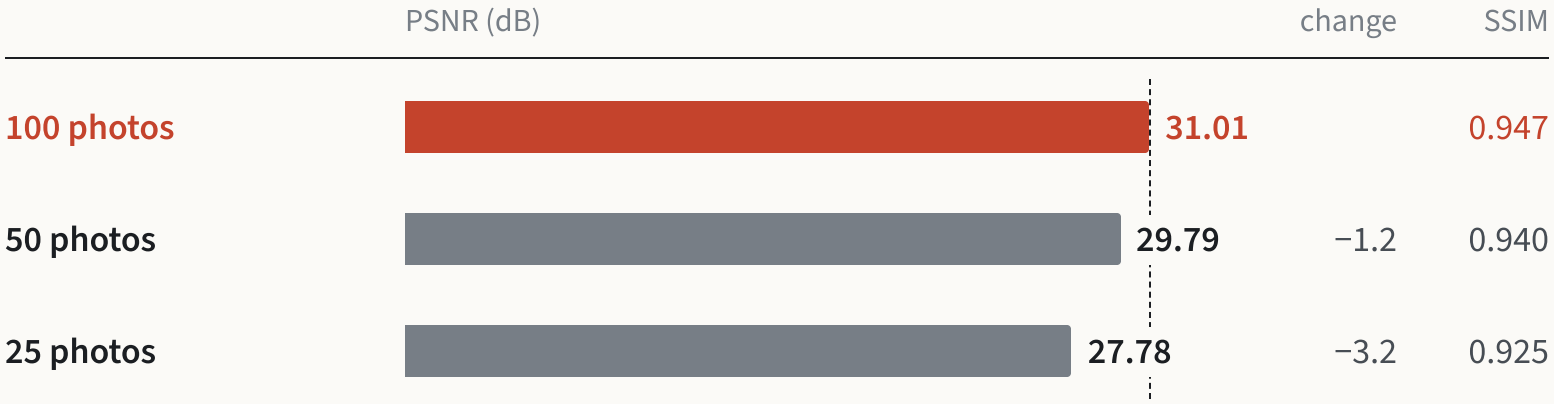
On these shiny scenes the viewing direction matters most: without it the score drops by 3.4 dB. The encoding is worth 2.2 dB and coarse-to-fine sampling 1.0 dB.

Without all three, the model loses 4.3 dB.

Mildenhall et al. 2020, Table 2: mean over the eight Realistic Synthetic 360° scenes. The dashed line is the complete model, 31.01 dB.

What each part contributes

Remove a part | **Fewer photos** | Number of frequencies



PSNR is the pixel error, in decibels: higher is better, and the change column is the drop from the complete model. SSIM is how similar the local structure of the render is to the ground-truth image, where 1 means identical.

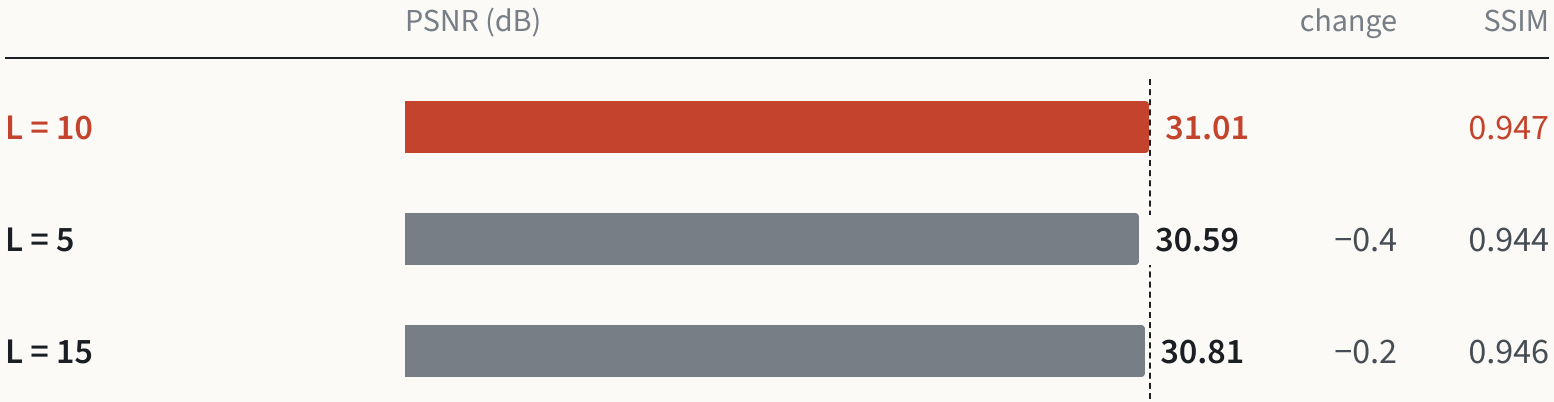
With 50 photos instead of 100 NeRF loses 1.2 dB, with 25 photos 3.2 dB.

Even with 25 photos it still beats every baseline that gets all 100.

Mildenhall et al. 2020, Table 2: mean over the eight Realistic Synthetic 360° scenes. The dashed line is the complete model, 31.01 dB.

What each part contributes

Remove a part | Fewer photos | **Number of frequencies**



L is the number of sine and cosine frequencies in the positional encoding; NeRF uses 10.

PSNR is the pixel error, in decibels: higher is better, and the change column is the drop from the complete model. SSIM is how similar the local structure of the render is to the ground-truth image, where 1 means identical.

Five frequencies are too few, and fifteen are no better than ten.

Once the finest sine is finer than the pixels, there is no more detail to fit.

Mildenhall et al. 2020, Table 2: mean over the eight Realistic Synthetic 360° scenes. The dashed line is the complete model, 31.01 dB.

What NeRF costs

5 MB

to store one scene

The two networks are the whole scene. LLFF, the best earlier method on real photos, keeps a stack of image layers for every input view and needs over 15 GB for the same scene.

1 to 2 days

to train one scene, on one V100 GPU

Every scene starts from scratch: 100,000 to 300,000 training steps. SRN and Neural Volumes also need at least 12 hours; LLFF is ready in minutes only because its network was trained beforehand on many scenes.

30 s

to render one 800 × 800 image

Every pixel means 192 network queries, so one image is 150 to 200 million of them. A short video of the scene takes hours to make.

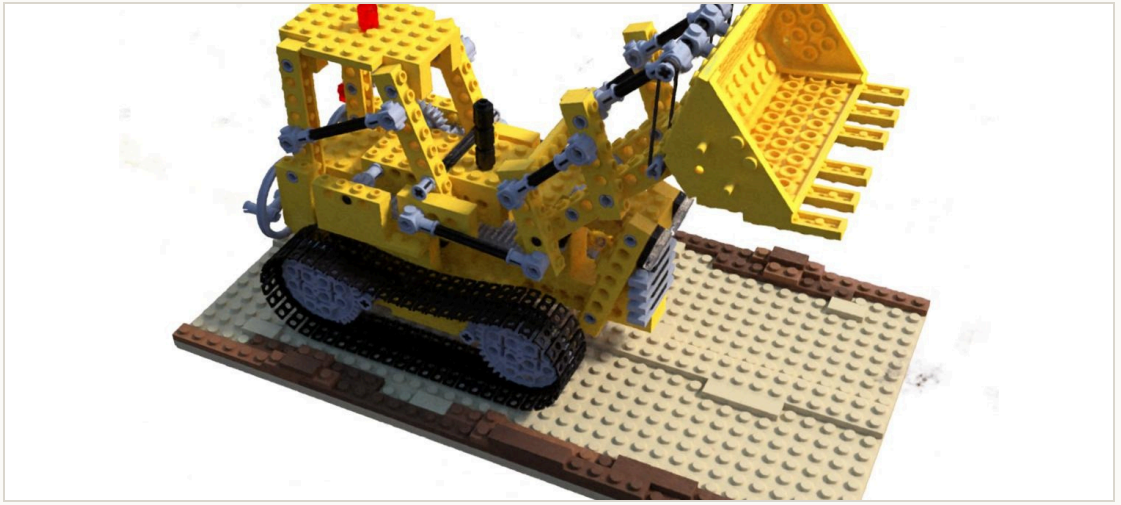
> NeRF is small to store, but it takes days to train and half a minute per image. The next two papers are about those two numbers.

Mildenhall et al. 2020, Sec. 6.3 (5 MB, LLFF over 15 GB, training times) and Sec. 5.3 and Appendix A (steps, 30 s per 800 × 800 image on a V100, 192 queries per ray).

Making it fast

NeRF needs one to two days to train a scene and half a minute to render one image. Two later papers removed most of that cost.

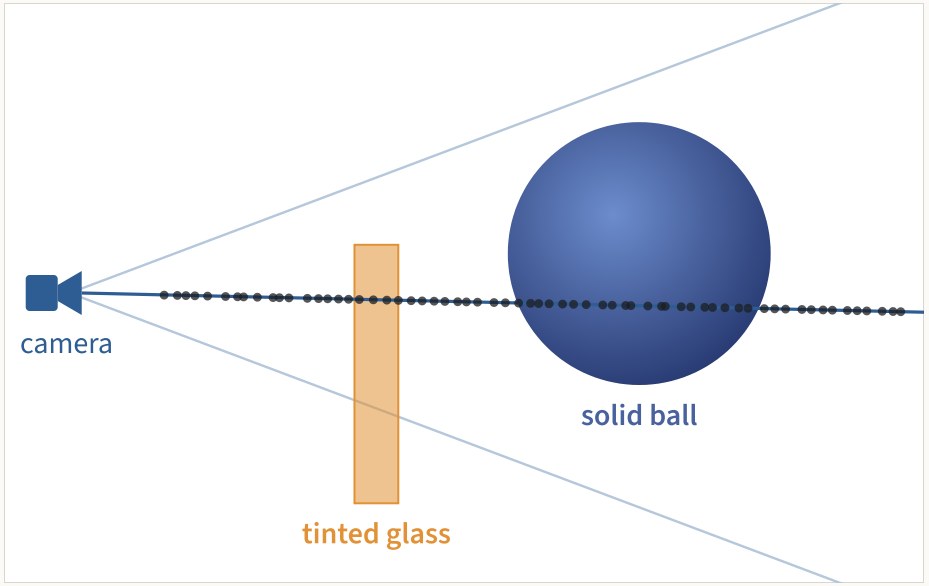
Instant-NGP (2022) keeps the rays but moves most of the work from the network into a learned lookup table. 3D Gaussian splatting (2023) uses neither a network nor rays.



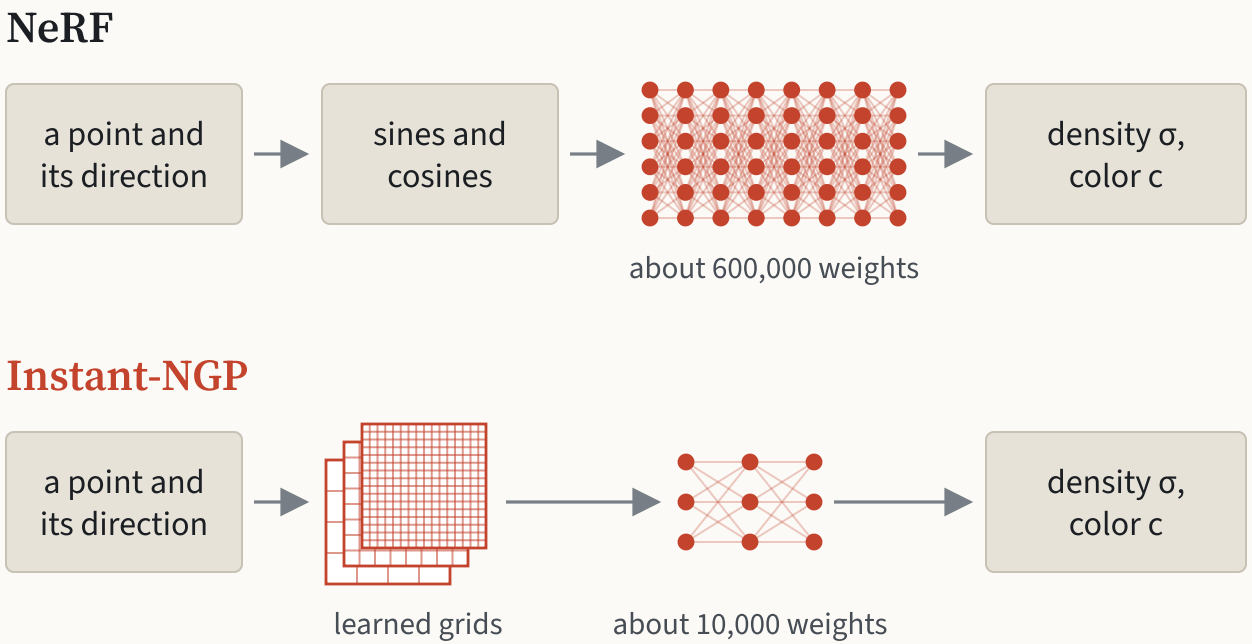
Instant-NGP, after 15 seconds of training

3D Gaussian splatting, rendered in real time

What Instant-NGP changes in NeRF



Rays, the points along them and the rendering sum stay as in NeRF.

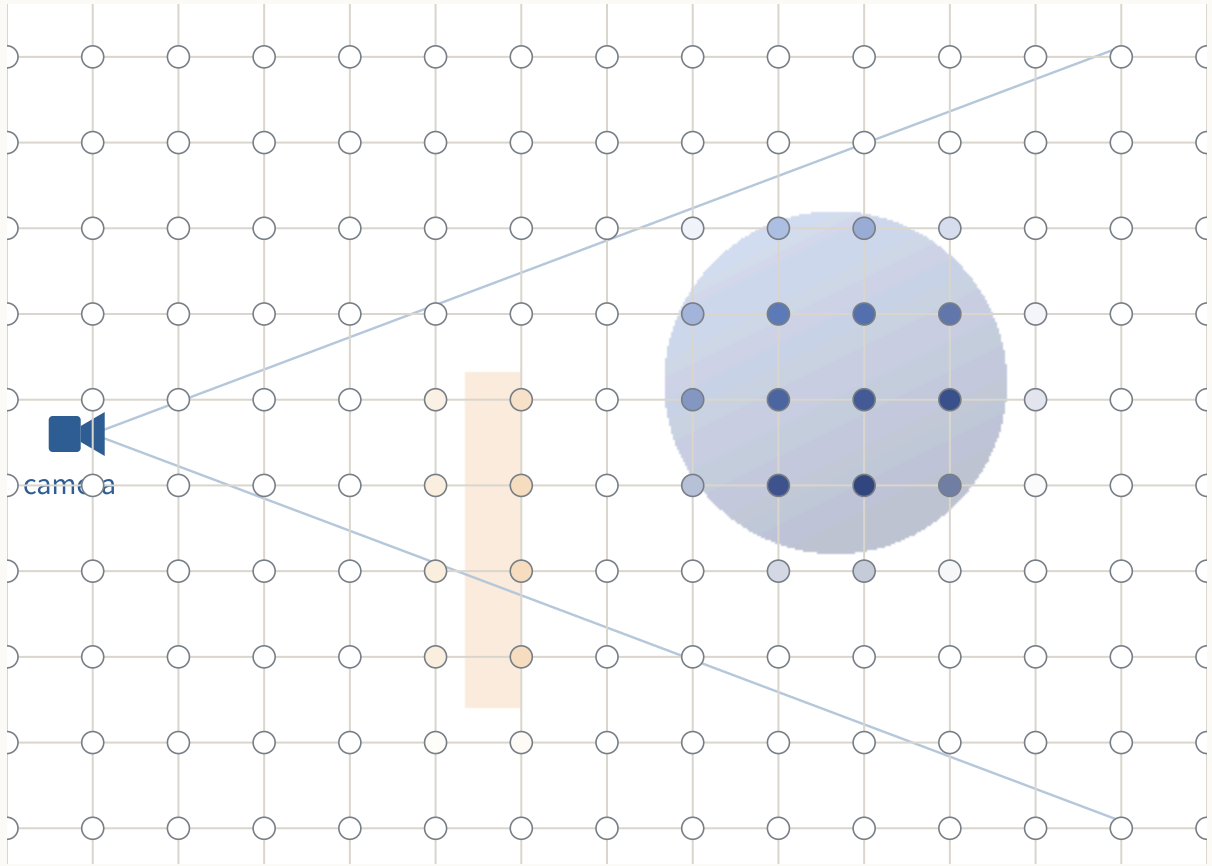


NeRF asks one big network about every point. The whole scene is hidden in its 600,000 weights, and every training step changes all of them.

Instant-NGP stores learned numbers on grids in space. A point reads the few numbers around it, and a tiny network of about 10,000 weights does the rest.

Müller et al. 2022, Sec. 3 and 5.4: one network of 64 neurons for density and one of two layers for color, together about 10,000 weights (Fig. 2). NeRF's network: Mildenhall et al. 2020, Fig. 7, about 594,000 weights.

A grid of learned numbers



Step 1 of 3

Instant-NGP lays a grid over the scene. Every corner stores two learned numbers, drawn here as the color they come to stand for after training. In 3D the grid has cubes, so a cell has 8 corners.

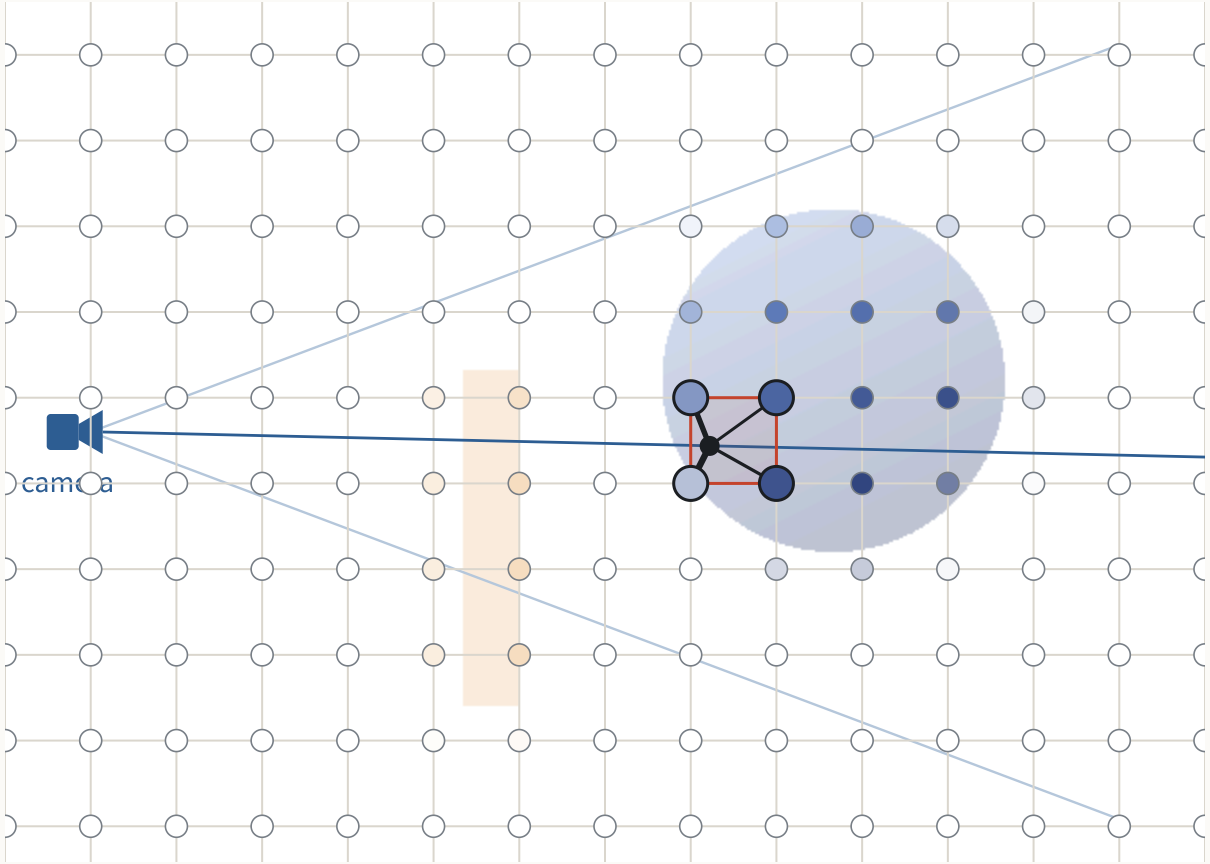
in the air

at the glass

in the ball

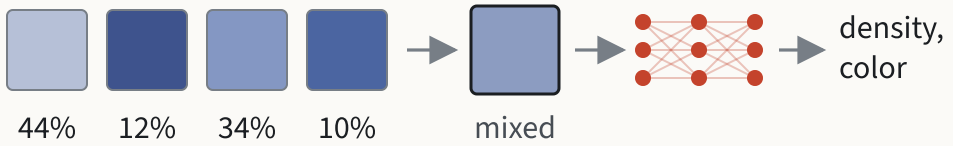
Müller et al. 2022, Sec. 3, Fig. 3: the corner values are blended by (bi/tri)linear interpolation. Each corner stores 2 learned numbers that only the network interprets; they are drawn here as the color they come to stand for.

A grid of learned numbers



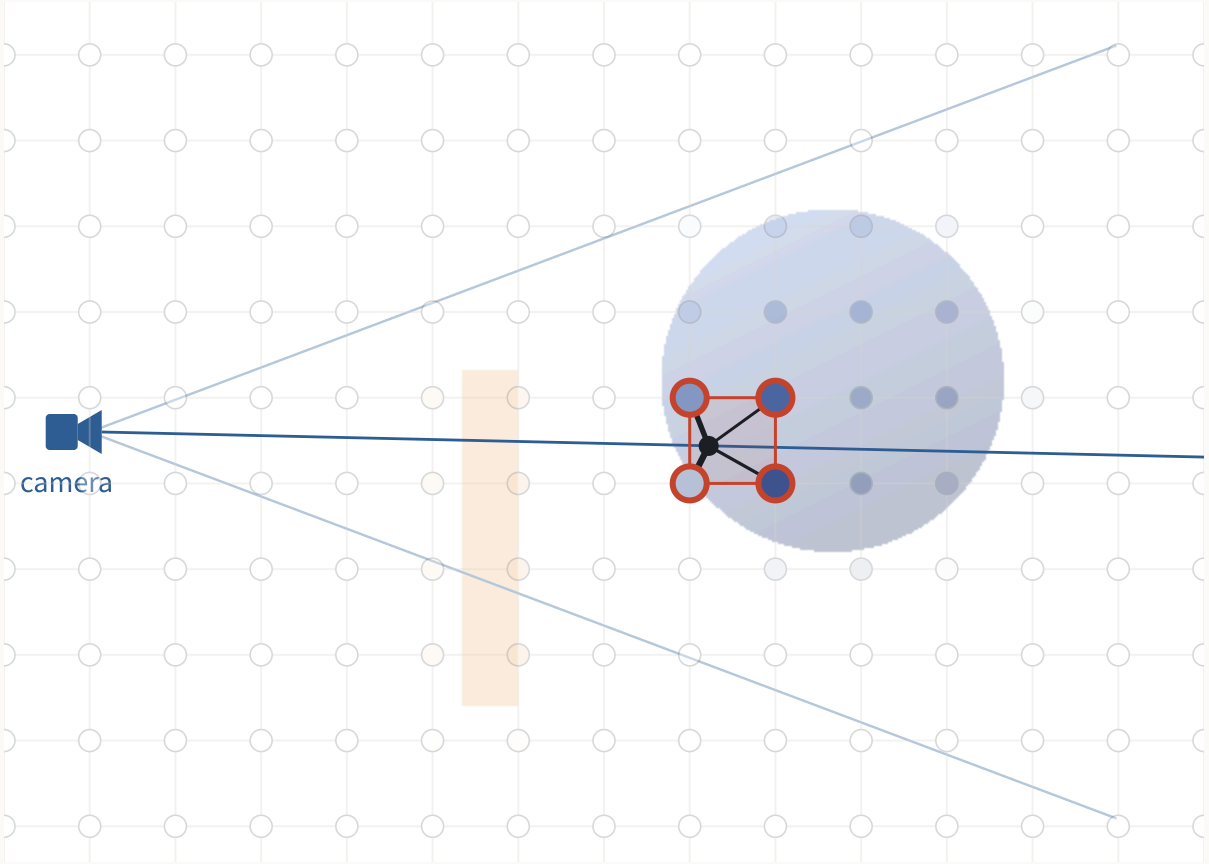
Step 2 of 3

To ask about a point, find its cell and mix the numbers of its corners; the nearer a corner, the more it counts. The tiny network turns the mix into density and color, the answer NeRF's big network gives.



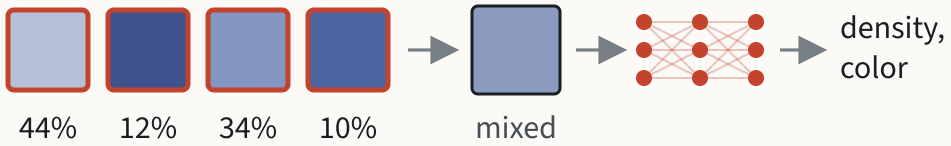
Müller et al. 2022, Sec. 3, Fig. 3: the corner values are blended by (bi/tri)linear interpolation. Each corner stores 2 learned numbers that only the network interprets; they are drawn here as the color they come to stand for.

A grid of learned numbers



Step 3 of 3

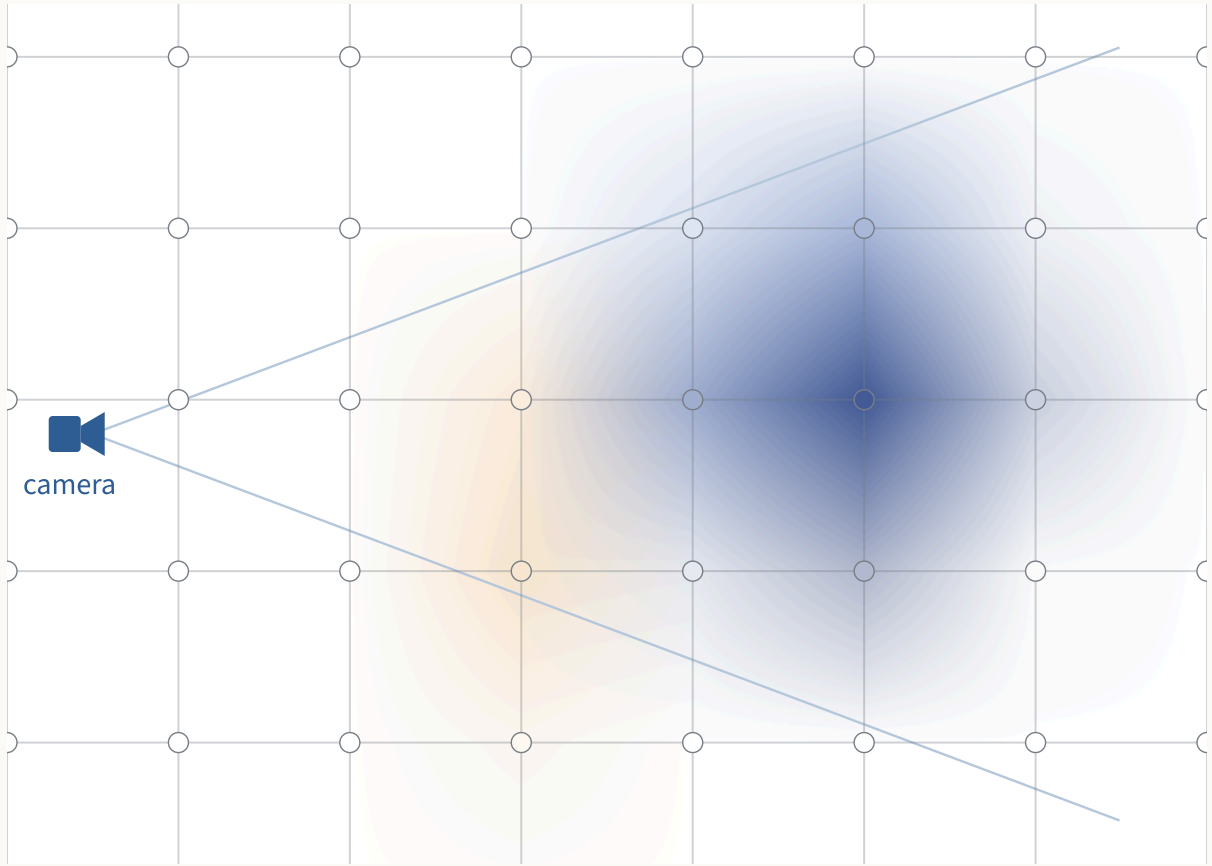
Training compares the pixel with the photo, as in NeRF. The error flows back only into the corners around the points used, here these 4, and into the tiny network. NeRF changes all of its 600,000 weights for every point.



Only these 4 corners and the tiny network change.

Müller et al. 2022, Sec. 3, Fig. 3: the corner values are blended by (bi/tri)linear interpolation. Each corner stores 2 learned numbers that only the network interprets; they are drawn here as the color they come to stand for.

Coarse and fine grids



Step 1 of 3

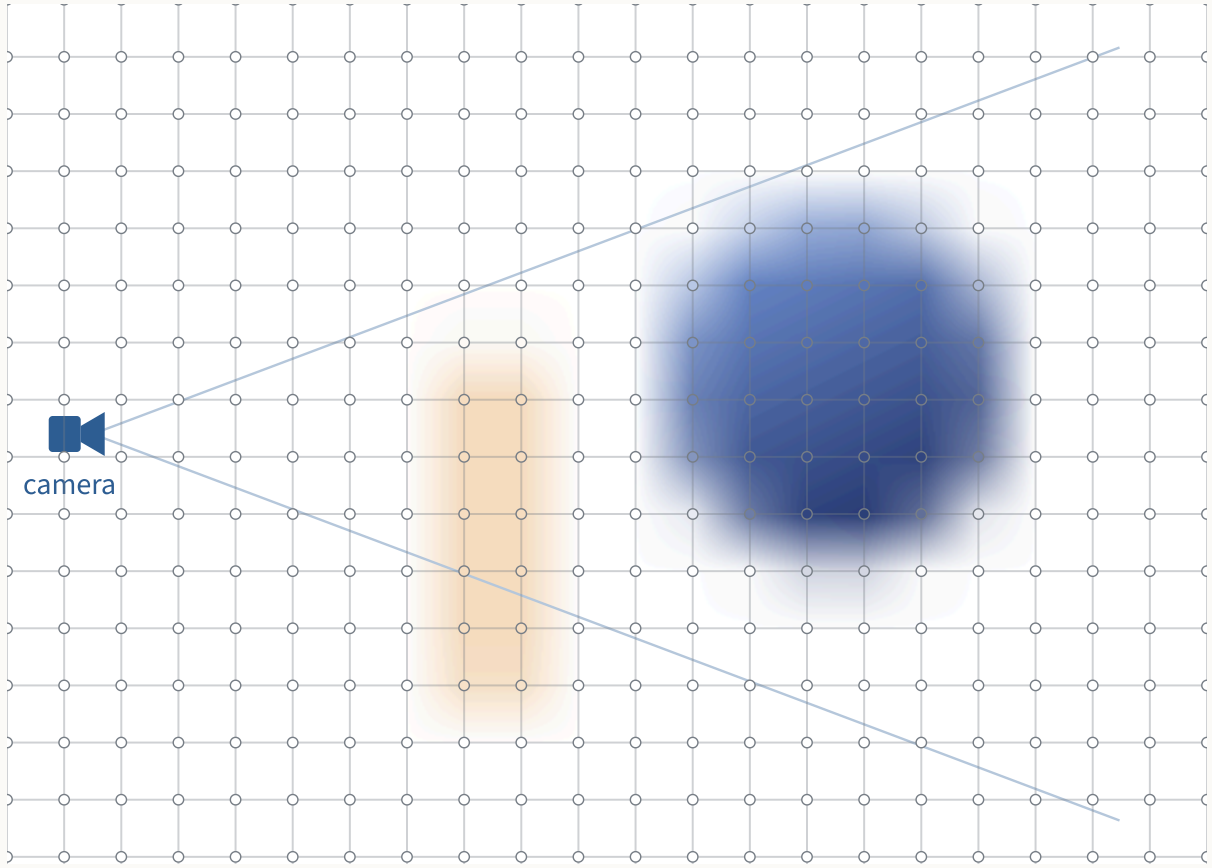
Between its corners a grid only mixes, so it can only show what is larger than its cells. With large cells the ball is a soft blob and the thin glass almost disappears.

Corners of one grid in 3D

16 cells per side	**4,913**
128 cells per side	2.1 million
2,048 cells per side	8.6 billion

Müller et al. 2022, Sec. 3 and Table 1: L = 16 grids, the coarsest with 16 cells per side, each finer one by a constant factor; the values of all 16 are concatenated before the network. Pictures: the toy scene rebuilt from one 2D grid with large, medium and small cells.

Coarse and fine grids



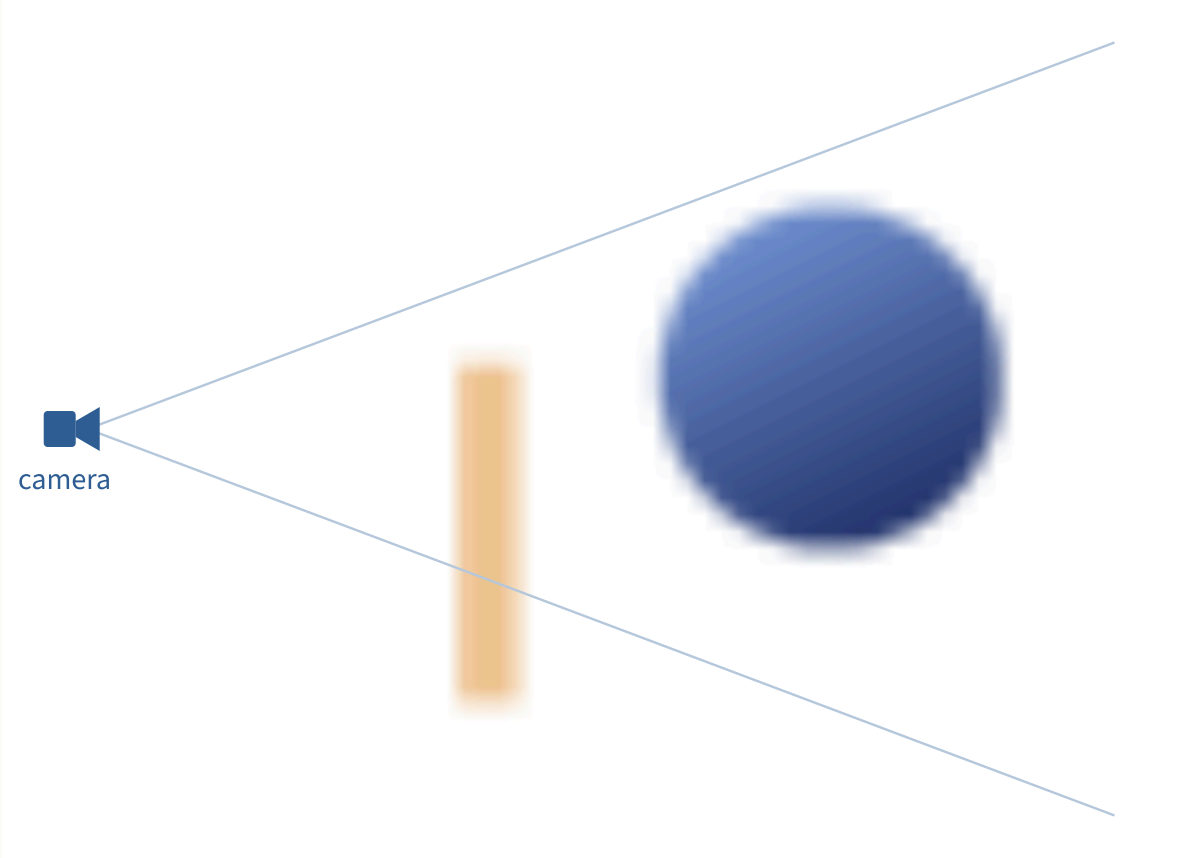
Step 2 of 3

With smaller cells the shapes appear. In 3D, every time the cells get half as big there are 8 times as many corners.

Corners of one grid in 3D	
16 cells per side	4,913
128 cells per side	**2.1 million**
2,048 cells per side	8.6 billion

Müller et al. 2022, Sec. 3 and Table 1: L = 16 grids, the coarsest with 16 cells per side, each finer one by a constant factor; the values of all 16 are concatenated before the network. Pictures: the toy scene rebuilt from one 2D grid with large, medium and small cells.

Coarse and fine grids



Step 3 of 3

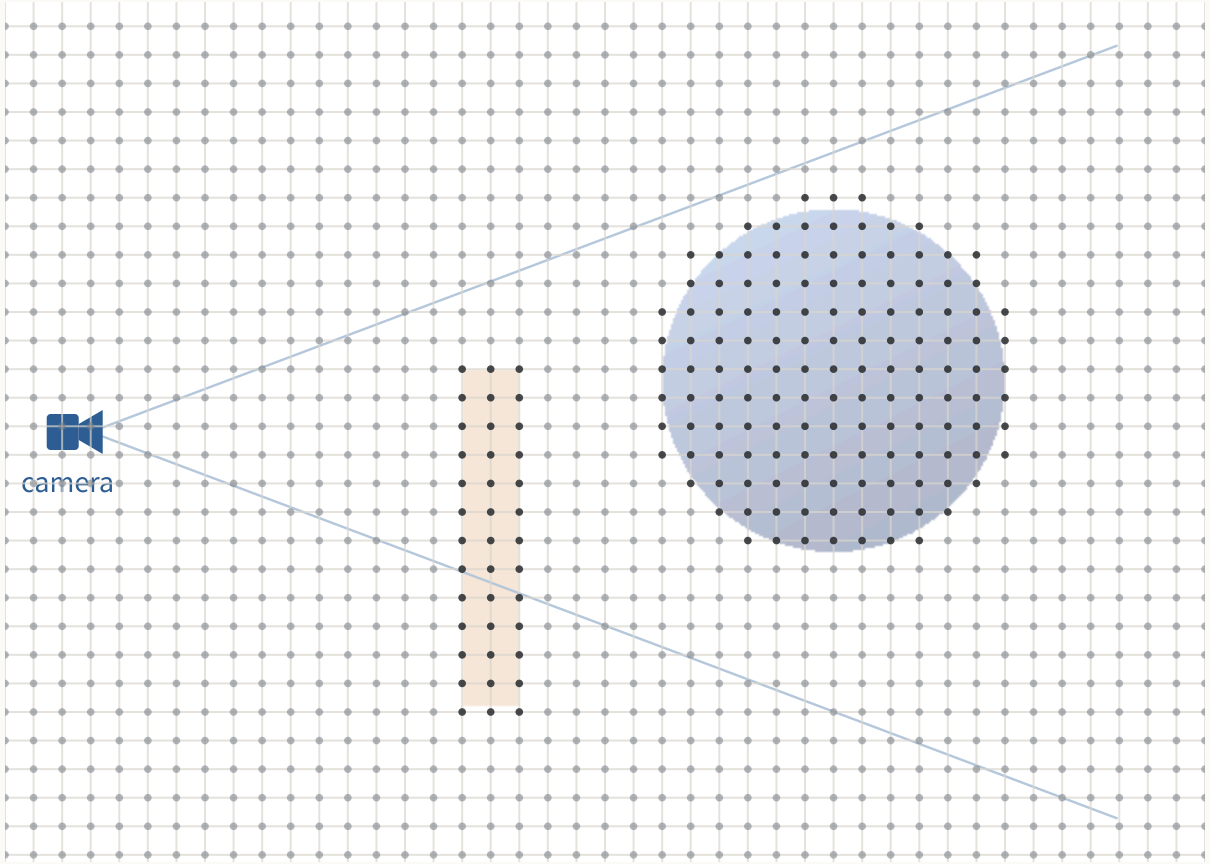
Small cells give a sharp picture, but one such grid of a real scene has billions of corners. Instant-NGP uses 16 grids from coarse to fine and passes what all 16 read to the tiny network: the coarse ones give the shape, the fine ones the detail.

Corners of one grid in 3D

16 cells per side	4,913
128 cells per side	2.1 million
2,048 cells per side	**8.6 billion**

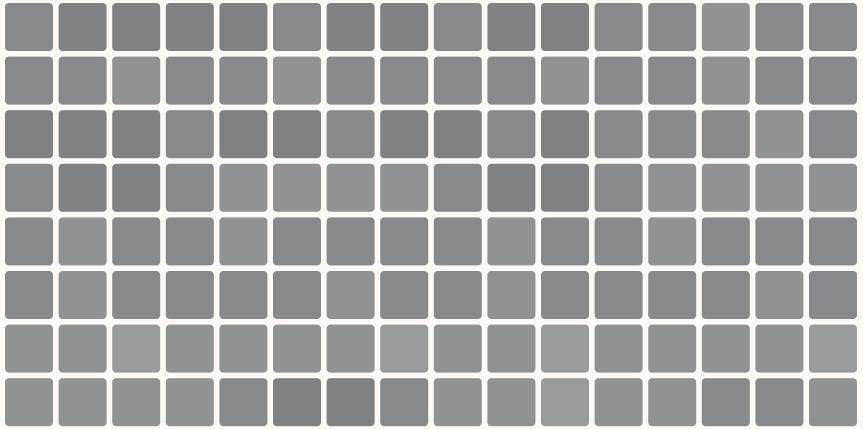
Müller et al. 2022, Sec. 3 and Table 1: L = 16 grids, the coarsest with 16 cells per side, each finer one by a constant factor; the values of all 16 are concatenated before the network. Pictures: the toy scene rebuilt from one 2D grid with large, medium and small cells.

Storing the fine grids in a hash table



Step 1 of 3

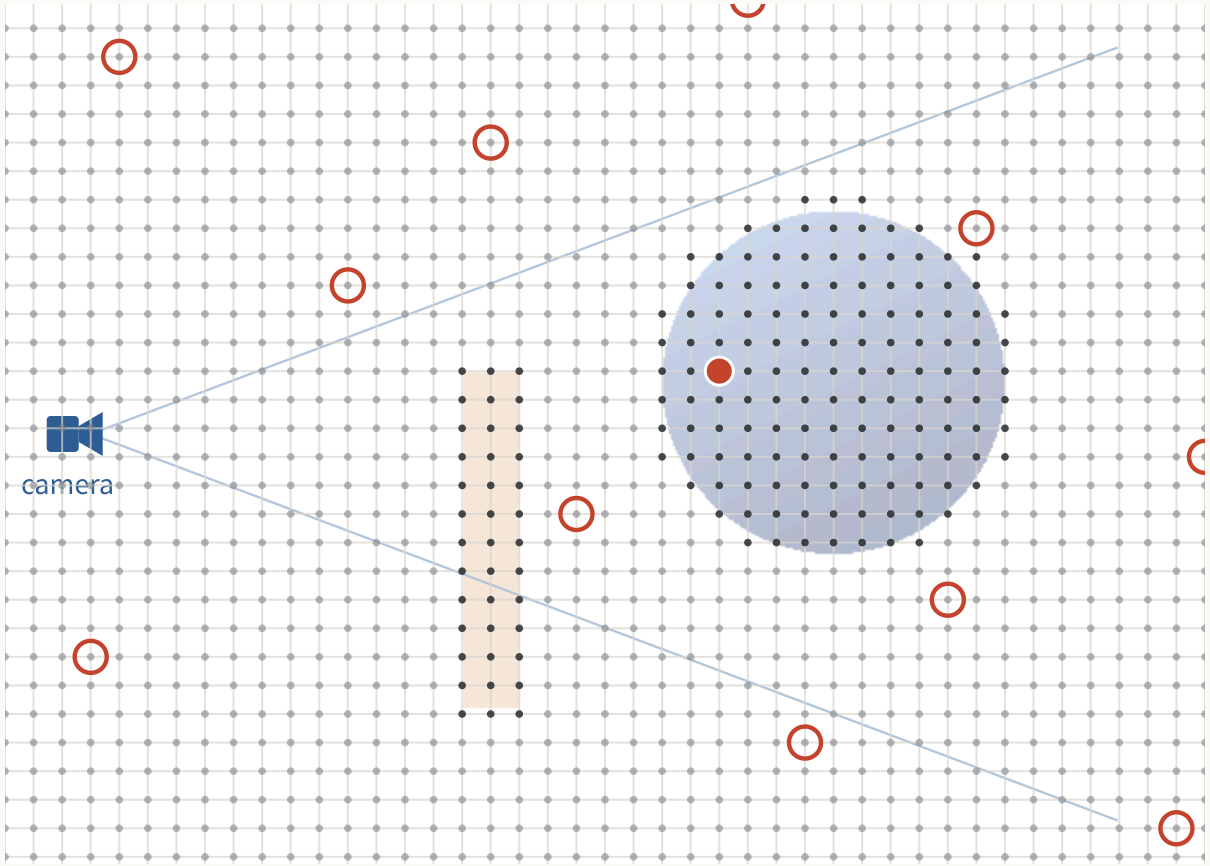
A fine grid has far too many corners to store. Each grid therefore gets a table with a fixed number of slots, and a formula, the hash, sends every corner to a slot. Many corners then share one slot.



1,505 corners, **128** slots: about 11.8 corners per slot.

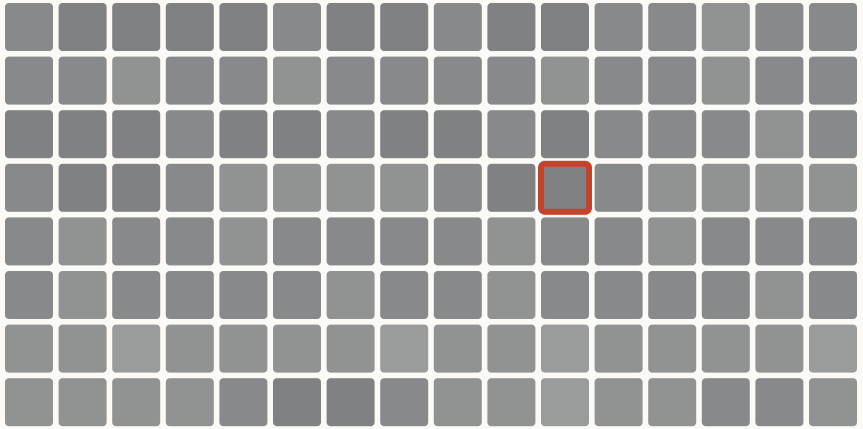
Illustration with 128 slots; the paper uses up to 2^{19} = 524,288 slots per grid, each with 2 learned numbers. Hash function of the paper (Eq. 4), in 2D: corner (i, j) goes to slot $(i \oplus 2{,}654{,}435{,}761\, j) \bmod T$, where $\oplus$ is bitwise XOR.

Storing the fine grids in a hash table



Step 2 of 3

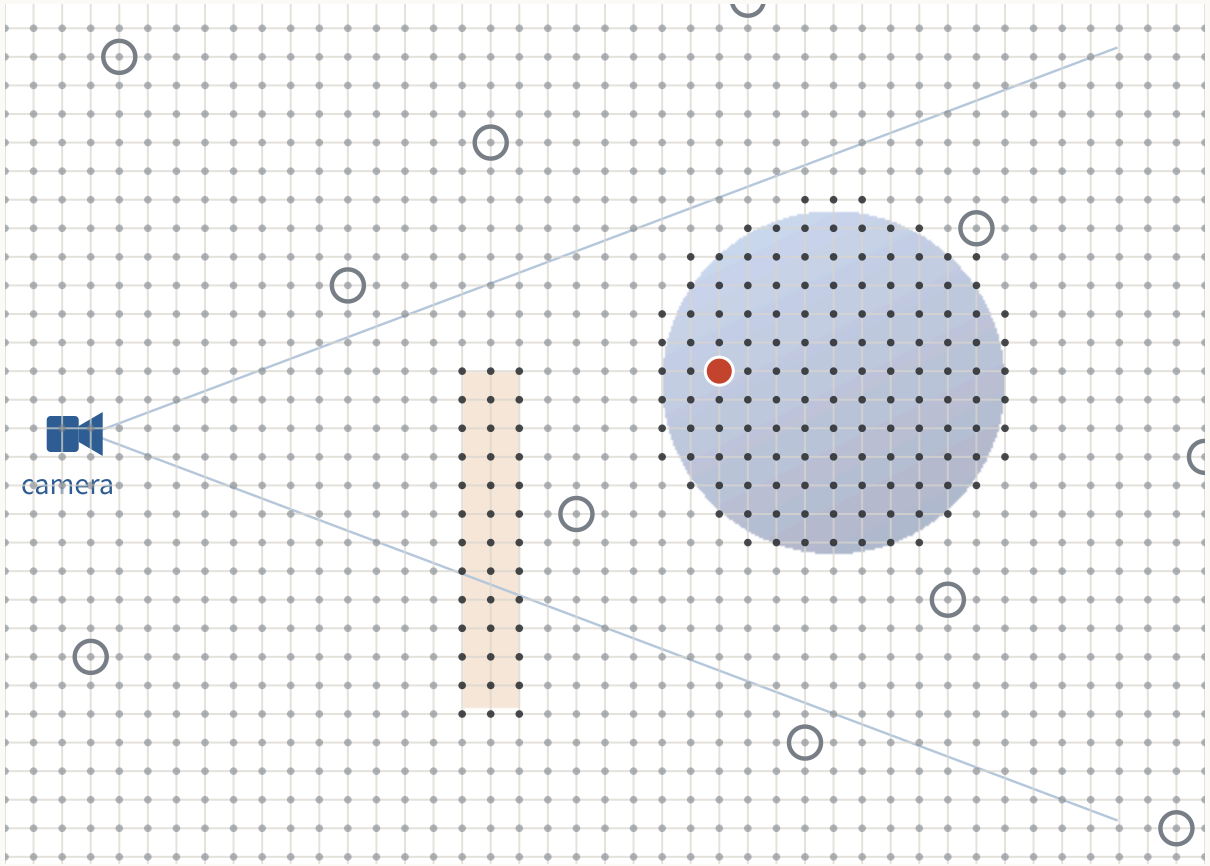
The red corner sits at the front of the ball. The 12 circled corners land in the same slot. They are spread over the whole scene, and all of them read the same two numbers.



13 corners share the red slot.

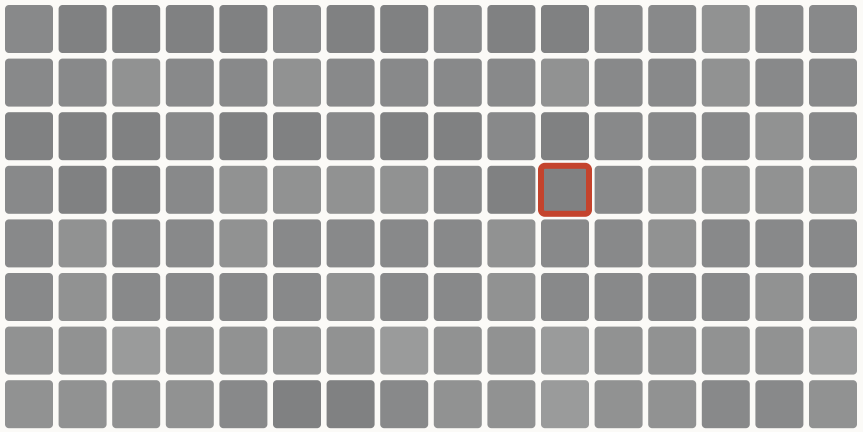
Illustration with 128 slots; the paper uses up to 2^{19} = 524,288 slots per grid, each with 2 learned numbers. Hash function of the paper (Eq. 4), in 2D: corner (i, j) goes to slot $(i \oplus 2{,}654{,}435{,}761\, j) \bmod T$, where $\oplus$ is bitwise XOR.

Storing the fine grids in a hash table



Step 3 of 3

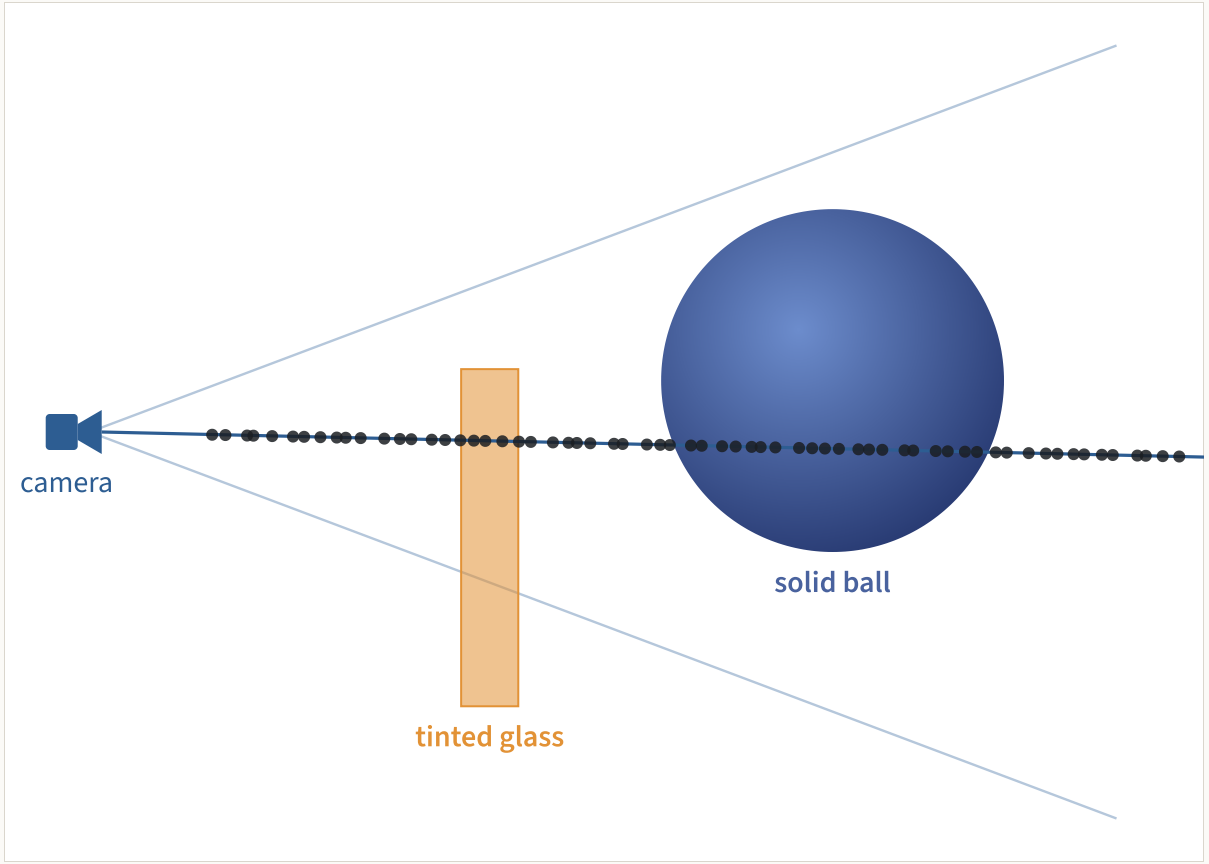
Only the red corner is near something; the others are in empty air, which no photo ever shows. Training fills the slot with what the red corner needs, and the coarse grids, which have no sharing, help the network tell the rest apart.



13 corners share the red slot: **1** near the ball, **12** in empty air.

Illustration with 128 slots; the paper uses up to 2^{19} = 524,288 slots per grid, each with 2 learned numbers. Hash function of the paper (Eq. 4), in 2D: corner (i, j) goes to slot $(i \oplus 2{,}654{,}435{,}761\, j) \bmod T$, where $\oplus$ is bitwise XOR.

Skipping empty space



Step 1 of 2

NeRF spreads its points over the whole ray, 64 and then 192 more, and sends each one through a network of 600,000 weights, even in empty air.

Multiply-adds in the network, per ray

NeRF — **about 152 million**

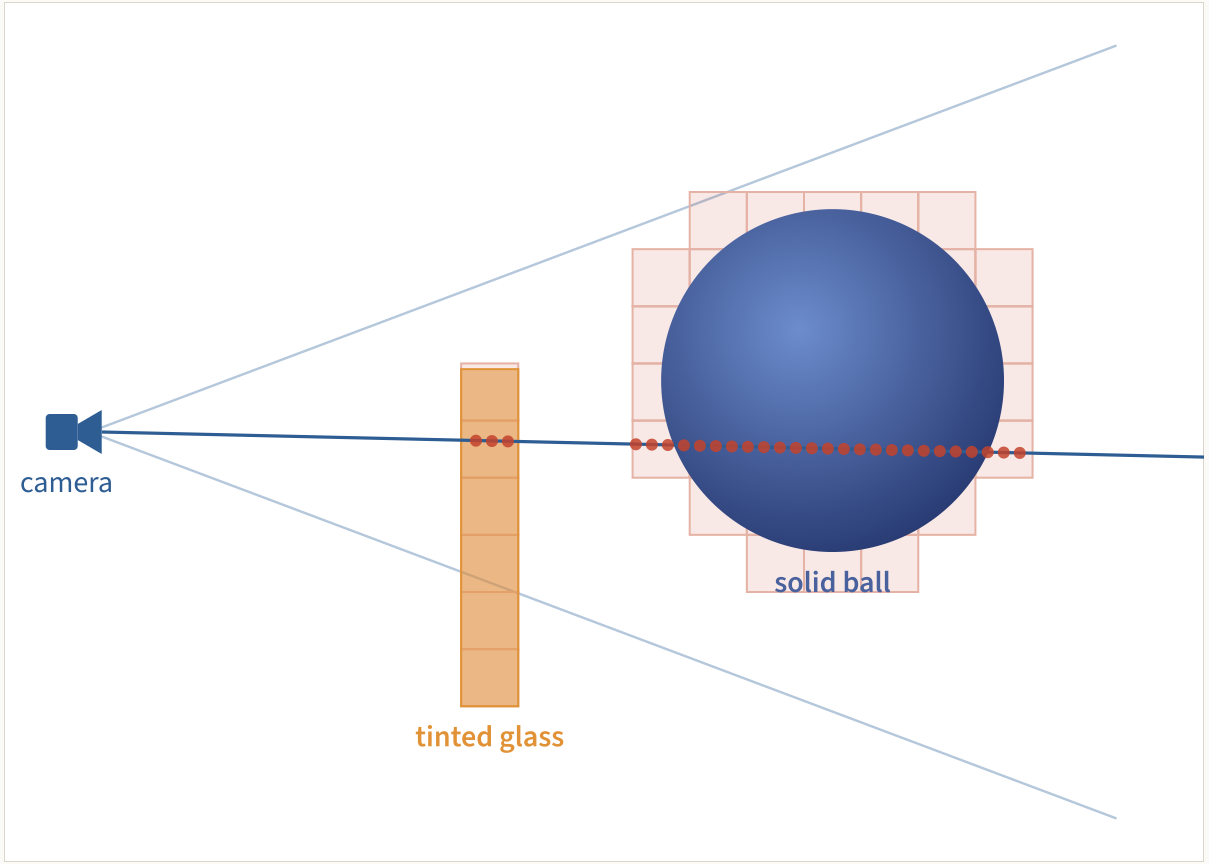
256 points × 594,000 weights

Instant-NGP — **at most about 0.26 million**

3 to 26 points × 10,000 weights

NeRF: 64 + 192 points per ray through networks of about 594,000 weights (Mildenhall et al. 2020, Sec. 5.3, Fig. 7). Instant-NGP: 3.1 to 25.7 points per ray on the NeRF scenes (Müller et al. 2022, Table 3, Sec. 5.4: occupancy grid), about 10,000 weights. Toy scene with occupancy cells of my choosing.

Skipping empty space



Step 2 of 2

Instant-NGP keeps a coarse map of the cells that hold anything, shaded, updated during training. A ray skips the empty cells and takes points only inside shaded ones: 3 to 26 per ray in the paper. With the tiny network that is about 600 times less arithmetic.

Multiply-adds in the network, per ray

NeRF — **about 152 million**

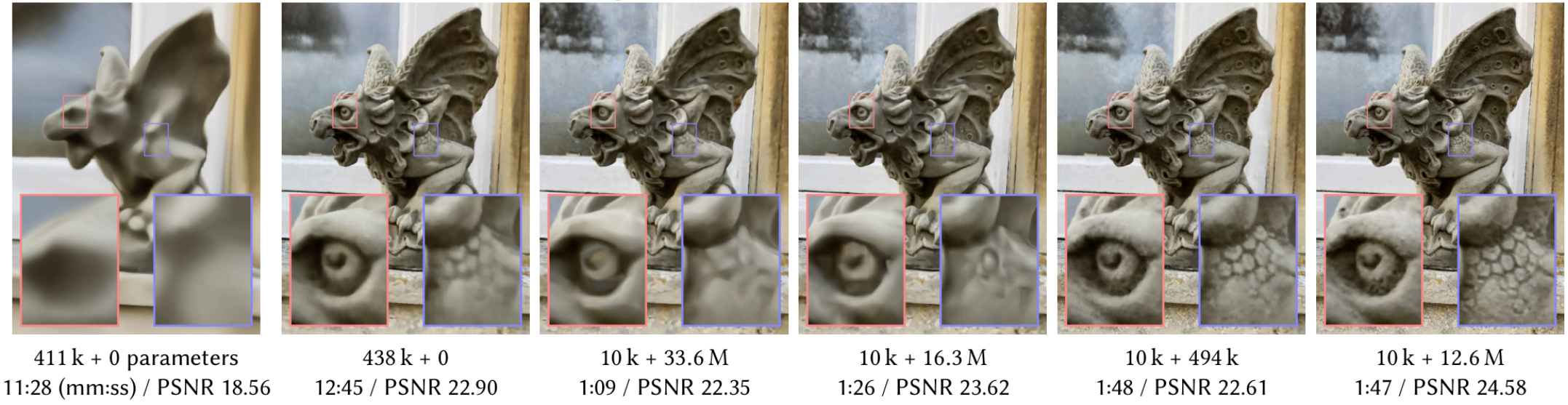
256 points × 594,000 weights

Instant-NGP — **at most about 0.26 million**

3 to 26 points × 10,000 weights

NeRF: 64 + 192 points per ray through networks of about 594,000 weights (Mildenhall et al. 2020, Sec. 5.3, Fig. 7). Instant-NGP: 3.1 to 25.7 points per ray on the NeRF scenes (Müller et al. 2022, Table 3, Sec. 5.4: occupancy grid), about 10,000 weights. Toy scene with occupancy cells of my choosing.

Six encodings of the same scene

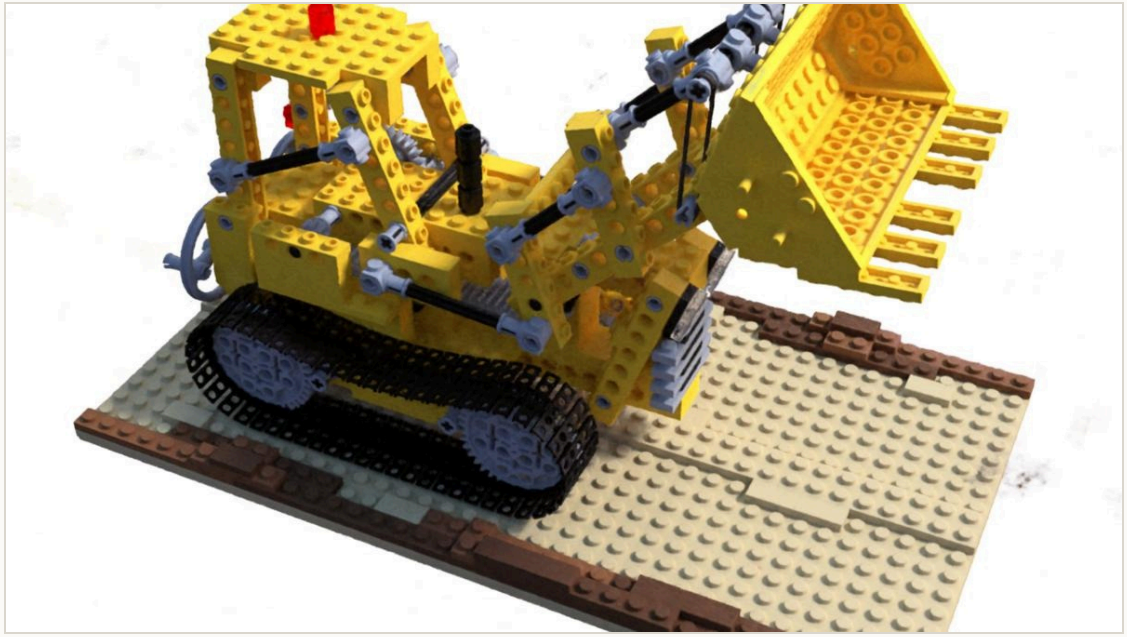


The paper trains the same scene six ways for 11,000 steps each. NeRF's sines (b) need a network of 438,000 weights and almost 13 minutes.

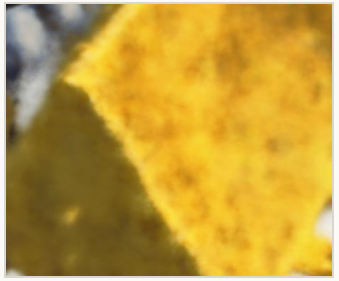
Grids with a tiny network (c, d) train in about a minute but need 16 to 34 million numbers. The hash tables (e, f) keep the speed, cap the memory, and the larger one gives the sharpest image, 24.6 dB.

Müller et al. 2022, Fig. 2. Under each image: network weights + grid or table numbers, training time (minutes:seconds) and PSNR.

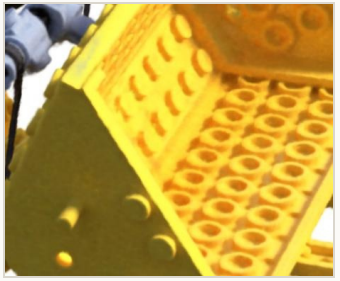
The same scene after 1, 15 and 60 seconds



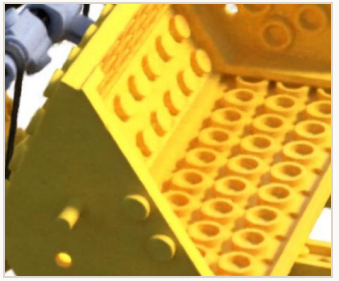
The whole view after 15 seconds of training



1 second



15 seconds



60 seconds

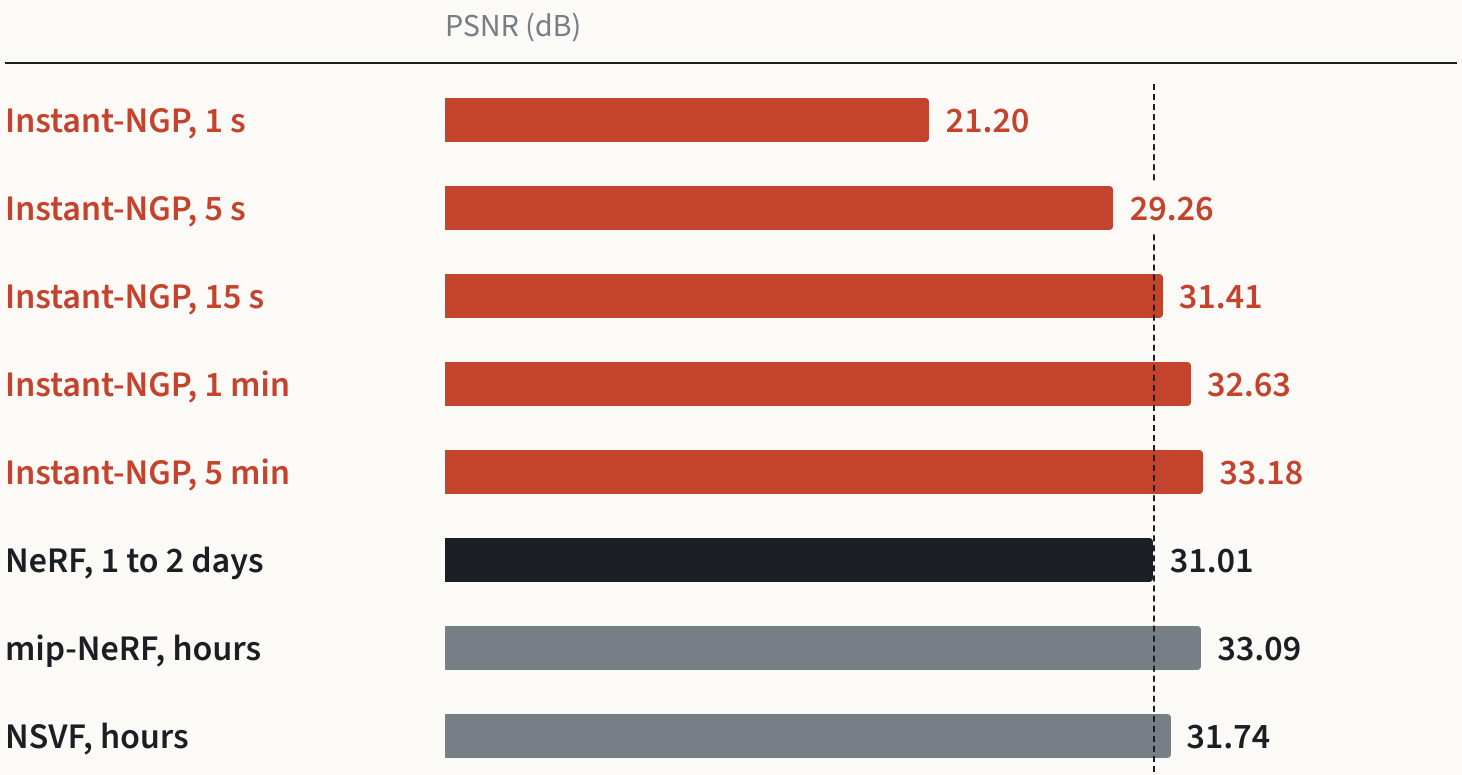
The Lego scene trained from scratch on one RTX 3090 graphics card. After one second the bucket is a yellow blur; after 15 seconds it is sharp, and a minute adds the last details.

NeRF needed one to two days for the same scene on a V100.

Müller et al. 2022, Fig. 1 (NeRF row): a zoom on the bucket of the Lego scene at three training times. NeRF: Mildenhall et al. 2020, Sec. 5.3.

Instant-NGP against NeRF

Training time | Same code, NeRF's encoding



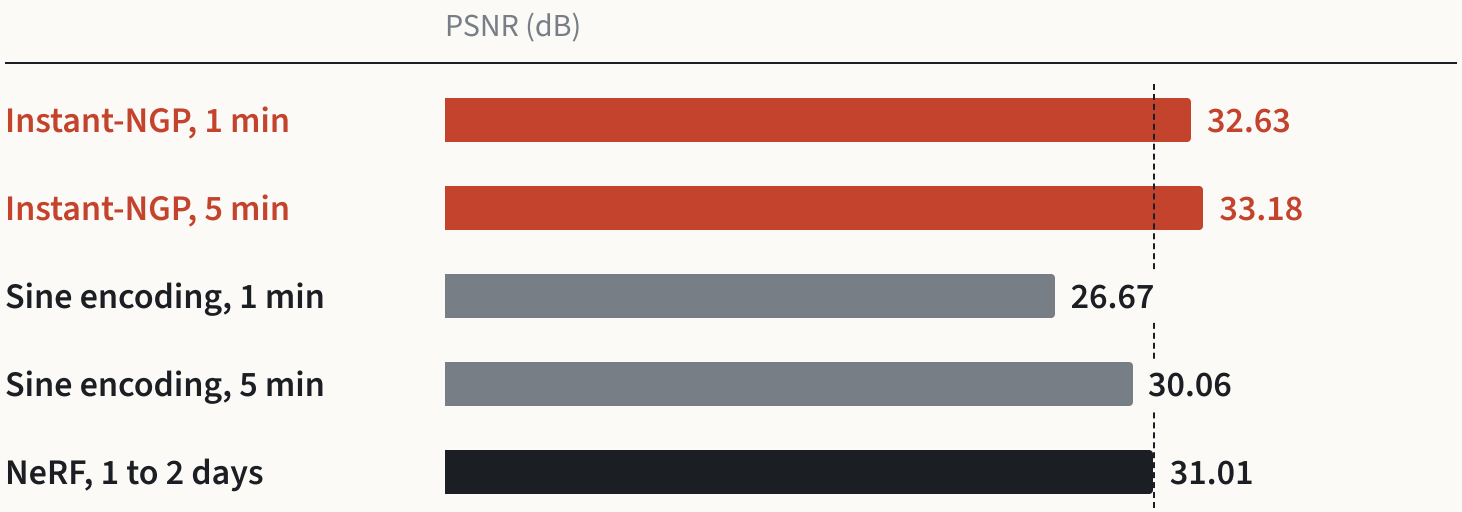
After 15 seconds of training, the average over the eight synthetic scenes passes NeRF's 31.0 dB, which took one to two days.

After five minutes it matches mip-NeRF, the best method of the time, which also trains for hours.

Müller et al. 2022, Table 2: mean PSNR over the eight Realistic Synthetic 360° scenes from the NeRF paper. NeRF, mip-NeRF and NSVF values from their papers; the dashed line is NeRF, 31.01 dB.

Instant-NGP against NeRF

Training time | **Same code, NeRF's encoding**



To see whether the speed comes from the code or from the encoding, the authors ran NeRF's sine encoding and big network in the same fast code. Five minutes give only 30.1 dB.

The authors credit the hash encoding itself with a 20 to 60 times speed-up.

Müller et al. 2022, Table 2: mean PSNR over the eight Realistic Synthetic 360° scenes from the NeRF paper. NeRF, mip-NeRF and NSVF values from their papers; the dashed line is NeRF, 31.01 dB.

What Instant-NGP stores

5 MB
NeRF

25 MB
Instant-NGP

Step 1 of 2

Instant-NGP stores more than NeRF: 12.6 million learned numbers, ten times NeRF's 1.2 million weights, about 25 MB. NeRF's weights are shared by every point in space, while a table slot only describes the region around its corners, so many more are needed. Instant-NGP trades memory for speed.

Instant-NGP: 12.6 million numbers with $T = 2^{19}$ (Müller et al. 2022, Fig. 2f), 2 bytes each in half precision; my own Instant-NGP file of the globe is 24 MB. NeRF 5 MB and LLFF over 15 GB: Mildenhall et al. 2020, Sec. 6.3. Dense grid: $2{,}048^3$ corners × 2 numbers × 2 bytes. Squares are sized by area.

What Instant-NGP stores

5 MB
NeRF

25 MB
Instant-NGP

over 15 GB
LLFF, before NeRF

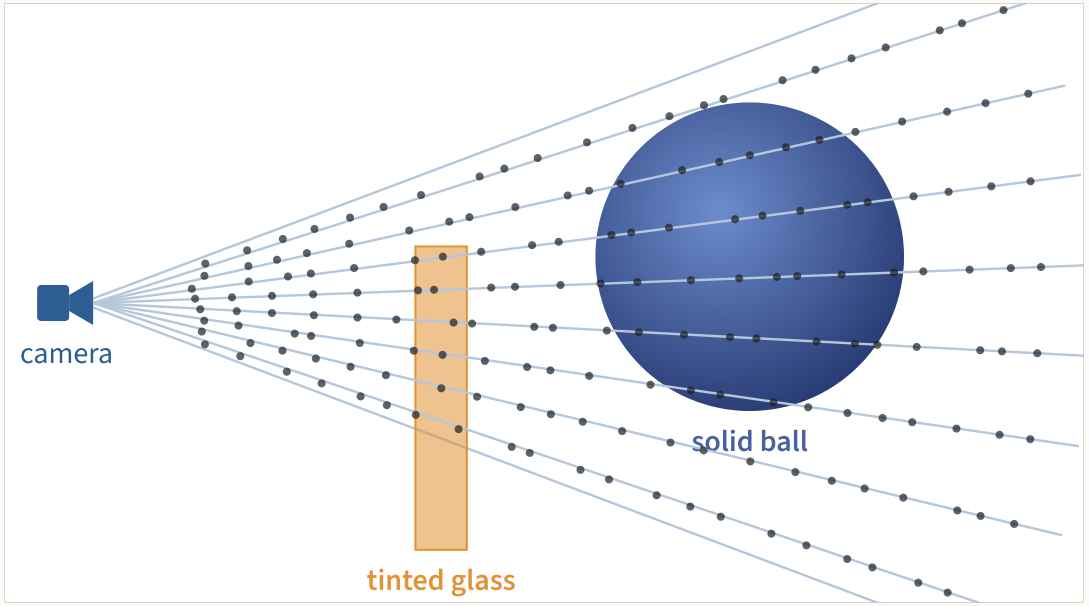
34 GB
one dense grid, 2,048 per side

Step 2 of 2

It is still small next to the older ways of storing a scene. LLFF keeps a stack of image layers for every view and needs over 15 GB, and one dense grid with 2,048 cells per side would need about 34 GB. The hash tables are what keep Instant-NGP at 25 MB.

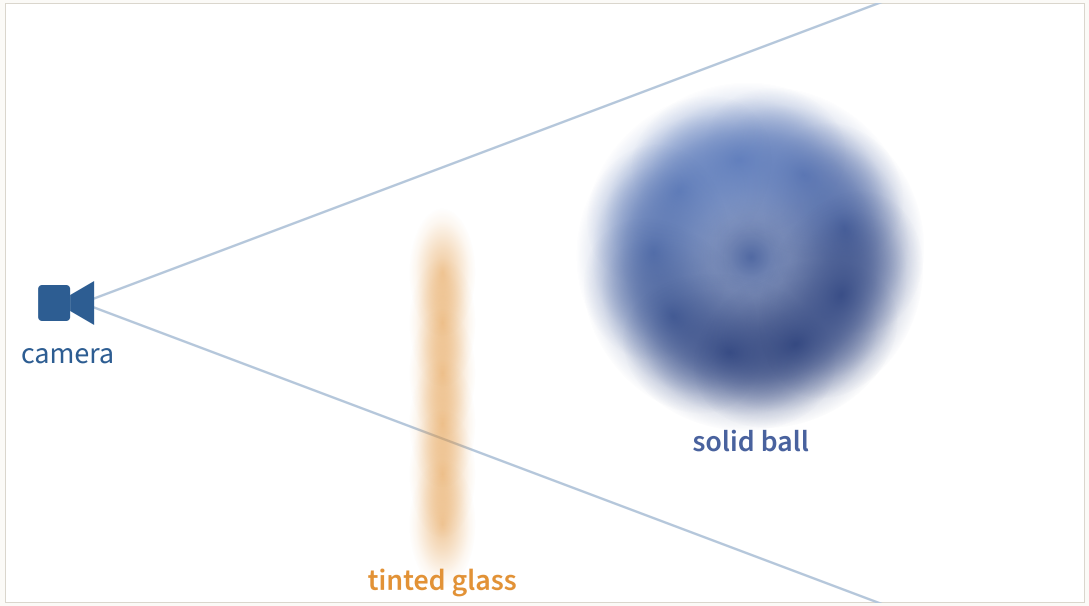
Instant-NGP: 12.6 million numbers with $T = 2^{19}$ (Müller et al. 2022, Fig. 2f), 2 bytes each in half precision; my own Instant-NGP file of the globe is 24 MB. NeRF 5 MB and LLFF over 15 GB: Mildenhall et al. 2020, Sec. 6.3. Dense grid: $2{,}048^3$ corners × 2 numbers × 2 bytes. Squares are sized by area.

The idea of Gaussian splatting



NeRF and Instant-NGP: ask about points along every ray

NeRF and Instant-NGP ask what is at points along every ray, and most of those points are empty air.

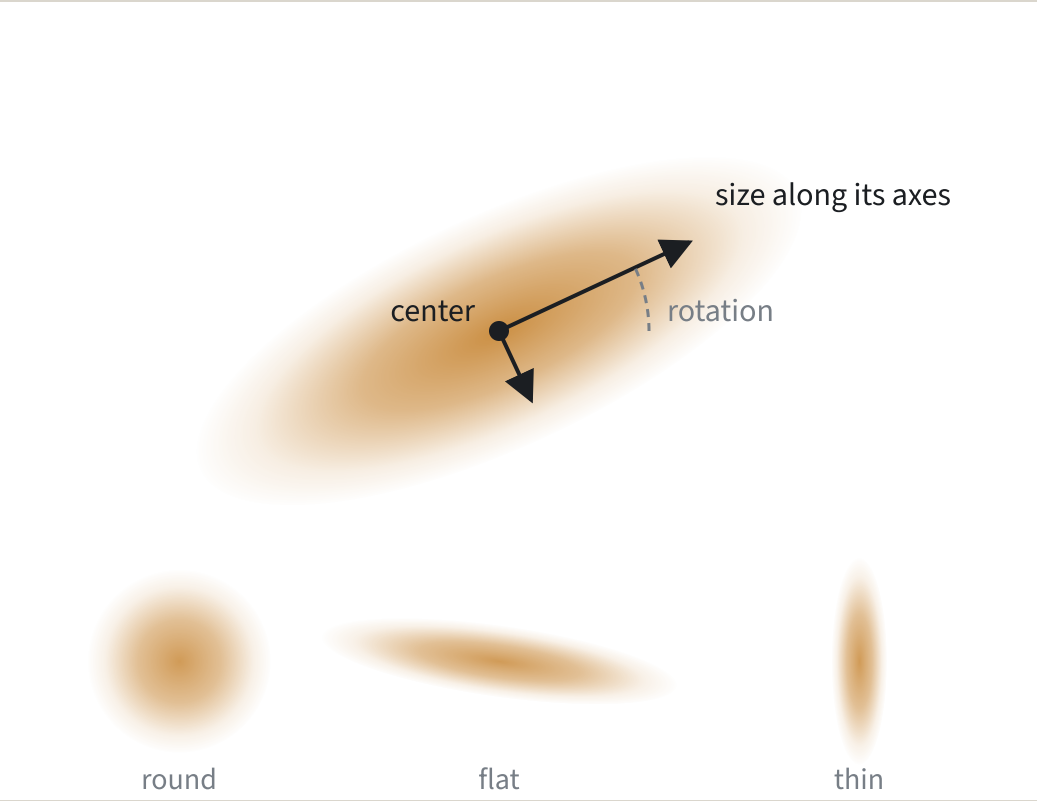


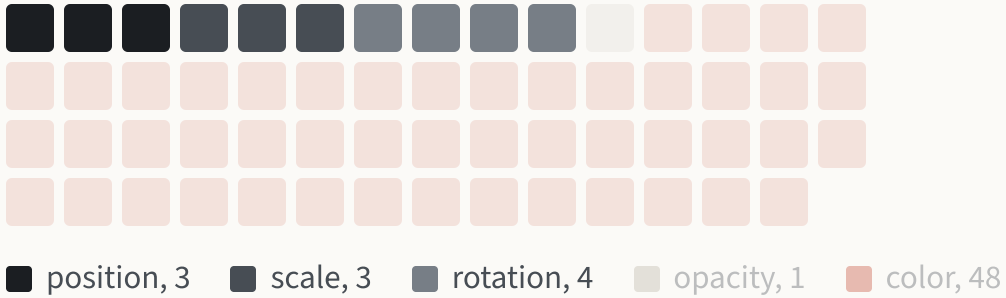
Gaussian splatting: keep a list of soft blobs and draw them

Gaussian splatting stores only where something is: a list of soft, colored, see-through blobs called 3D Gaussians. An image is made by drawing them onto the screen, with no network and no rays.

Kerbl et al. 2023, Sec. 1 and 4. Toy scene as in the NeRF part, here made of 16 Gaussians; a real scene has 1 to 5 million.

What one Gaussian stores





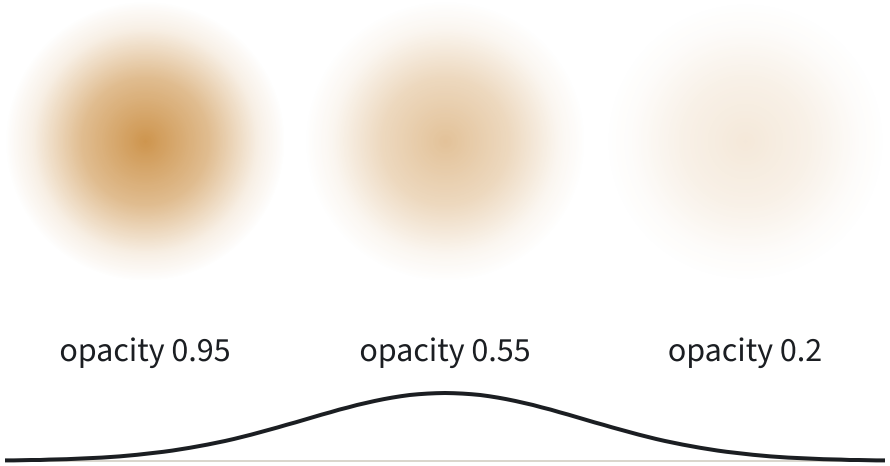
Step 1 of 3

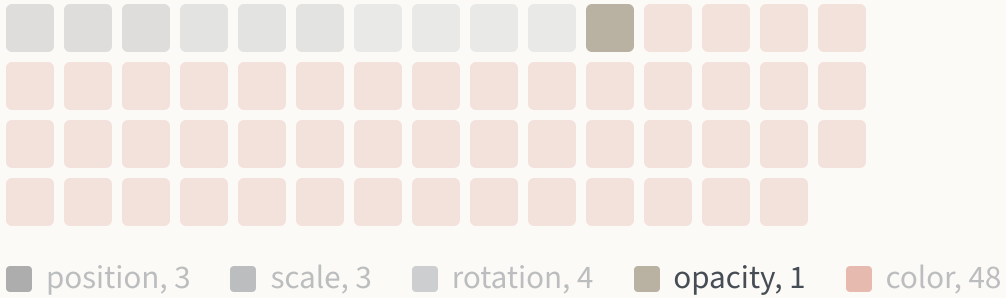
A Gaussian is a soft ellipsoid. Its center is 3 numbers. Its shape is 3 sizes along its own axes and a rotation of 4 numbers, so it can be a round blob, a flat disc or a thin needle.

> 59 numbers per Gaussian, and 1 to 5 million Gaussians per scene.

Kerbl et al. 2023, Sec. 4: the shape is a covariance $\Sigma = R\,S\,S^{\mathsf{T}}\,R^{\mathsf{T}}$, stored as a scale vector and a rotation quaternion; color as spherical harmonics up to degree 3 (16 per channel).

What one Gaussian stores





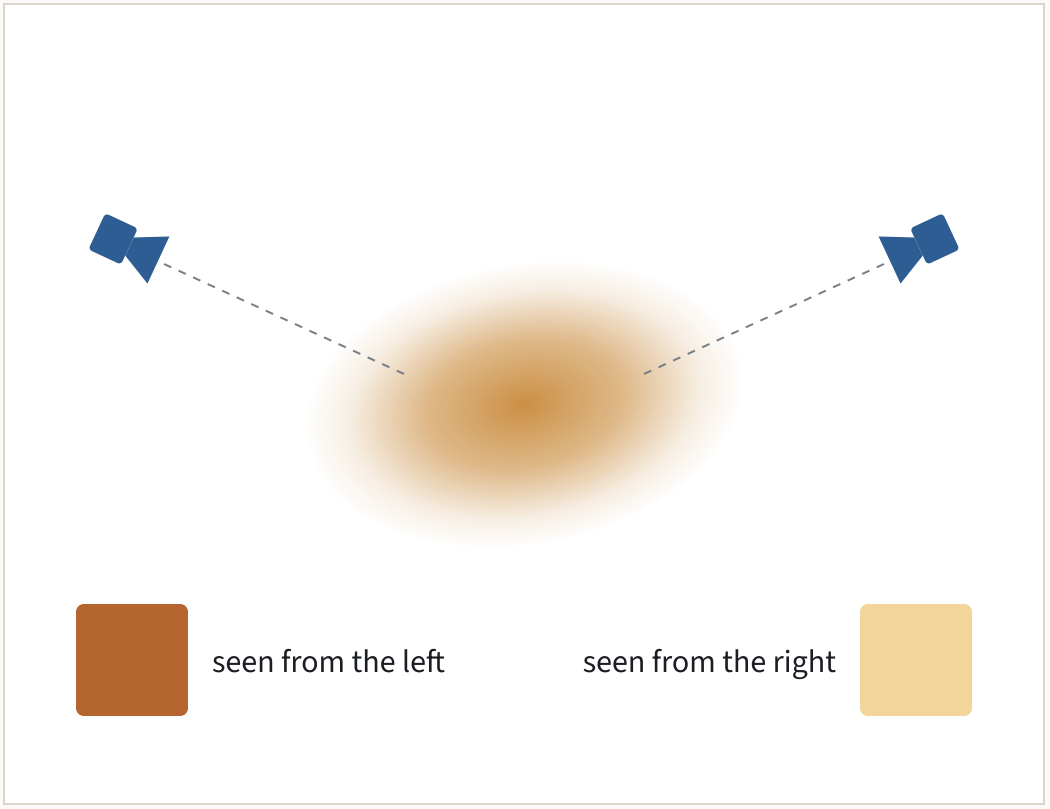
Step 2 of 3

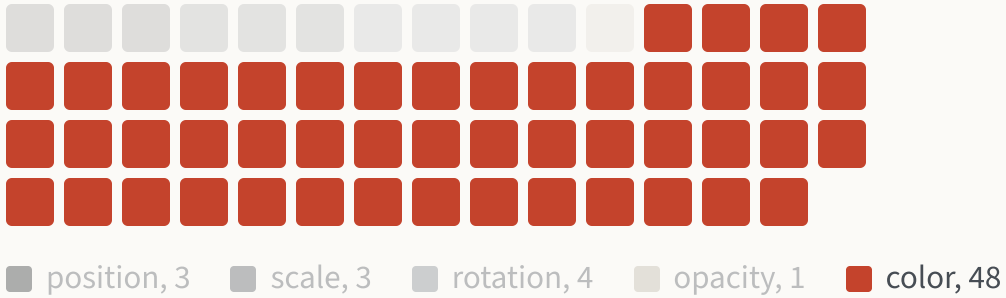
One number says how much light it blocks at its center. The blocking fades smoothly toward the edges, so neighboring Gaussians blend into each other.

> 59 numbers per Gaussian, and 1 to 5 million Gaussians per scene.

Kerbl et al. 2023, Sec. 4: the shape is a covariance $\Sigma = R\,S\,S^{\mathsf{T}}\,R^{\mathsf{T}}$, stored as a scale vector and a rotation quaternion; color as spherical harmonics up to degree 3 (16 per channel).

What one Gaussian stores





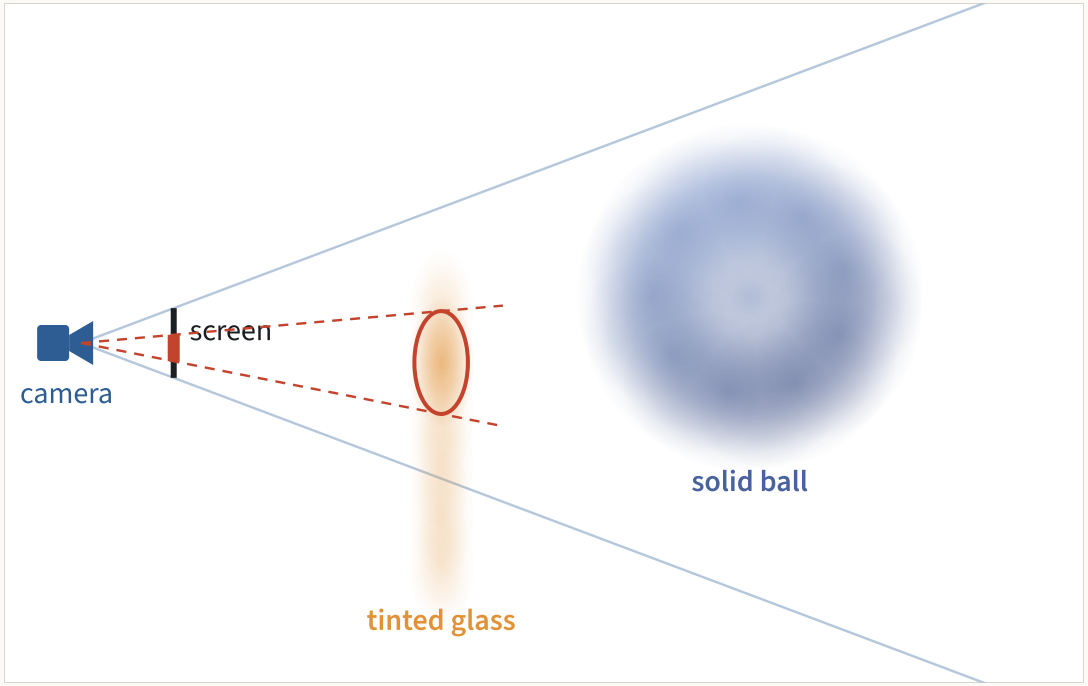
Step 3 of 3

Its color can change with the viewing direction, as in NeRF. It is stored as 16 spherical-harmonic coefficients per color channel: 48 numbers, more than everything else together.

> 59 numbers per Gaussian, and 1 to 5 million Gaussians per scene.

Kerbl et al. 2023, Sec. 4: the shape is a covariance $\Sigma = R\,S\,S^{\mathsf{T}}\,R^{\mathsf{T}}$, stored as a scale vector and a rotation quaternion; color as spherical harmonics up to degree 3 (16 per channel).

Projecting the Gaussians onto the screen



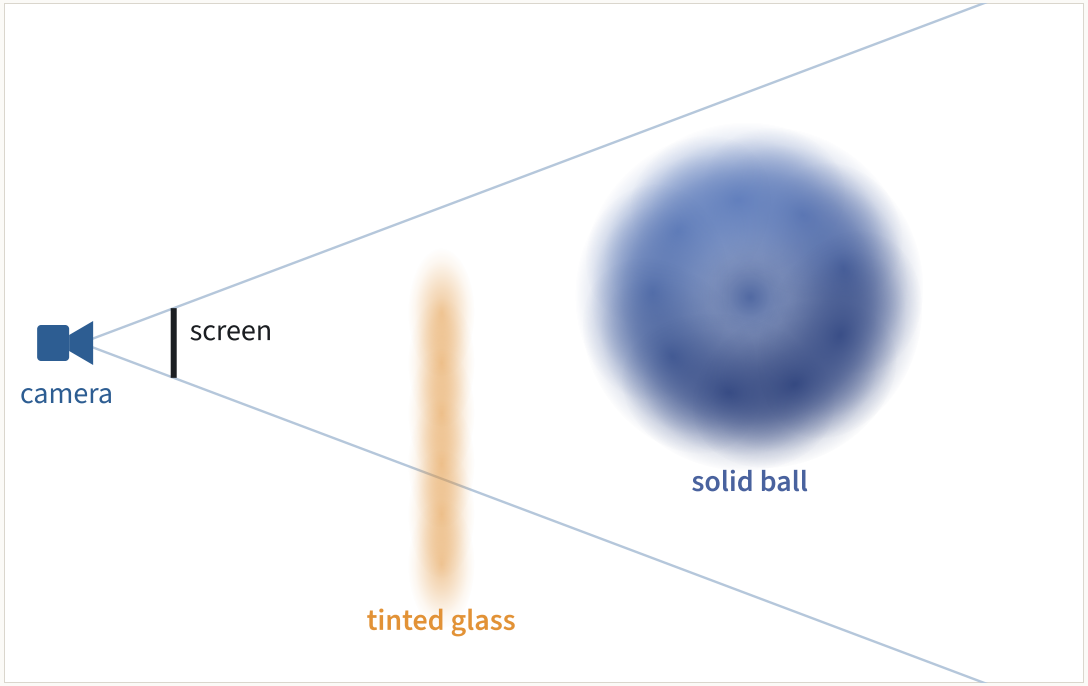
Step 1 of 3

One Gaussian on the screen: a soft bump, the splat

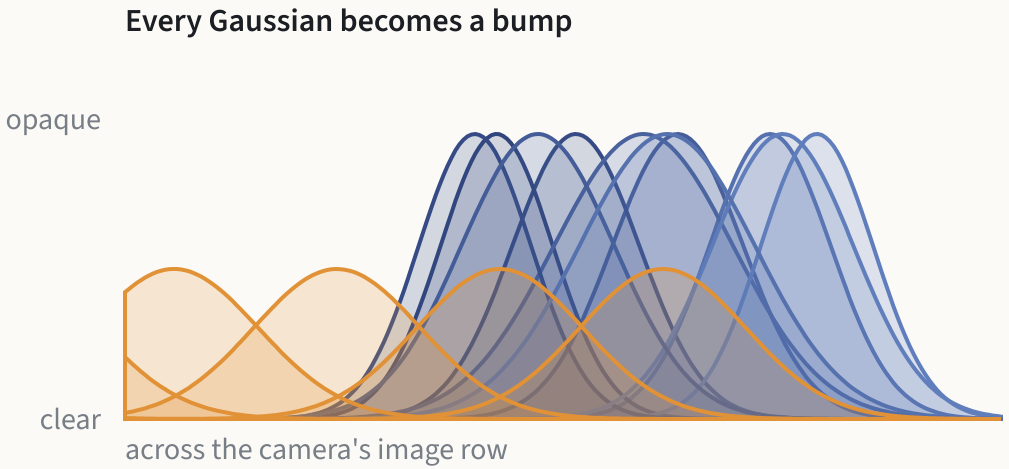
Each Gaussian is projected onto the camera's screen, where it becomes a soft 2D patch, the splat. Our toy scene is flat, so the screen is one row of pixels and the splat is a bump: tall where the Gaussian blocks much light, wide when it is near or large.

Kerbl et al. 2023, Sec. 4 and 6: each 3D Gaussian is projected to a 2D Gaussian on the screen (the splat), using the Jacobian of the projection. The toy scene is 2D, so the screen is one row of pixels and each splat is a 1D bump.

Projecting the Gaussians onto the screen



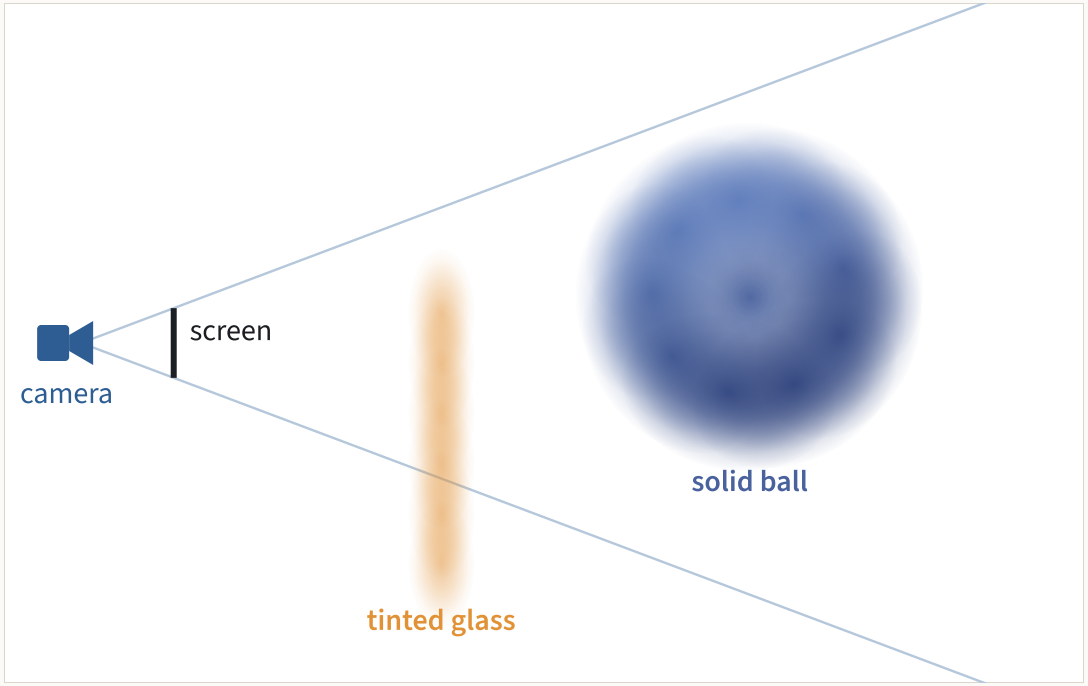
Step 2 of 3



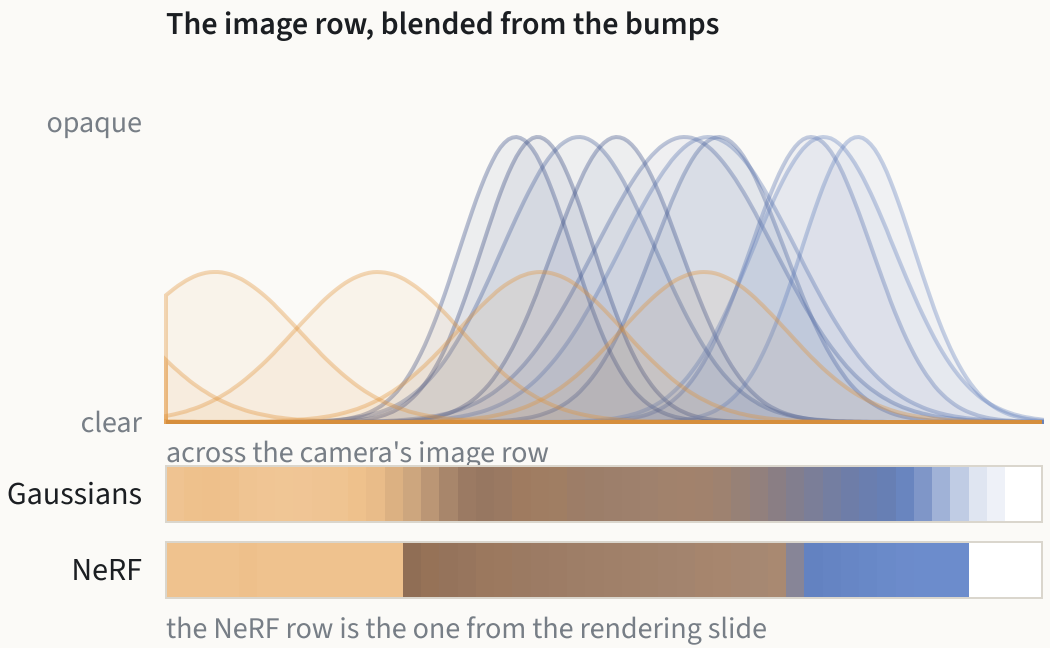
Every Gaussian is projected the same way, each one on its own, like the triangles of a video game. Graphics cards are built to do this for millions of them per frame.

Kerbl et al. 2023, Sec. 4 and 6: each 3D Gaussian is projected to a 2D Gaussian on the screen (the splat), using the Jacobian of the projection. The toy scene is 2D, so the screen is one row of pixels and each splat is a 1D bump.

Projecting the Gaussians onto the screen



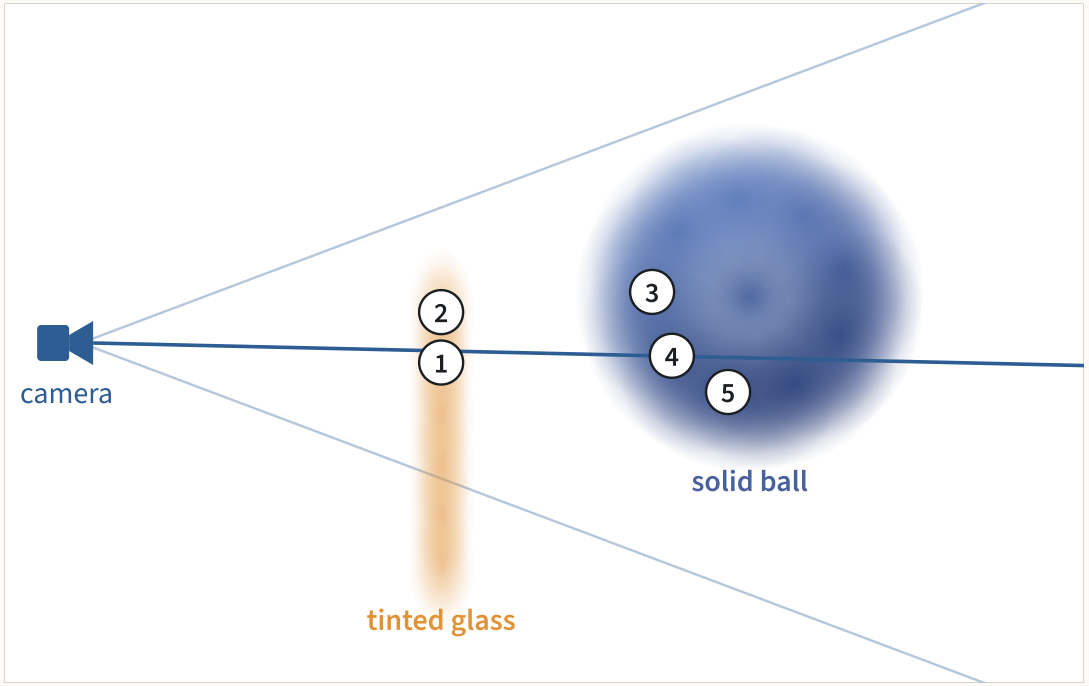
Step 3 of 3



Blending the bumps at every pixel gives the image row. It shows the same glass, ball and background as NeRF's row, with softer edges, but no network was asked and no ray was marched.

Kerbl et al. 2023, Sec. 4 and 6: each 3D Gaussian is projected to a 2D Gaussian on the screen (the splat), using the Jacobian of the projection. The toy scene is 2D, so the screen is one row of pixels and each splat is a 1D bump.

Rendering one pixel from the Gaussians



nearest first	opacity α	
1		0.45
2		0.16
3		0.32
4		0.95
5		0.52

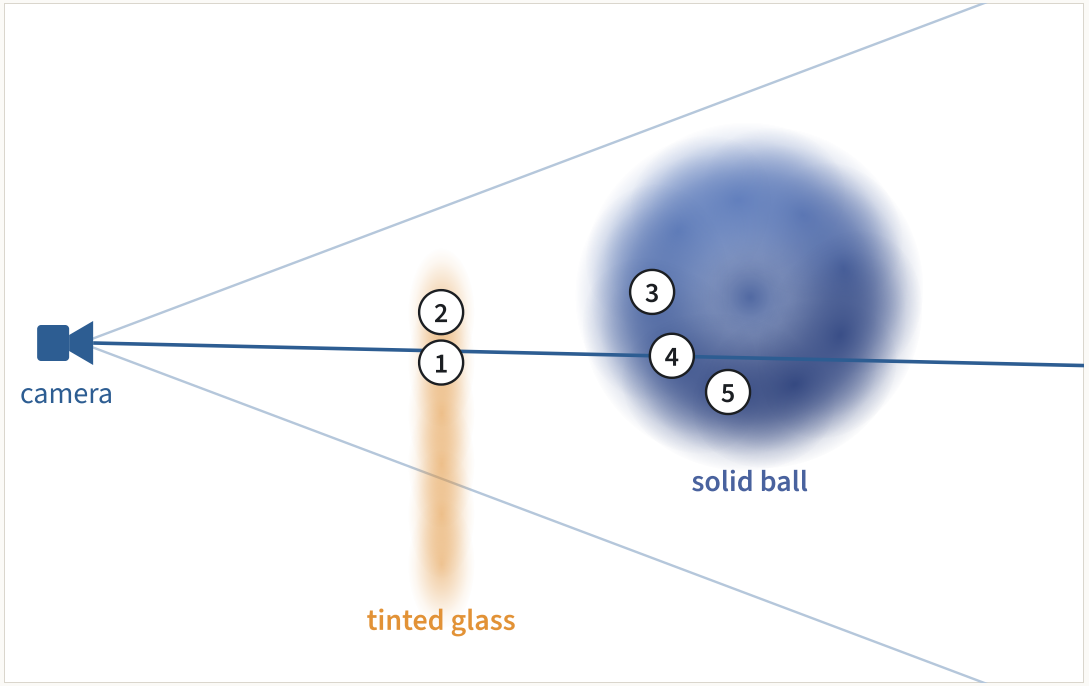
$$C = \Sigma_i \, c_i \, \alpha_i \, \Pi_{j<i} \, (1 - \alpha_j)$$

Step 1 of 3

Take one pixel. The splats that cover it are sorted, nearest first; the GPU does one sort for each 16 × 16 tile of pixels. A splat's opacity α at this pixel is its own opacity, weakened by how far the pixel is from the splat's center.

Kerbl et al. 2023, Sec. 2 and 6, Eq. 3: the blending sum is NeRF's Eq. 3 with $\alpha_i = 1 - e^{-\sigma_i \delta_i}$. Sorting is done once per 16 × 16-pixel tile with one radix sort. Same pixel as on NeRF's rendering slide.

Rendering one pixel from the Gaussians



Step 2 of 3

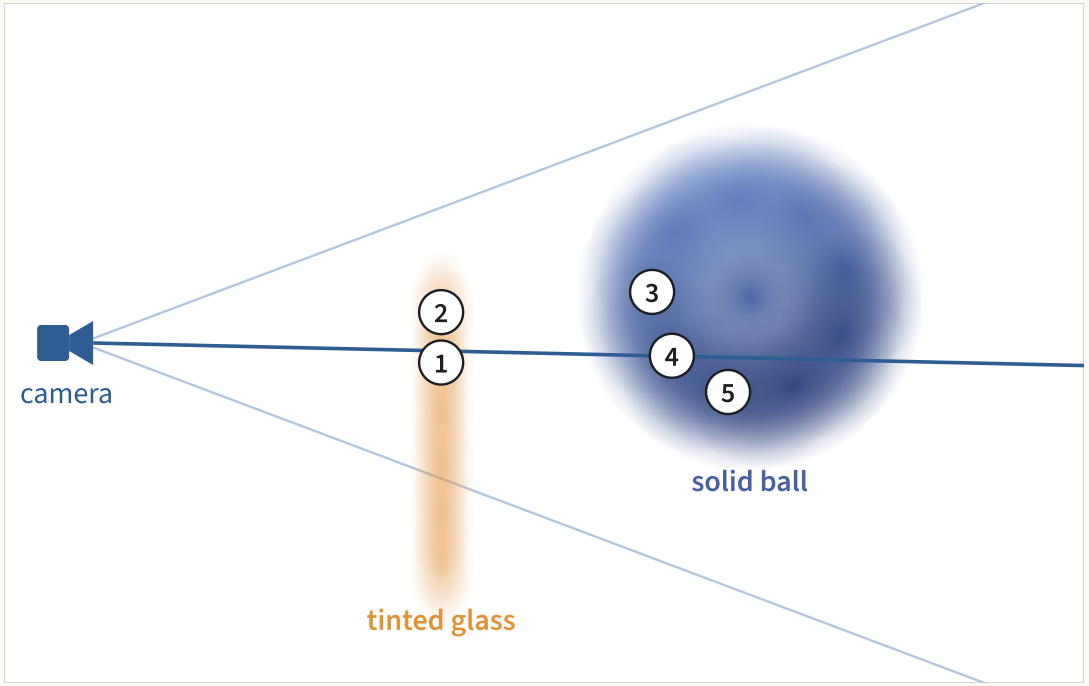
nearest first	opacity α		light getting through	
1		0.45		1.00
2		0.16		0.55
3		0.32		0.46
4		0.95		0.31
5		0.52		0.02

$$C = \Sigma_i \, c_i \, \alpha_i \, \Pi_{j<i} \, (1 - \alpha_j)$$

The light still getting through starts at 1 and shrinks behind every Gaussian, exactly like T in NeRF. Once almost no light gets through, the Gaussians behind are skipped.

Kerbl et al. 2023, Sec. 2 and 6, Eq. 3: the blending sum is NeRF's Eq. 3 with $\alpha_i = 1 - e^{-\sigma_i \delta_i}$. Sorting is done once per 16 × 16-pixel tile with one radix sort. Same pixel as on NeRF's rendering slide.

Rendering one pixel from the Gaussians



nearest first	opacity α		light getting through		contribution	
1		0.45		1.00		0.45
2		0.16		0.55		0.09
3		0.32		0.46		0.15
4		0.95		0.31		0.30
5		0.52		0.02		0.01

$$C = \Sigma_i \, c_i \, \alpha_i \, \Pi_{j<i} \, (1 - \alpha_j)$$

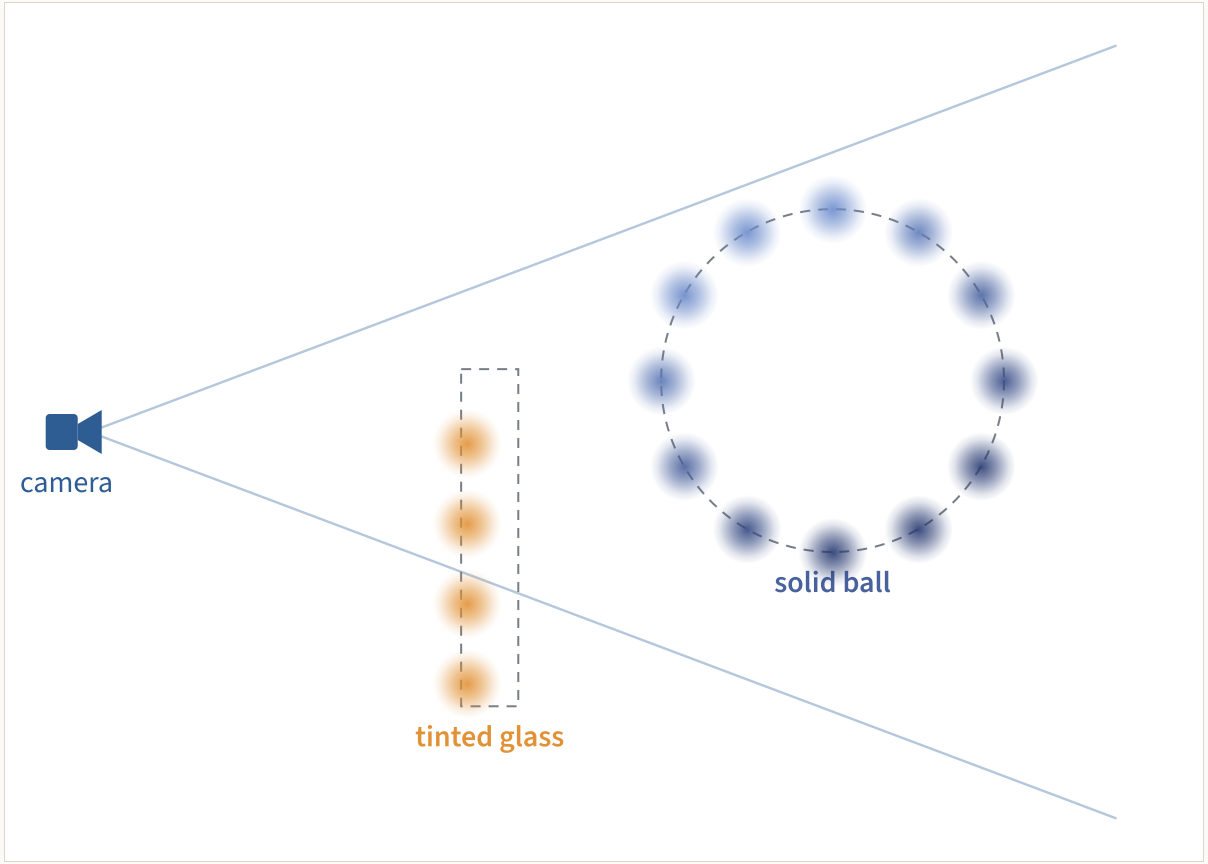
this pixel

Step 3 of 3

Each Gaussian adds its color × α × the light still getting through, and the sum is the pixel. It is NeRF's rendering sum, but every term is read from a Gaussian instead of asked from a network.

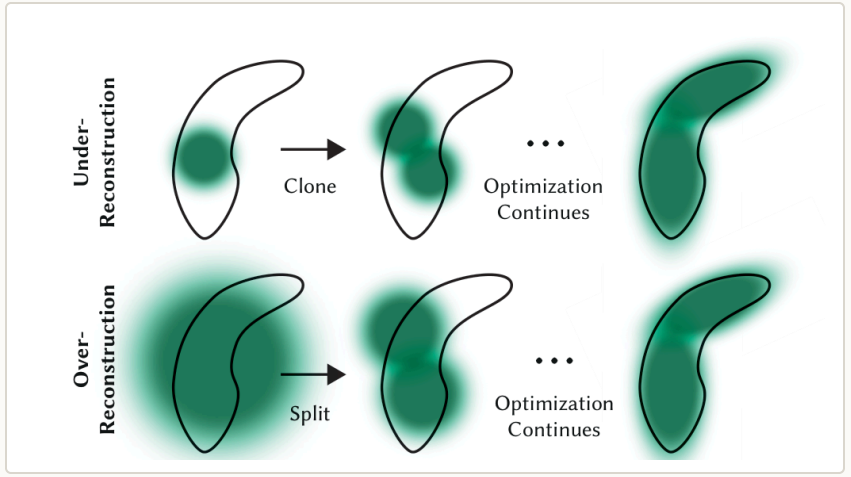
Kerbl et al. 2023, Sec. 2 and 6, Eq. 3: the blending sum is NeRF's Eq. 3 with $\alpha_i = 1 - e^{-\sigma_i \delta_i}$. Sorting is done once per 16 × 16-pixel tile with one radix sort. Same pixel as on NeRF's rendering slide.

Training the Gaussians



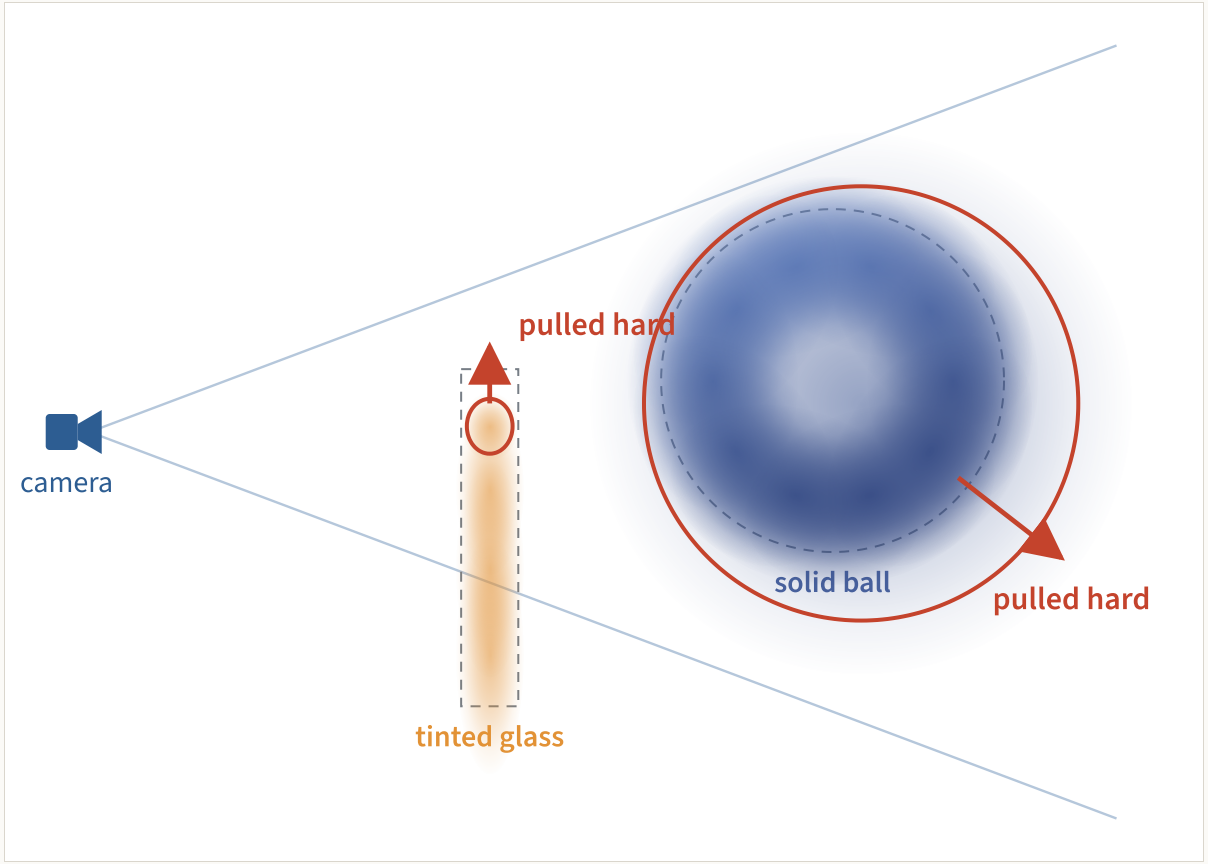
Step 1 of 4

Training starts from the 3D points COLMAP found while computing the camera positions. Each point becomes a small round Gaussian with the color of its point. The dashed outlines are what the photos show.



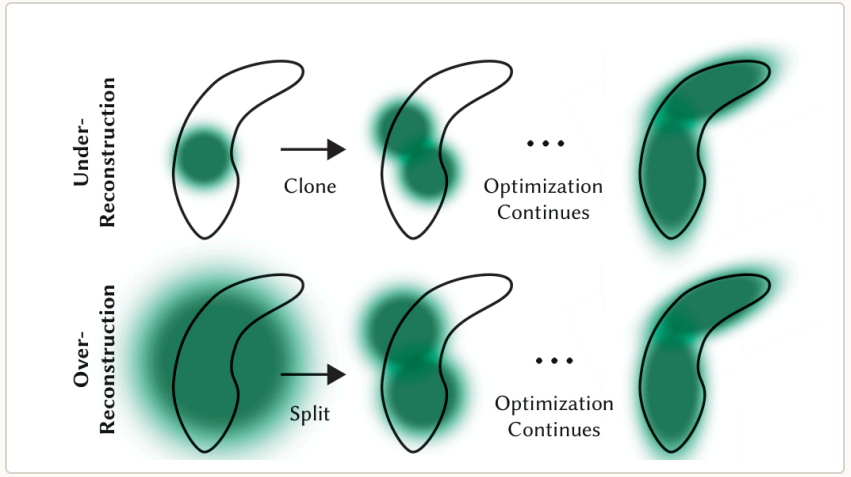
Kerbl et al. 2023, Fig. 4 and Sec. 5: loss 0.8 × L1 + 0.2 × D-SSIM between render and photo; every 100 steps, Gaussians with large position gradients are cloned (small) or split (large), and nearly transparent ones removed; 30,000 steps. Toy drawing of the four moments.

Training the Gaussians



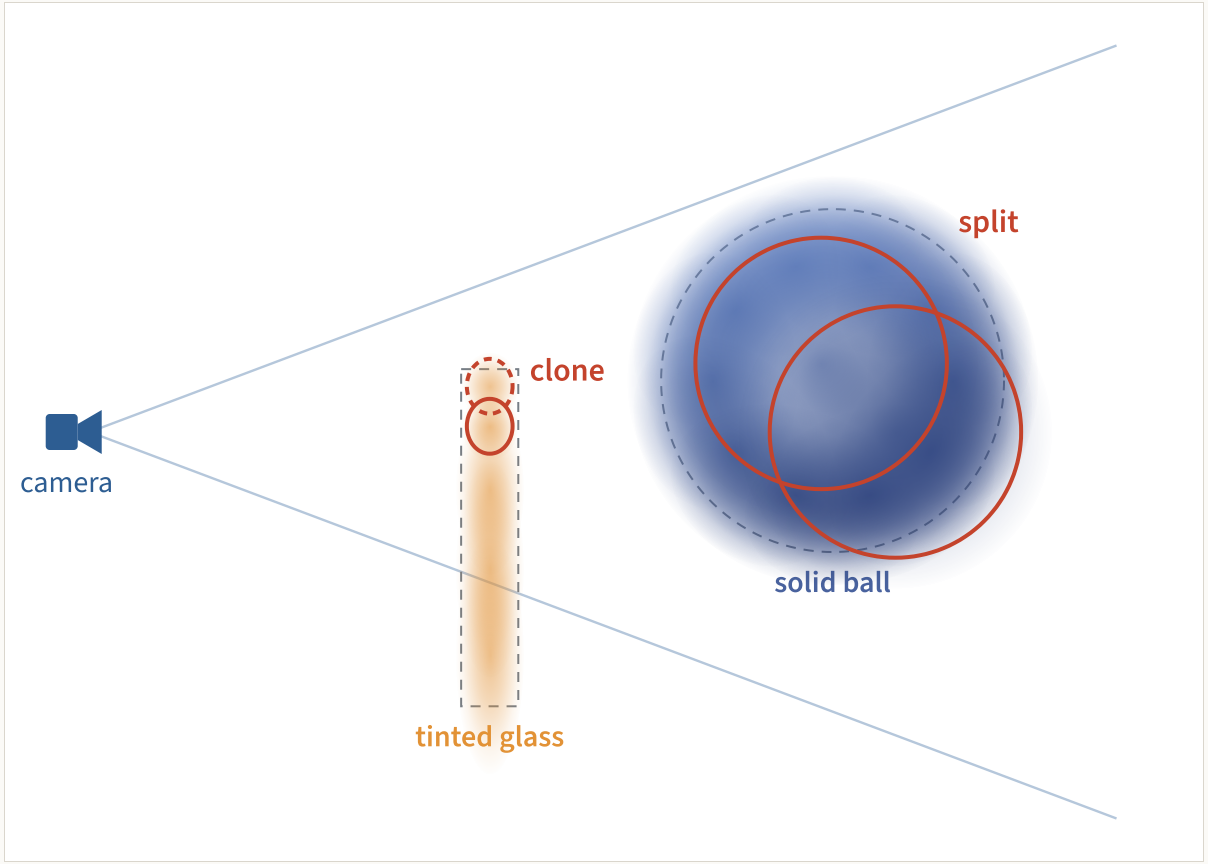
Step 2 of 4

Images are rendered and compared with the photos, as in NeRF, and gradient descent moves, stretches, turns and recolors every Gaussian. Where a region is still wrong, a Gaussian keeps being pulled hard: the top of the glass is empty, and one big Gaussian spills over the ball.



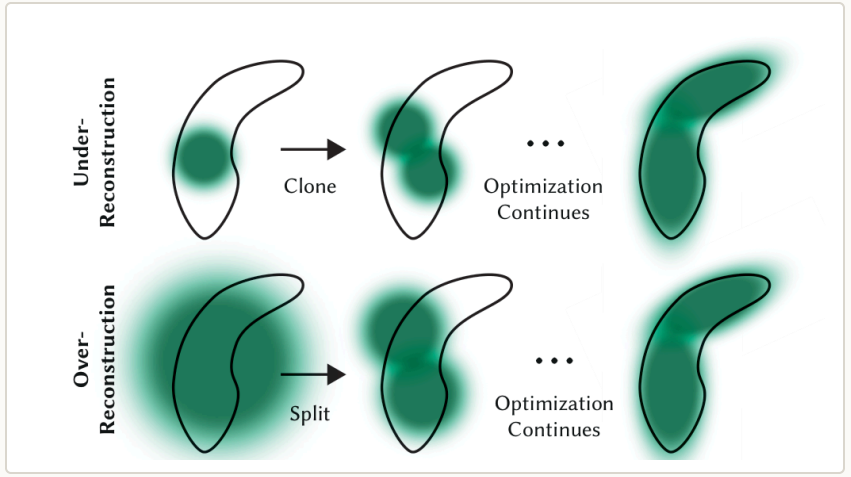
Kerbl et al. 2023, Fig. 4 and Sec. 5: loss 0.8 × L1 + 0.2 × D-SSIM between render and photo; every 100 steps, Gaussians with large position gradients are cloned (small) or split (large), and nearly transparent ones removed; 30,000 steps. Toy drawing of the four moments.

Training the Gaussians



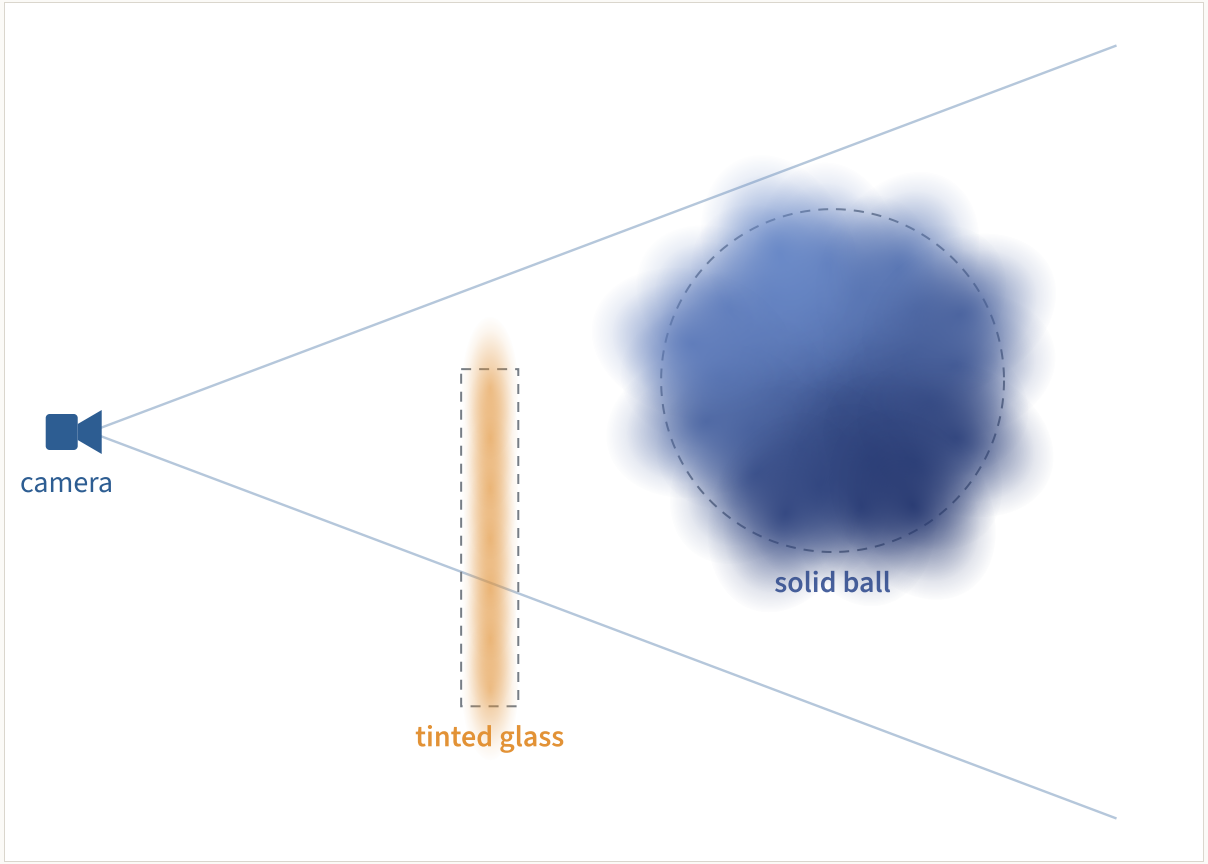
Step 3 of 4

Every 100 steps the set itself changes. A small Gaussian that is pulled hard is cloned, and the copy moves into the gap. A large one is split into two smaller ones. Nearly transparent Gaussians are deleted.



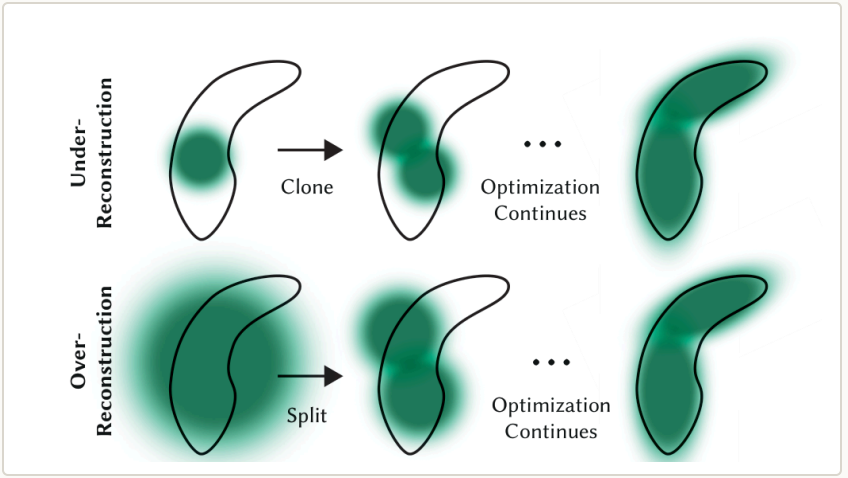
Kerbl et al. 2023, Fig. 4 and Sec. 5: loss 0.8 × L1 + 0.2 × D-SSIM between render and photo; every 100 steps, Gaussians with large position gradients are cloned (small) or split (large), and nearly transparent ones removed; 30,000 steps. Toy drawing of the four moments.

Training the Gaussians



Step 4 of 4

After 30,000 steps a few thousand starting points have become 1 to 5 million Gaussians on the paper's scenes. My globe ended with about 500,000.



Kerbl et al. 2023, Fig. 4 and Sec. 5: loss 0.8 × L1 + 0.2 × D-SSIM between render and photo; every 100 steps, Gaussians with large position gradients are cloned (small) or split (large), and nearly transparent ones removed; 30,000 steps. Toy drawing of the four moments.

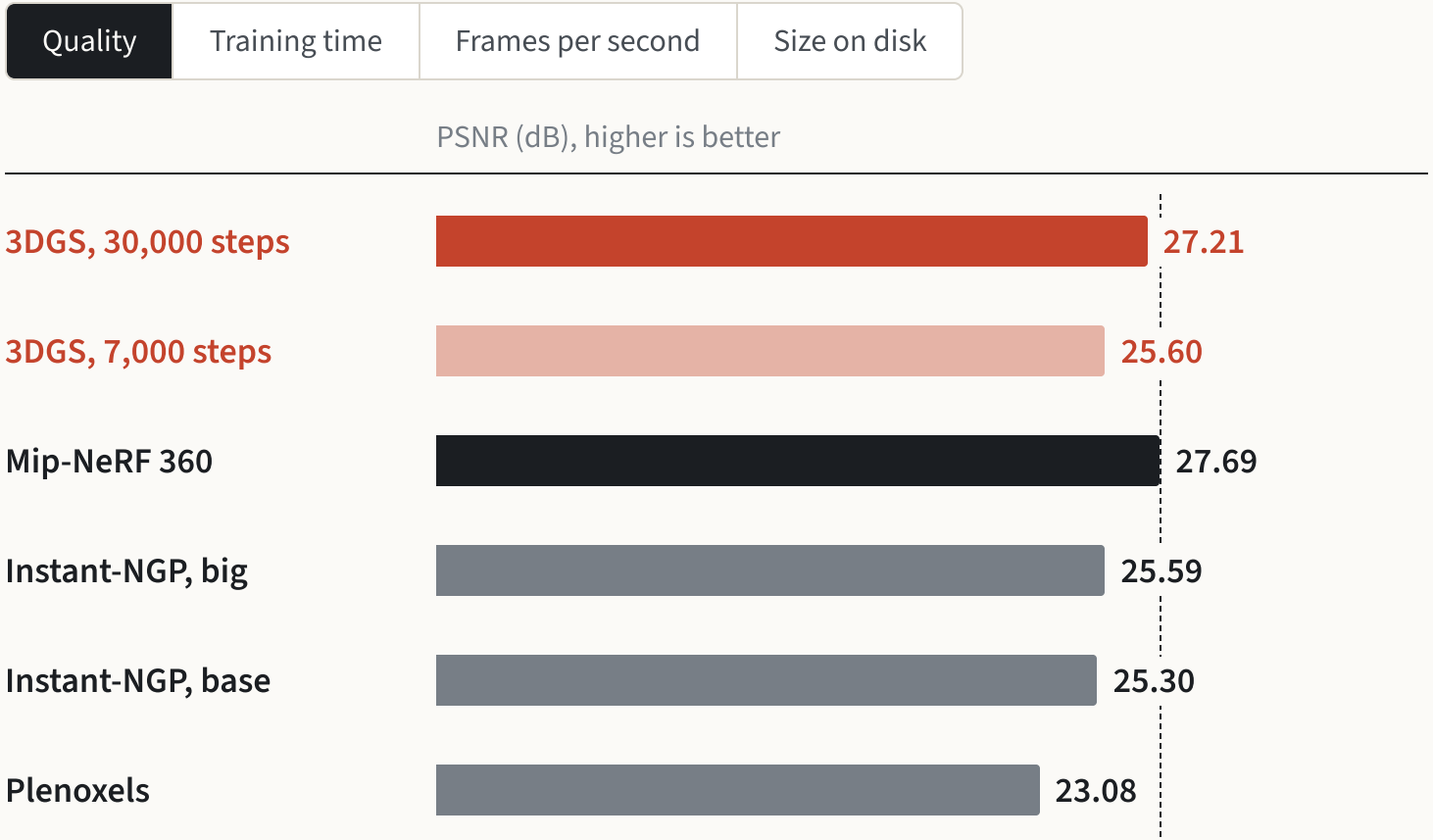
Speed and quality on one test view

Under each image: frames per second, training time and PSNR. Instant-NGP and Plenoxels train in minutes, but they render about 9 frames per second and lose detail.

After 6 minutes of training, Gaussian splatting renders 135 frames per second. After 51 minutes it beats Mip-NeRF 360, which trained for 48 hours and needs 14 seconds per frame.

Kerbl et al. 2023, Fig. 1: a test view of the Bicycle scene; one A6000 GPU. Mip-NeRF 360 (Barron et al. 2022) was the best-quality NeRF variant at the time; Plenoxels (Fridovich-Keil and Yu et al. 2022) is a sparse voxel grid without a network.

Gaussian splatting results

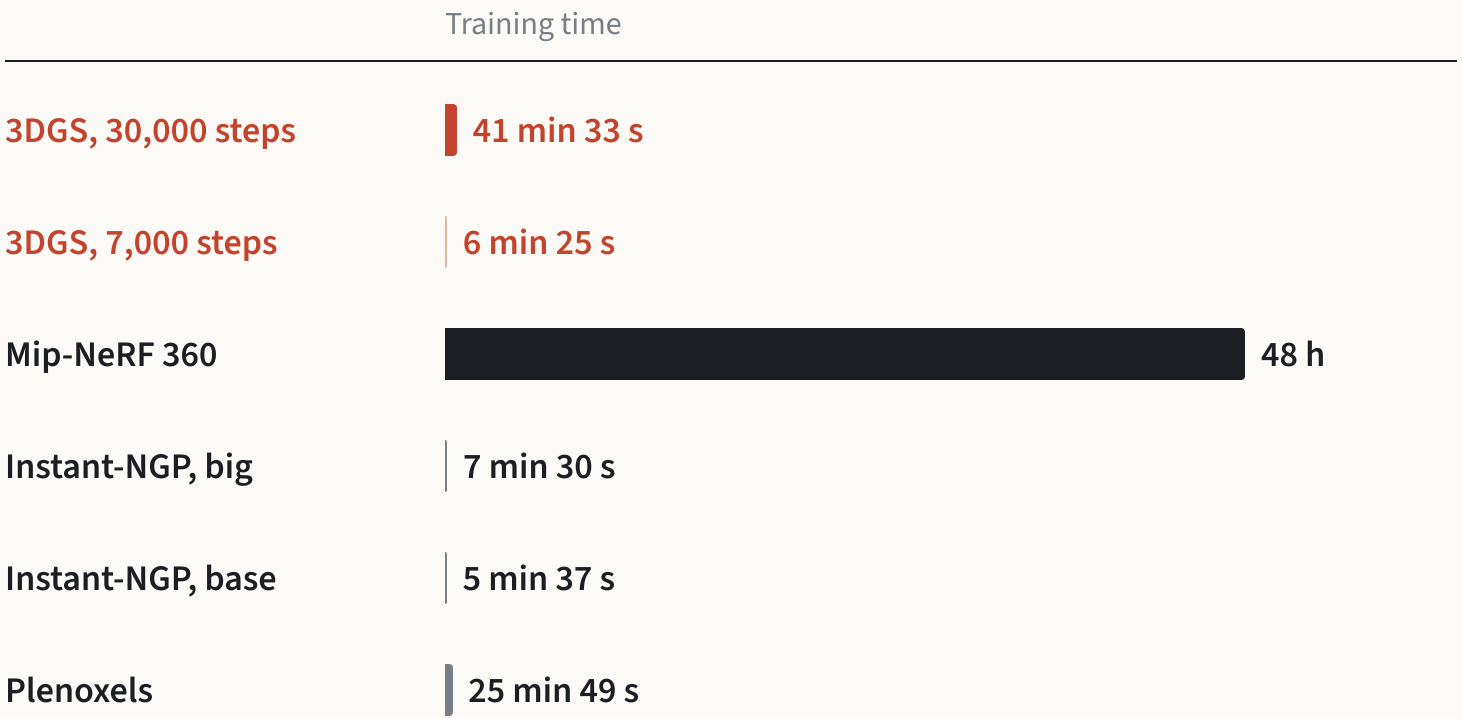


On nine large real scenes, Gaussian splatting reaches 27.2 dB after 30,000 steps, close to Mip-NeRF 360's 27.7 and above both fast methods.

Kerbl et al. 2023, Table 1: mean over the nine Mip-NeRF 360 scenes, one A6000 GPU. Mip-NeRF 360's numbers are from its own paper (Barron et al. 2022); the others are the authors' runs.

Gaussian splatting results

Quality	Training time	Frames per second	Size on disk

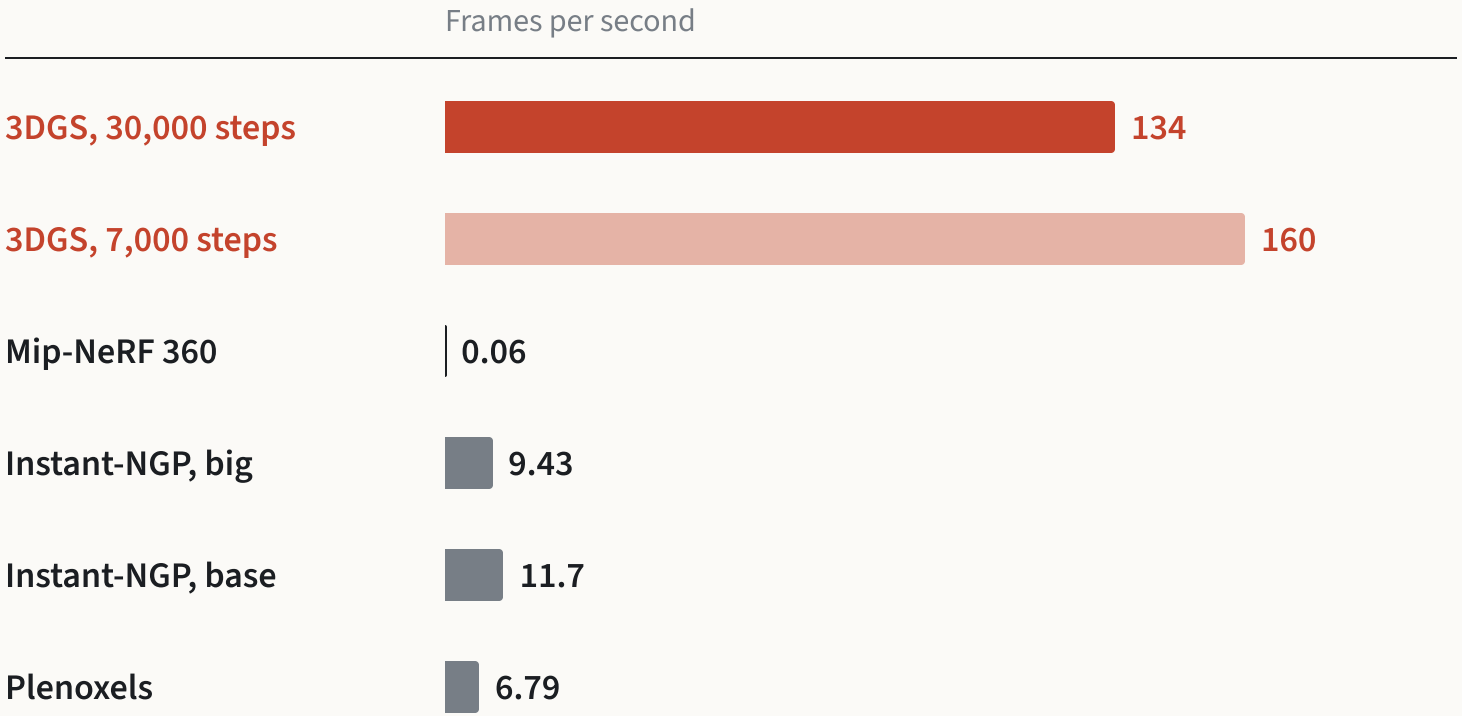


It trains in 42 minutes, against 48 hours for Mip-NeRF 360. After 7,000 steps, about 6 minutes, it already matches Instant-NGP.

Kerbl et al. 2023, Table 1: mean over the nine Mip-NeRF 360 scenes, one A6000 GPU. Mip-NeRF 360's numbers are from its own paper (Barron et al. 2022); the others are the authors' runs.

Gaussian splatting results

Quality | Training time | **Frames per second** | Size on disk

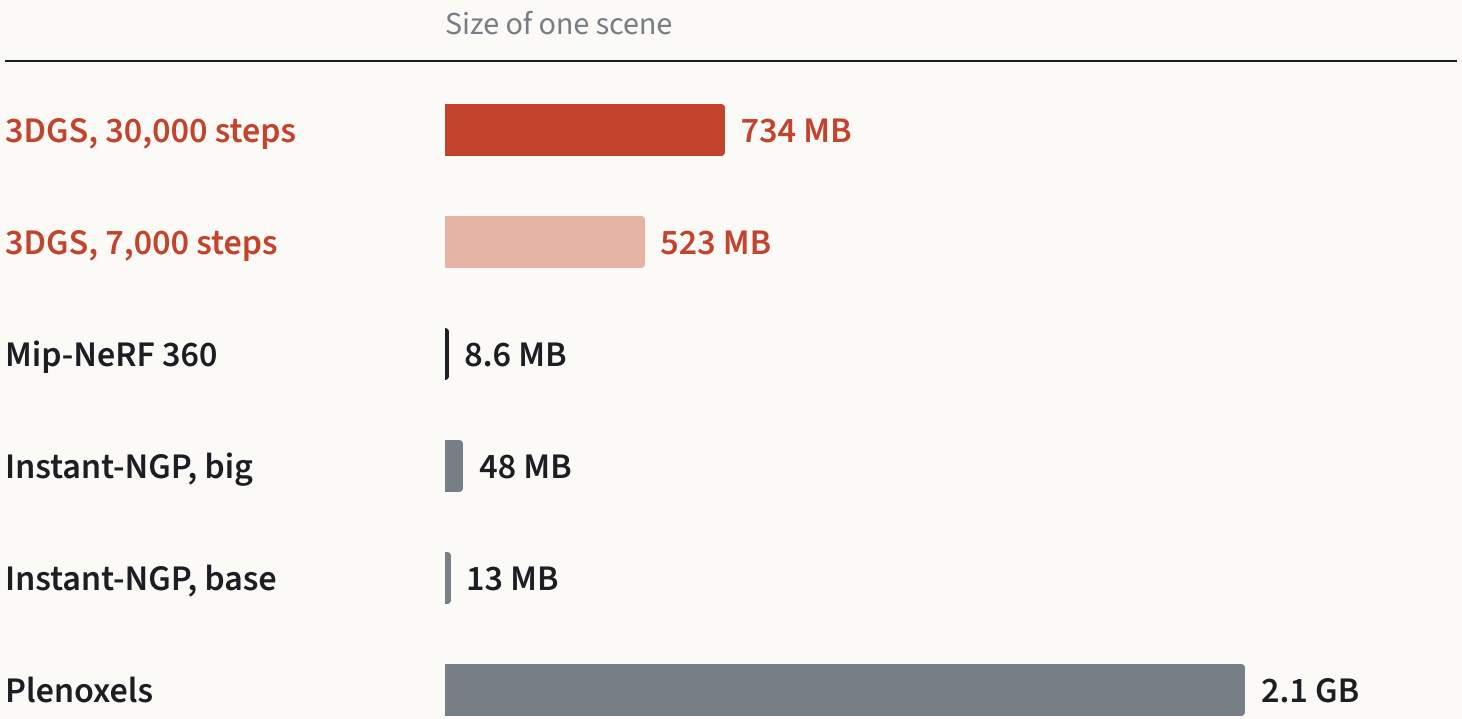


Rendering speed is the big difference. It draws 134 frames per second, against 0.06 for Mip-NeRF 360 and about 10 for Instant-NGP.

Kerbl et al. 2023, Table 1: mean over the nine Mip-NeRF 360 scenes, one A6000 GPU. Mip-NeRF 360's numbers are from its own paper (Barron et al. 2022); the others are the authors' runs.

Gaussian splatting results

Quality | Training time | Frames per second | **Size on disk**



The price is size. A scene takes 734 MB, against 8.6 MB for Mip-NeRF 360 and 13 to 48 MB for Instant-NGP. Plenoxels' voxel grid is even larger.

Kerbl et al. 2023, Table 1: mean over the nine Mip-NeRF 360 scenes, one A6000 GPU. Mip-NeRF 360's numbers are from its own paper (Barron et al. 2022); the others are the authors' runs.

What the difference looks like on real scenes

Bicycle | Garden | Stump | Dr Johnson

Photo

Gaussian splatting

Mip-NeRF 360

Instant-NGP

Plenoxels

Instant-NGP leaves dark floaters on the path, and Plenoxels blurs the grass. Gaussian splatting and Mip-NeRF 360 both stay close to the photo.

Kerbl et al. 2023, Fig. 5: held-out test views, images taken from the paper at full resolution. Bicycle, Garden and Stump are from the Mip-NeRF 360 dataset, Dr Johnson from Deep Blending (Hedman et al. 2018).

What the difference looks like on real scenes

Bicycle | **Garden** | Stump | Dr Johnson

Photo

Gaussian splatting

Mip-NeRF 360

Instant-NGP

Plenoxels

All methods are close on this scene; the fast ones lose some sharpness in the plants and houses at the back.

Kerbl et al. 2023, Fig. 5: held-out test views, images taken from the paper at full resolution. Bicycle, Garden and Stump are from the Mip-NeRF 360 dataset, Dr Johnson from Deep Blending (Hedman et al. 2018).

What the difference looks like on real scenes

Bicycle | Garden | **Stump** | Dr Johnson

Photo

Gaussian splatting

Mip-NeRF 360

Instant-NGP

Plenoxels

Instant-NGP fills the background with ghostly blobs and Plenoxels with colored smears. Gaussian splatting stays close to the photo.

Kerbl et al. 2023, Fig. 5: held-out test views, images taken from the paper at full resolution. Bicycle, Garden and Stump are from the Mip-NeRF 360 dataset, Dr Johnson from Deep Blending (Hedman et al. 2018).

What the difference looks like on real scenes

Bicycle | Garden | Stump | **Dr Johnson**

Photo

Gaussian splatting

Mip-NeRF 360

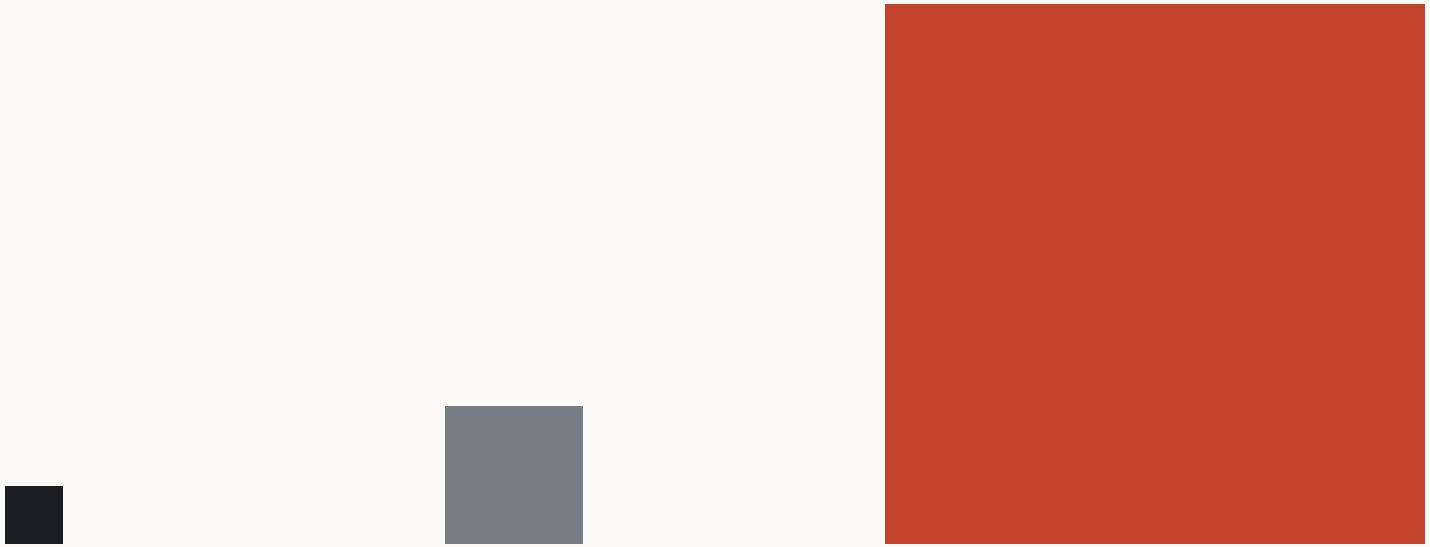
Instant-NGP

Plenoxels

On the plain white walls of this indoor scene, Instant-NGP leaves grainy dark noise and Plenoxels cloudy patches. Gaussian splatting stays clean.

Kerbl et al. 2023, Fig. 5: held-out test views, images taken from the paper at full resolution. Bicycle, Garden and Stump are from the Mip-NeRF 360 dataset, Dr Johnson from Deep Blending (Hedman et al. 2018).

What Gaussian splatting stores



8.6 MB
Mip-NeRF 360

48 MB
Instant-NGP, big

734 MB
Gaussian splatting

Step 1 of 2

Gaussian splatting needs the most space of the three, 734 MB per scene on average. Nothing is shared: every Gaussian keeps its own 59 numbers, about 240 bytes, and a scene needs about 3 million of them. 48 of the 59 are the color, which changes with the viewing direction.

Kerbl et al. 2023, Table 1 (mean size per scene, Mip-NeRF 360 dataset) and Sec. 7.4 (limitations). 59 numbers × 4 bytes = 236 bytes per Gaussian; 734 MB ÷ 236 bytes ≈ 3 million Gaussians. Squares are sized by area.

What Gaussian splatting stores

8.6 MB
Mip-NeRF 360

48 MB
Instant-NGP, big

734 MB
Gaussian splatting

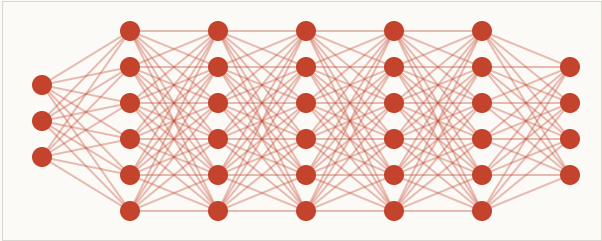
2.1 GB
Plenoxels, a voxel grid

Step 2 of 2

It is still smaller than Plenoxels, a voxel grid without a network, which needs 2.1 GB for the same scenes and looks worse. Writing everything down is what makes 134 frames per second possible. Training a large scene can need over 20 GB of GPU memory, and where the photos see little, Gaussians become blotchy.

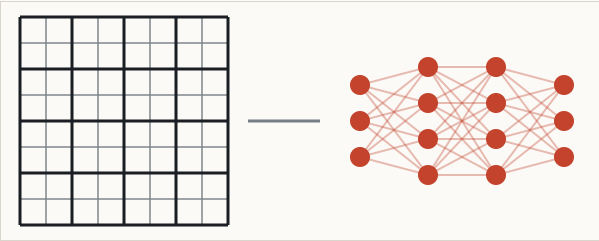
Kerbl et al. 2023, Table 1 (mean size per scene, Mip-NeRF 360 dataset) and Sec. 7.4 (limitations). 59 numbers × 4 bytes = 236 bytes per Gaussian; 734 MB ÷ 236 bytes ≈ 3 million Gaussians. Squares are sized by area.

Three ways to store the same scene



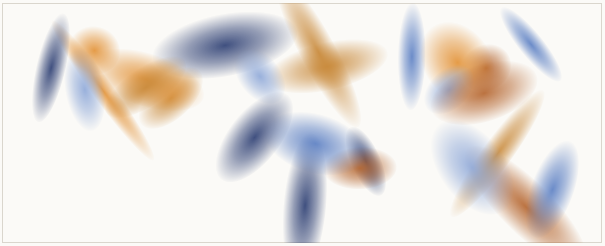
NeRF, 2020

The scene is two networks with 1.2 million weights, 5 MB. Training takes one to two days on a V100, and an image about 30 seconds, because every pixel queries the network 256 times.



Instant-NGP, 2022

The scene is 16 hashed grids with 12.6 million learned entries and a network of 10,000 weights. Training takes seconds to minutes on an RTX 3090, and images are still rendered by marching rays.



3D Gaussian splatting, 2023

The scene is one to five million Gaussians of 59 numbers each, about 734 MB. Training takes about 40 minutes on an A6000, and rendering runs at over 100 frames per second.

> All three render with the same sum along a ray; they differ in where the density and color come from.

Numbers from each paper as shown on the previous slides; the GPUs and test scenes differ between papers, so compare orders of magnitude, not exact values.

What NeRF leaves open

Blur at other distances	One thin ray per pixel misses detail when the camera is closer or further than in training. Mip-NeRF (Barron et al. 2021) traces cones instead, and Mip-NeRF 360 (Barron et al. 2022) extends it to large outdoor scenes.
One network per scene	Nothing is learned across scenes, so each needs many photos. pixelNeRF (Yu et al. 2021) learns a prior from many scenes and works from one or a few photos.
A still scene, fixed light	Moving objects and changing lighting break the assumption that every photo shows the same scene. NeRF in the Wild (Martin-Brualla et al. 2021) handles tourist photos taken at different times.
Medical imaging	CT already measures density along rays. NAF (Zha et al. 2022) keeps only NeRF's density and reconstructs cone-beam CT from few X-ray projections.

Limits from Mildenhall et al. 2020, Sec. 7. Follow-up work as cited; see the references.

My experiments

I filmed a gemstone globe on a brass stand with my phone, walking around it at four heights.

On the same photos I trained all three methods, NeRF, Instant-NGP and 3D Gaussian splatting, and compared their images, training time, rendering speed and storage.

Finding the camera positions

0 s

63 s

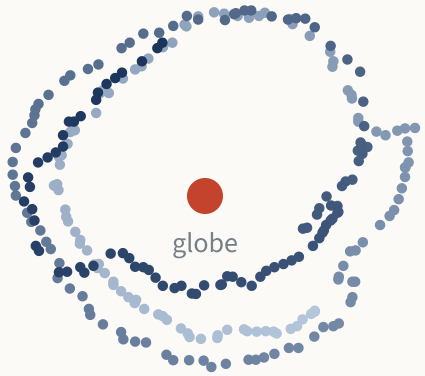
130 s

157 s

I filmed the globe for almost three minutes and kept the sharpest frame out of every ten. Every second one of those, 257 photos at 1080 × 607 pixels, went to COLMAP.

COLMAP matches features between the photos and works out where each camera stood and where it looked. It placed 255 of the 257, with an average error of 0.58 pixels. All three methods start from these outputs.

seen from above



seen from the side

each dot is one camera, light at the start of the video and dark at the end

My capture: iPhone 17, Blackmagic Camera app, 1/120 s shutter, 2 min 54 s at 30 frames per second. Camera positions from COLMAP 4.2 (pycolmap), one shared SIMPLE_RADIAL camera; the error is the mean reprojection error.

Cutting the globe out of the background

Step 1 of 2

The full photo, which COLMAP gets

The mask from SAM 2, room darkened

None of the three methods knows what an object is; they rebuild whatever the photos show. To keep only the globe, SAM 2, a video segmentation model, followed it through all 255 photos from one box and two clicks on the first. COLMAP still gets the full photos, because the room gives it far more to match.

My capture, frame 1868 of the video. Masks: SAM 2.1 Hiera-L (Ravi et al. 2024) through Hugging Face transformers, prompted with a box and two points in the first frame, then propagated. White background as in NeRF's synthetic scenes; Instant-NGP sees the same masks with a random background color, so it cannot fill empty space with white fog.

Cutting the globe out of the background

Step 2 of 2

The full photo, which COLMAP gets

The mask from SAM 2, room darkened

What the three models are trained on

None of the three methods knows what an object is; they rebuild whatever the photos show. To keep only the globe, SAM 2, a video segmentation model, followed it through all 255 photos from one box and two clicks on the first. COLMAP still gets the full photos, because the room gives it far more to match.

The three models are trained on the masked photos. The cheapest way to match an empty background is to leave that space empty, so the room never enters the models.

My capture, frame 1868 of the video. Masks: SAM 2.1 Hiera-L (Ravi et al. 2024) through Hugging Face transformers, prompted with a box and two points in the first frame, then propagated. White background as in NeRF's synthetic scenes; Instant-NGP sees the same masks with a random background color, so it cannot fill empty space with white fog.

The 3D points COLMAP found on my globe

Step 1 of 2

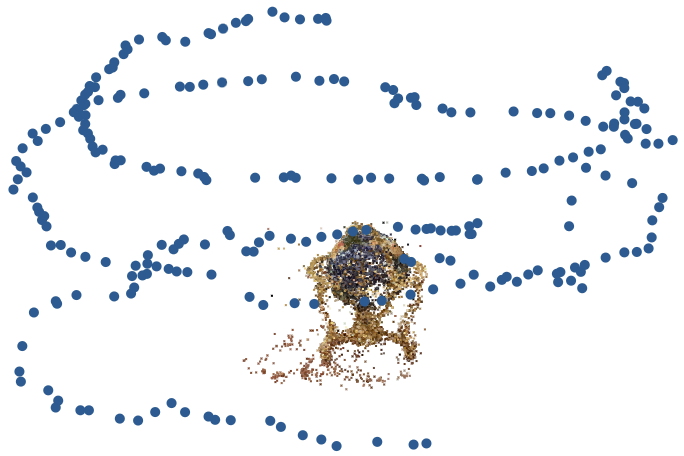
drag to turn

While placing the cameras, COLMAP also found 41,128 points in 3D, each with the color of the photos it came from. It saw the whole room; these are the 28,184 points on the globe, its stand and the table.

My capture: COLMAP sparse model of the 257 frames (mean reprojection error 0.58 px). 16,000 of the 28,184 points within 1.2 units of the globe are drawn; the view is orthographic.

The 3D points COLMAP found on my globe

Step 2 of 2



drag to turn

While placing the cameras, COLMAP also found 41,128 points in 3D, each with the color of the photos it came from. It saw the whole room; these are the 28,184 points on the globe, its stand and the table.

Zoomed out, the blue dots are my 255 cameras, the four rings I walked. NeRF and Instant-NGP use only these cameras. Gaussian splatting also starts from the colored points: each one becomes a small Gaussian, which training then moves, copies and splits.

My capture: COLMAP sparse model of the 257 frames (mean reprojection error 0.58 px). 16,000 of the 28,184 points within 1.2 units of the globe are drawn; the view is orthographic.

Training the three models

NeRF | Instant-NGP | Gaussian splatting

Held-out photo, never trained on

After 2,500 steps (13 min)

After 25,000 steps (2.8 h)

At the end, 100,000 steps (9.0 h)

The paper's model and settings: 100,000 steps of 4,096 rays, with 64 + 128 samples on every ray, 9.0 hours in total. After 2,500 steps (13 min) the shape is there but the ball is a blur; by 25,000 steps (2.8 h) most of the detail is in.

My runs on my laptop's RTX 5070 Ti, all on the same 223 training photos; the photo is one of the 32 held out (every 8th, as in the NeRF paper). NeRF: the paper's model and settings. Instant-NGP: the paper's hash grid and networks, my implementation. Gaussian splatting: gsplat 1.5.3, default recipe; its earlier times are estimated from the total. All times depend on the GPU.

Training the three models

NeRF | **Instant-NGP** | Gaussian splatting

Held-out photo, never trained on

After 6,000 steps (5 min)

After 12,000 steps (9 min)

At the end, 20,000 steps (15 min)

The paper's hash grid, 16 levels with up to 2^{19} entries each, and two tiny networks: 20,000 steps of 4,096 rays, 15 minutes in total. After 6,000 steps (5 min) the continents are already there; the rest of the run sharpens the edges but leaves a grainy noise on the surface.

My runs on my laptop's RTX 5070 Ti, all on the same 223 training photos; the photo is one of the 32 held out (every 8th, as in the NeRF paper). NeRF: the paper's model and settings. Instant-NGP: the paper's hash grid and networks, my implementation. Gaussian splatting: gsplat 1.5.3, default recipe; its earlier times are estimated from the total. All times depend on the GPU.

Training the three models

NeRF | Instant-NGP | **Gaussian splatting**

Held-out photo, never trained on

After 1,000 steps (about 1 min)

After 7,000 steps (about 7 min)

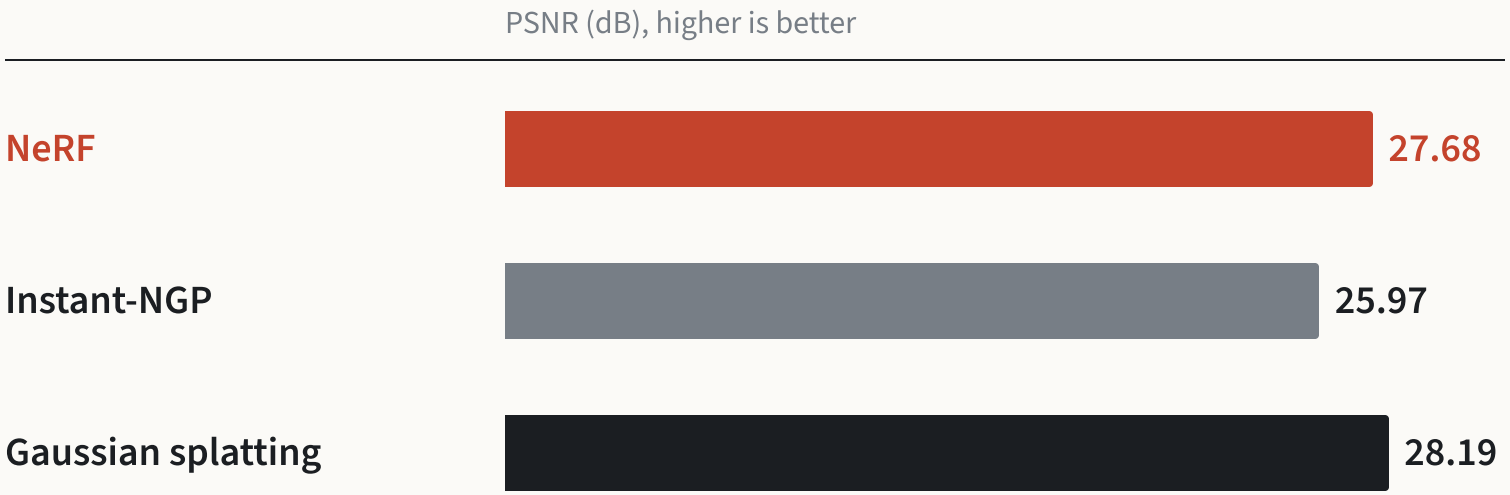
At the end, 30,000 steps (31 min)

It starts from COLMAP's points and renders one whole photo per step, 30,000 steps and 31 minutes in total. Along the way it copies and splits Gaussians where detail is missing and ends with 500,482. After 1,000 steps (about 1 min) the globe is recognizable but blotchy.

My runs on my laptop's RTX 5070 Ti, all on the same 223 training photos; the photo is one of the 32 held out (every 8th, as in the NeRF paper). NeRF: the paper's model and settings. Instant-NGP: the paper's hash grid and networks, my implementation. Gaussian splatting: gsplat 1.5.3, default recipe; its earlier times are estimated from the total. All times depend on the GPU.

Image quality on my globe

PSNR | SSIM | LPIPS



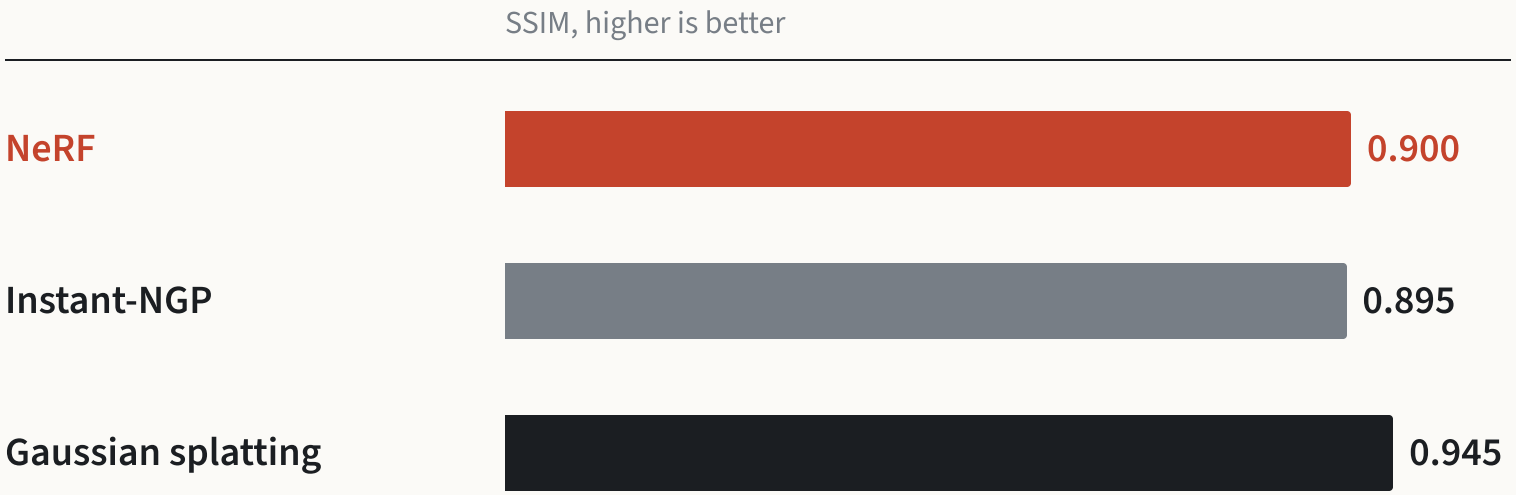
Gaussian splatting has the best average and is the best of the three on 24 of the 32 photos; NeRF on 8, Instant-NGP on 0.

The bars look alike, but the 2.2 dB between first and last is 40% less squared error.

My capture. All three trained on the same 223 photos, cameras and masks, and scored on the same 32 held-out photos after correcting the lens to a pinhole camera (NeRF on the original photos: 27.27 dB). NeRF: the paper's model, 100,000 steps. Instant-NGP: the paper's hash grid and networks, my implementation, 20,000 steps. Gaussian splatting: gsplat 1.5.3 (Ye et al. 2025), 30,000 steps.

Image quality on my globe

PSNR | **SSIM** | LPIPS

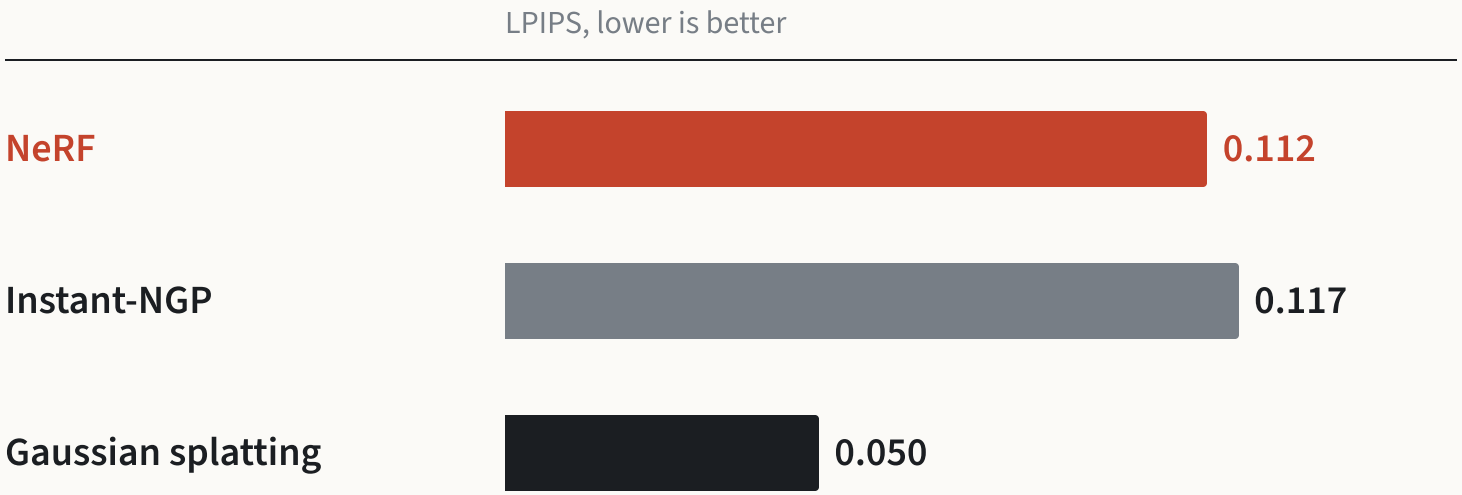


SSIM compares local structure such as edges and texture. Gaussian splatting keeps the most of it.

My capture. All three trained on the same 223 photos, cameras and masks, and scored on the same 32 held-out photos after correcting the lens to a pinhole camera (NeRF on the original photos: 27.27 dB). NeRF: the paper's model, 100,000 steps. Instant-NGP: the paper's hash grid and networks, my implementation, 20,000 steps. Gaussian splatting: gsplat 1.5.3 (Ye et al. 2025), 30,000 steps.

Image quality on my globe

PSNR | SSIM | **LPIPS**



LPIPS punishes blur the most. Gaussian splatting has 2.3 times less of it than Instant-NGP.

My capture. All three trained on the same 223 photos, cameras and masks, and scored on the same 32 held-out photos after correcting the lens to a pinhole camera (NeRF on the original photos: 27.27 dB). NeRF: the paper's model, 100,000 steps. Instant-NGP: the paper's hash grid and networks, my implementation, 20,000 steps. Gaussian splatting: gsplat 1.5.3 (Ye et al. 2025), 30,000 steps.

Where each method looks better

Step 1 of 2

Photo

NeRF, 28.3 dB

Instant-NGP, 27.4 dB

Gaussian splatting, 31.0 dB

Close up, Gaussian splatting keeps the grid lines and place names that NeRF blurs. Instant-NGP recovers more of them than NeRF, but its fine hash levels leave a grainy noise over the surface.

Held-out frames 808 and 3685 of my capture, never used in training; PSNR over the whole photo; both rows are enlarged close-ups. Limitation: Kerbl et al. 2023, Sec. 7.4.

Where each method looks better

Step 2 of 2

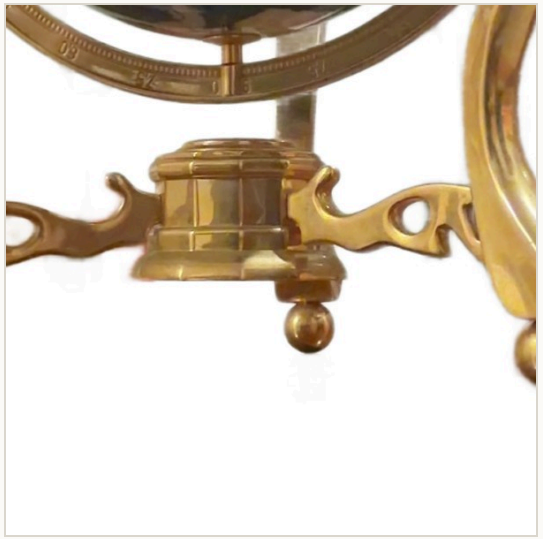
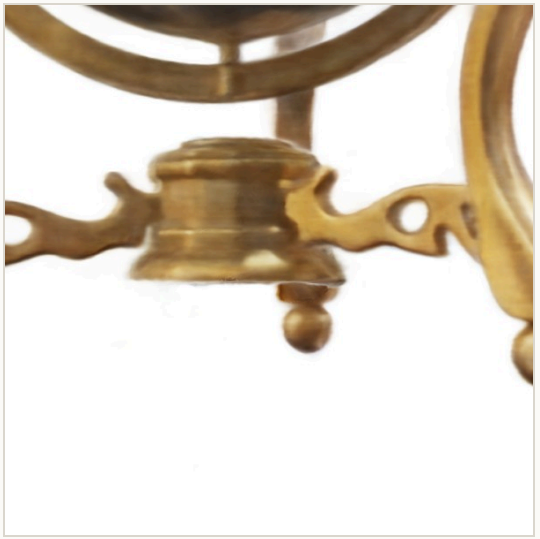
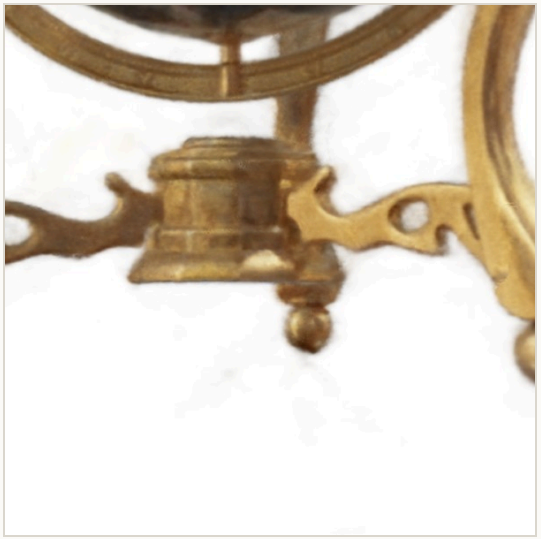


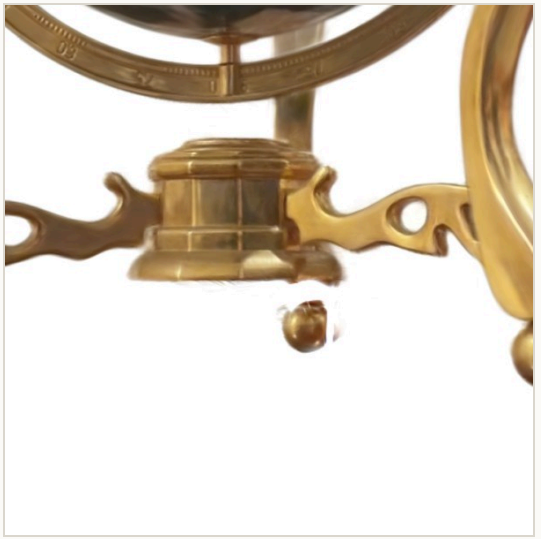
Photo from below



NeRF, 27.9 dB



Instant-NGP, 25.6 dB



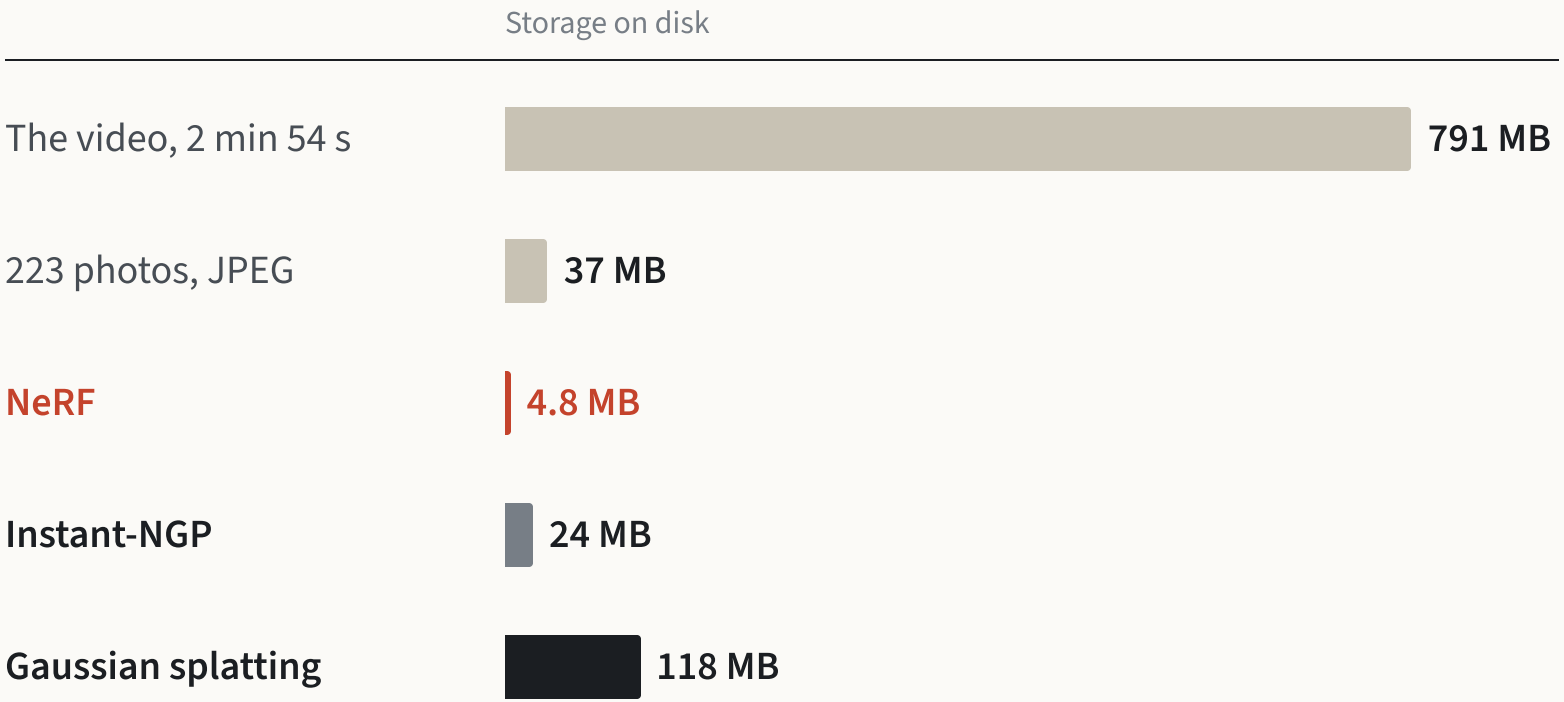
Gaussian splatting, 25.1 dB

From below, where few of my photos look, NeRF is best. Gaussian splatting breaks one brass foot into loose Gaussians, the limitation its paper names; Instant-NGP keeps the foot but leaves a faint haze around the stand.

Held-out frames 808 and 3685 of my capture, never used in training; PSNR over the whole photo; both rows are enlarged close-ups. Limitation: Kerbl et al. 2023, Sec. 7.4.

What each method costs on my globe

Storage | Training | One image



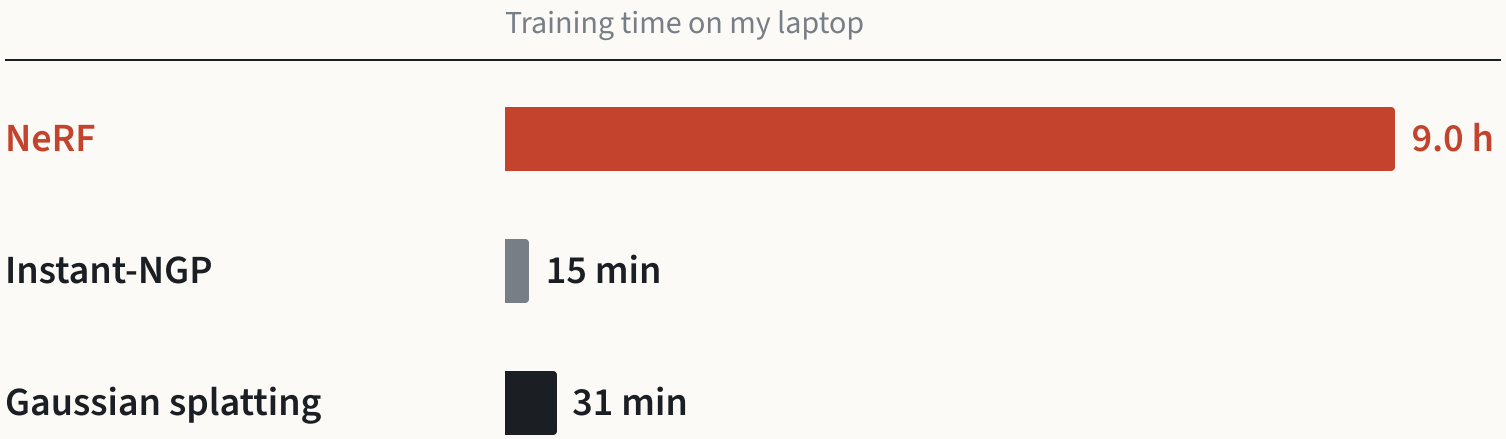
NeRF keeps the whole globe, from every direction, in 4.8 MB: 8 times less than the 223 photos it learned from.

Instant-NGP's hash tables take 24 MB, and Gaussian splatting's 500,482 Gaussians 118 MB.

My capture. Video: 174 s of HEVC at about 36 Mbit/s (its size from the bitrate). Photos: the 223 training frames as 1080 × 607 JPEG (439 MB uncompressed). Models as stored: NeRF 1,187,848 weights at 32 bits, Instant-NGP 12,207,469 numbers at 16 bits as in the paper, Gaussian splatting 29,528,438 numbers at 32 bits (.ply). Same laptop GPU (RTX 5070 Ti) for all times.

What each method costs on my globe

Storage | **Training** | One image



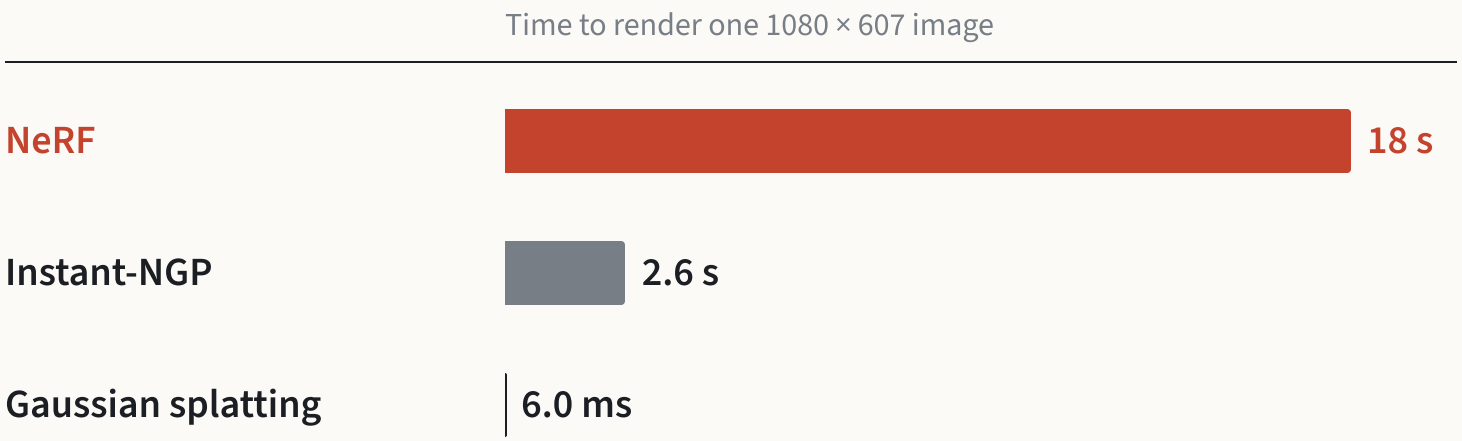
9.0 hours for NeRF, 15 minutes for Instant-NGP and 31 minutes for Gaussian splatting, on the same GPU.

My Instant-NGP runs partly in PyTorch; with the paper's fully fused code it would take seconds to minutes.

My capture. Video: 174 s of HEVC at about 36 Mbit/s (its size from the bitrate). Photos: the 223 training frames as 1080 × 607 JPEG (439 MB uncompressed). Models as stored: NeRF 1,187,848 weights at 32 bits, Instant-NGP 12,207,469 numbers at 16 bits as in the paper, Gaussian splatting 29,528,438 numbers at 32 bits (.ply). Same laptop GPU (RTX 5070 Ti) for all times.

What each method costs on my globe

Storage | Training | **One image**



18 seconds for NeRF, 2.6 s for Instant-NGP and 6.0 milliseconds for Gaussian splatting, about 165 images per second, which is why it is the only one I can show live.

My capture. Video: 174 s of HEVC at about 36 Mbit/s (its size from the bitrate). Photos: the 223 training frames as 1080 × 607 JPEG (439 MB uncompressed). Models as stored: NeRF 1,187,848 weights at 32 bits, Instant-NGP 12,207,469 numbers at 16 bits as in the paper, Gaussian splatting 29,528,438 numbers at 32 bits (.ply). Same laptop GPU (RTX 5070 Ti) for all times.

New views rendered by NeRF and Instant-NGP

NeRF | Instant-NGP

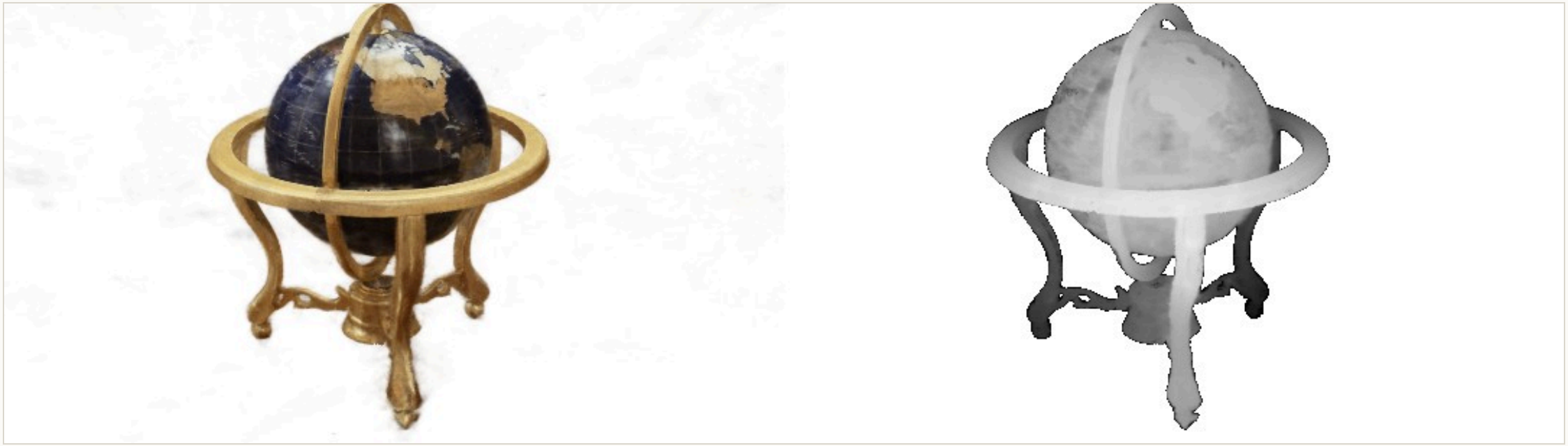
I rendered 120 images with my NeRF from a circle of new cameras, a little further back than mine, and joined them into this video. On the right is its depth, never given in training.

NeRF is too slow to show live: one image took about 4 seconds at this half resolution, and 18 seconds at the full 1080 × 607.

My trained models, the same 120 viewpoints on a circle around the globe for both, 1.3 times the distance of my cameras, at 540 × 303 pixels per image. Depth is the weighted mean distance along each ray, drawn only where the opacity is above 0.5. Times on my laptop's RTX 5070 Ti.

New views rendered by NeRF and Instant-NGP

NeRF | Instant-NGP



The same 120 cameras rendered by my Instant-NGP. Its depth is cleaner, without the floaters NeRF leaves around the stand, but the surface is a little grainier.

One image took 0.9 seconds at this half resolution and 2.6 seconds at full size: much faster than NeRF, but still not live with my implementation.

My trained models, the same 120 viewpoints on a circle around the globe for both, 1.3 times the distance of my cameras, at 540 × 303 pixels per image. Depth is the weighted mean distance along each ray, drawn only where the opacity is above 0.5. Times on my laptop's RTX 5070 Ti.

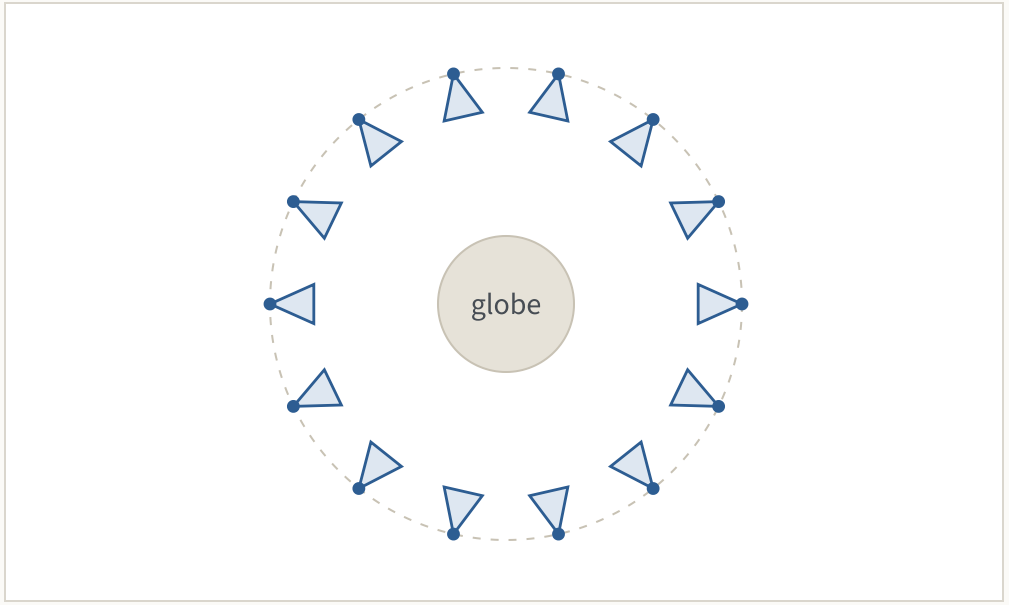
My globe as Gaussian splatting, live demo

The Gaussians from my training, drawn by this page on the laptop's GPU. Every frame they are sorted by depth and blended front to back with the same sum NeRF uses along a ray.

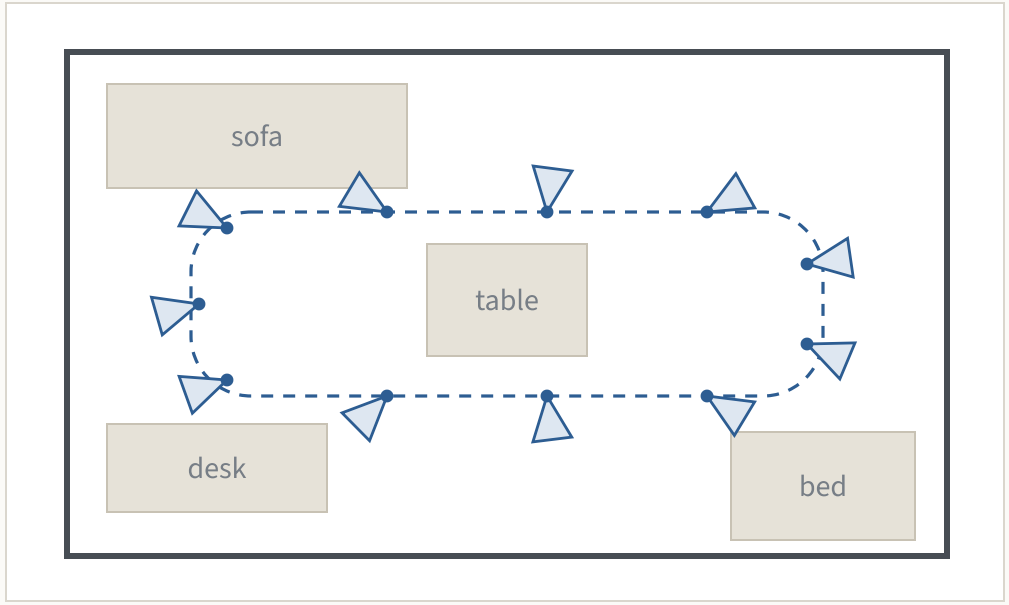
Shown live in the talk; here one frame of it. A frame took about 4 ms to draw on my laptop, while NeRF needs 18 seconds for one image.

My capture. Gaussians trained with gsplat 1.5.3, drawn by a WebGL2 renderer written for this talk, with view-dependent color up to degree 3; its images match gsplat's to 41–43 dB. Full window.

Outside an object, or inside a room



My globe: cameras around one object, looking in



A room: cameras walking a loop inside it, looking out

I filmed my globe from outside, with every camera looking in at one object. The next question is the opposite case: a whole room, filmed from inside with the cameras looking out. It needs no mask, since everything in the room belongs to the scene.

The original NeRF assumes the scene lies between a near and a far distance, so a room needs its successors; Mip-NeRF 360 and Gaussian splatting both test on indoor rooms. The camera has to walk a loop, because turning on one spot gives no depth.

Barron et al. 2022 (Mip-NeRF 360: unbounded scenes; its dataset includes the indoor scenes room, kitchen, counter and bonsai); Kerbl et al. 2023, Table 1 and Fig. 5 (Dr Johnson, an indoor scene from Hedman et al. 2018).

Summary

	Stores the scene as	What its paper showed	On my globe, same photos and laptop GPU			
			Training	One image	Storage	PSNR
NeRF	one network of 1.2 million weights	31.0 dB on synthetic scenes, 5 dB above earlier methods	9.0 h	18 s	**4.8 MB**	27.7 dB
Instant-NGP	hash grids and two tiny networks	NeRF's quality after 15 seconds of training	**15 min**	2.6 s	24 MB	26.0 dB
Gaussian splatting	half a million colored 3D Gaussians	134 images per second on real scenes, close to the best quality	31 min	**6 ms**	118 MB	**28.2 dB**

On my globe, Gaussian splatting gave the best images and the only live view, Instant-NGP trained fastest, and NeRF stored the scene in the least space and held up best where few photos looked.

Mildenhall et al. 2020, Table 1 (Realistic Synthetic 360°: NeRF 31.01 dB, Neural Volumes 26.05 dB); Müller et al. 2022, Table 2 (31.4 dB after 15 s); Kerbl et al. 2023, Table 1 (Mip-NeRF 360 dataset, 27.2 against Mip-NeRF 360's 27.7 dB). My globe: 32 held-out photos; one image at 1080 × 607; the best of the three in each column in red.

References

The three papers

Mildenhall, Srinivasan, Tancik, Barron, Ramamoorthi & Ng 2020. *NeRF: representing scenes as neural radiance fields for view synthesis.* ECCV. arXiv:2003.08934.

Müller, Evans, Schied & Keller 2022. *Instant neural graphics primitives with a multiresolution hash encoding.* SIGGRAPH. arXiv:2201.05989.

Kerbl, Kopanas, Leimkühler & Drettakis 2023. *3D Gaussian splatting for real-time radiance field rendering.* SIGGRAPH. arXiv:2308.04079.

Methods compared in the three papers

Sitzmann, Zollhöfer & Wetzstein 2019. *Scene representation networks: continuous 3D-structure-aware neural scene representations* (SRN). NeurIPS.

Lombardi et al. 2019. *Neural volumes: learning dynamic renderable volumes from images.* SIGGRAPH.

Mildenhall et al. 2019. *Local light field fusion: practical view synthesis with prescriptive sampling guidelines* (LLFF). SIGGRAPH.

Sitzmann et al. 2019. *DeepVoxels: learning persistent 3D feature embeddings.* CVPR.

Liu et al. 2020. *Neural sparse voxel fields* (NSVF). NeurIPS.

Fridovich-Keil, Yu et al. 2022. *Plenoxels: radiance fields without neural networks.* CVPR.

Barron et al. 2021. *Mip-NeRF: a multiscale representation for anti-aliasing neural radiance fields.* ICCV.

Barron et al. 2022. *Mip-NeRF 360: unbounded anti-aliased neural radiance fields.* CVPR.

Methods and tools used

Levoy 1990. *Efficient ray tracing of volume data.* ACM Transactions on Graphics.

Park et al. 2019. *DeepSDF: learning continuous signed distance functions for shape representation.* CVPR.

Rahaman et al. 2019. *On the spectral bias of neural networks.* ICML.

Tancik et al. 2020. *Fourier features let networks learn high frequency functions.* NeurIPS.

Schönberger & Frahm 2016. *Structure-from-motion revisited* (COLMAP). CVPR.

Ravi et al. 2024. *SAM 2: segment anything in images and videos.* arXiv:2408.00714.

Ye et al. 2025. *gsplat: an open-source library for Gaussian splatting.* JMLR.

Datasets and metrics

Hedman et al. 2018. *Deep blending for free-viewpoint image-based rendering.* SIGGRAPH Asia.

Wang, Bovik, Sheikh & Simoncelli 2004. *Image quality assessment: from error visibility to structural similarity* (SSIM). IEEE TIP.

Zhang, Isola, Efros, Shechtman & Wang 2018. *The unreasonable effectiveness of deep features as a perceptual metric* (LPIPS). CVPR.

Other follow-up work

Yu, Ye, Tancik & Kanazawa 2021. *pixelNeRF: neural radiance fields from one or few images.* CVPR.

Martin-Brualla et al. 2021. *NeRF in the wild: neural radiance fields for unconstrained photo collections.* CVPR.

Zha, Zhang & Li 2022. *NAF: neural attenuation fields for sparse-view CBCT reconstruction.* MICCAI.